Lecture 4: Transformers: Architecture, Computation, and Language Modeling

Attention lets a model read: at each position it compares a query with the key of every position it may see, and takes a weighted average of those positions' values, with more weight on the better matches. A translator built from this read alone, Lecture 3's toy, reads the source once for every word it writes and does nothing else. It fails in four ways, each answered by a part of the Transformer:

1. it cannot tell where a word stands, so it confuses word orders and, once a word recurs in its translation, repeats itself forever (**position vectors**, §2);
2. it sees each source word without its neighbors (**self-attention**, §3, in a **stack of blocks**, §4);
3. it knows nothing of what it has written beyond the word it just wrote (**masked self-attention** over the translation, §3);
4. one weighted average per word can gather only one kind of information and transform none of it (**several heads**, and a multilayer perceptron, an **MLP**, at each position, §4).

The Transformer is what it takes to turn that read into a model that translates, and then into a model of language. This lecture builds it from the toy, one part at a time, then weighs what it costs, what it does well, and where it falls short.

Order and context. The toy's read depends on which words it reads but not on where they stand. Two sentences with the same words in a different order give the same reads, and two positions of the translation that follow the same word ask the same question and get the same answer, which is why the toy loops. Adding to each word's vector a vector that marks its position gives the model the order of the words. A word's meaning also depends on its neighbors, yet its vector, looked up in a table, is the same in every sentence. So the positions of a sentence should read one another and add what they read to their own vectors: every source word reads the whole source, and every word of the translation reads the words before it. This is **self-attention**: for a whole sentence, a few matrix products and a **mask** that hides the later positions, so that training computes all positions at once.

A deep stack. One read is one weighted average. Several reads side by side, called **heads**, gather several kinds of information, and a small network applied at every position, the **MLP**, computes with what they gather; together they make a **block**. Stacking blocks lets later reads use what earlier ones found, but a deep stack is hard to train: the size of the vectors drifts as the weights change, and when each layer replaces its input, the layers above can lose what the layers below computed. **Layer normalization** rescales each vector to a fixed size, and **residual connections** add each layer's output to its input instead of replacing it, so information and gradients pass through every layer. With a training recipe, the translator becomes the Transformer of 2017.

One model for text. Most text comes without a translation. Without its encoder, the Transformer's decoder predicts each token of a text from the tokens before it: a language model in the sense of Lecture 2, and the design of nearly every large language model since.

Strengths, limitations, and variants. Training costs about six arithmetic operations per weight per token, and generating text needs a memory of keys and values that grows with the text. Reading every earlier position in one step is the design's strength. Its prices are work and memory that grow with the context, and the same amount of computation for every token, however hard the token is to predict. Recent variants answer the first by keeping a state of

fixed size, as recurrent networks did, and the second by applying one block several times in a loop.

The lecture has three parts:

1. building the Transformer (§§1–7), from the toy translator to the encoder–decoder of 2017, one part at a time;
2. the language model (§§8–9): the decoder alone, and positions as rotations;
3. strengths, limitations, and variants (§§10–11): every weight and its cost; what the design does well and where it falls short; two families of variants.

1. A translator built from attention alone

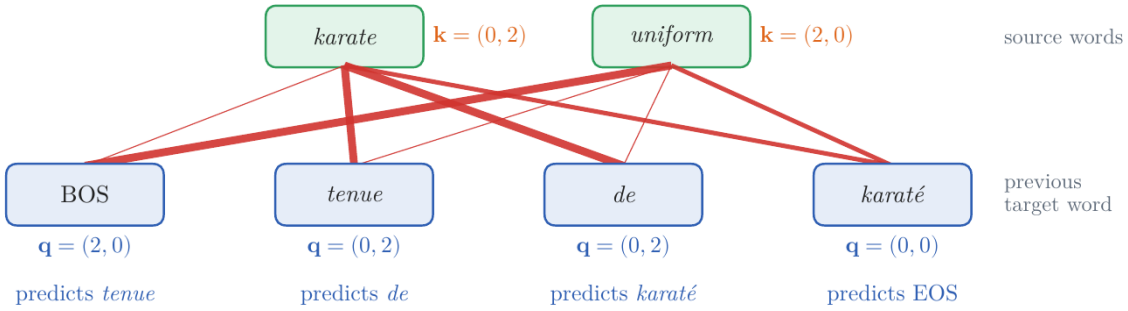


Figure 1. Lecture 3's toy translator: one read per target word, and nothing else. On *karate uniform*. Each target position (bottom) reads the source words (top) with a query computed from the previous target word alone; the width of each line is the weight. No line joins two target positions or two source words.

The toy of Lecture 3 (Section 9), drawn in Figure 1, translates a source sentence $x_1, \dots, x_n$ into a target sentence $y_1, \dots, y_m$, one target word at a time, with y_0 the start symbol BOS. As in Lecture 3, vectors are rows, and $[\mathbf{a}; \mathbf{b}]$ places two vectors side by side. Each source word x_j has a key $\mathbf{k}_j = E_K(x_j)$ and a value $\mathbf{v}_j = E_V(x_j)$, looked up in two tables, and each target word has a query vector, from a table E_Q, and a word vector, from a table E. Recall that the **softmax** of a row of numbers $\mathbf{z} = (z_1, \dots, z_n)$ is the row of positive weights that sum to one,

$$\text{softmax}(\mathbf{z})_j = \frac{\exp(z_j)}{\sum_{k=1}^{n} \exp(z_k)}.$$

At target position t, the toy

1. takes its query from the previous target word, $\mathbf{q}_t = E_Q(y_{t-1})$;
2. scores every source word, $e_{tj} = \mathbf{q}_t \mathbf{k}_j^\top$, turns the scores into weights, $\alpha_t = \text{softmax}(e_{t1}, \dots, e_{tn})$, and reads $\mathbf{c}_t = \sum_j \alpha_{tj} \mathbf{v}_j$;
3. predicts the next target word from the previous one and the read, $q(y_t \mid y_{<t}, \mathbf{x}) = \text{softmax}([E(y_{t-1}); \mathbf{c}_t]\, U)$, a distribution over the target vocabulary.

Its read is the core of the Transformer of Vaswani et al. (2017), drawn in Figure 2.

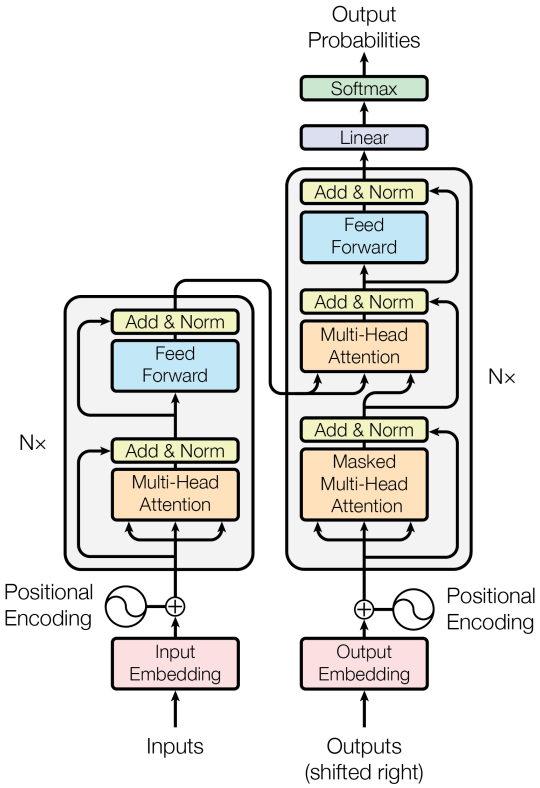


Figure 2. The Transformer, as drawn by Vaswani et al. (2017, their Fig. 1). Reproduced from *Attention Is All You Need* (arXiv:1706.03762v7). The encoder (left) reads the source and the decoder (right) writes the translation; each is a stack of N identical layers of multi-head attention and a feed-forward network, each followed by "Add & Norm", and positional encodings are added to the embeddings at the bottom.

The toy, trained

Every translator in §§1–7 is trained in the same way, on the data of Lecture 3: the 29,000 English captions of Multi30k with their French translations, and the same vocabularies (the 5,894 English and 6,430 French words seen at least twice, and four special tokens). Each is trained for 3,000 steps on batches of 64 caption pairs of similar length, with vectors of $d = 192$ numbers, and with Adam and a learning rate that warms up and then decays, as in the Transformer paper (§7). Two numbers measure each translator: the **cross-entropy** on the 1,014 validation captions, the average negative log-probability, in nats, of each French word of the reference given the English caption and the reference's earlier words (Lecture 2, Section 3); and **BLEU** on the 1,000 test captions, translated greedily, one most probable word at a time (Lecture 3, Section 2). Lecture 3's LSTM encoder–decoders reached 19.5 BLEU without attention and 49.2 with it, at cross-entropies of 2.01 and 1.20. Each translator here is trained once, so small differences between two of them may not survive another random seed.

The toy reaches a validation cross-entropy of 3.04 and a test BLEU of 1.7, far below even Lecture 3's translator without attention. The toy has 7,212,578 weights: four tables of 192 numbers per word and the output layer U, of $384 \times 6{,}434$ with a bias. Its translations show two of these failures, the word order and the loop (Figure 3). It translates *a dog is chasing a man .* and *a man is chasing a dog .* identically, as *un homme est assis sur un homme est assis sur un homme …*: its reads depend on the source words as a set. And it repeats itself without end on longer captions: for Lecture 3's karate caption, *a girl in karate uniform breaking a stick with a front kick .*, it writes *un bâton dans un bâton dans un bâton dans un bâton dans un bâton dans*

un bâton dans … Its query depends only on the previous French word, so each time it writes *un*, it makes the same read and predicts the same next word. Nothing tells it where it is in the sentence, or what it has written beyond the last word.

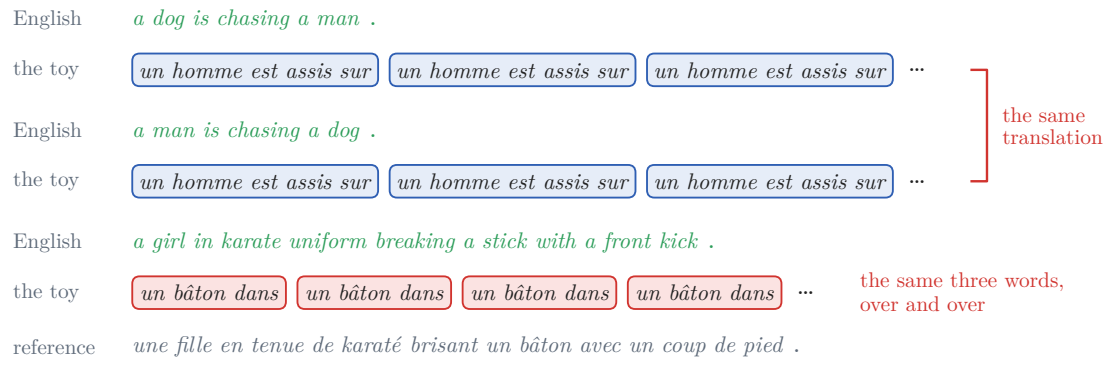


Figure 3. The toy cannot tell word orders apart, and it repeats itself without end. What the toy writes, greedily, for two sentences with the same words in a different order, and for Lecture 3's karate caption. Both chase sentences get the same translation, *un homme est assis sur* over and over; the karate caption gets *un bâton dans* over and over. The reference translation is in gray.

2. Word order: positions

The toy fails in two ways that have one cause: nothing in it depends on where a word stands. On the source side, a read depends on the positions it reads only through their keys and values (it "sees a set", Lecture 3, Section 6), and the toy's keys and values depend on the English words alone, so two captions with the same words in a different order give the same reads. On the target side, its query depends on the previous French word alone, so two target positions that follow the same word read the same thing wherever they are, and once a word recurs, the translation loops. A vector that marks each position, added on both sides, answers both: the source's keys and values then depend on where each English word stands, and every target position gets a query of its own.

A vector for each word, and maps to queries, keys, and values

Recall from Lecture 3 (Section 7) the three roles in a read: a target position's **query** says what it is looking for, a source position's **key** what it can be found by, and its **value** what it passes on when it is read. The toy looks each of them up in a table of its own.

Information about a position has to be added to something, and the toy has no vector for a source word that a position vector could be added to, only its key and its value, looked up in two tables. A table lookup is a matrix product: if $\mathbf{e}_x$ is the one-hot vector of word x, a row with a 1 in place x and zeros elsewhere, then $E_K(x) = \mathbf{e}_x E_K$. So give each word one vector, from one table per language, E_{src} and E_{tgt}; add a vector $\mathbf{p}_t$ that depends only on the position; and compute the query, the key, and the value from the sum by three learned $d \times d$ matrices. Writing $\mathbf{h}_j$ for the vector at source position j and $\mathbf{h}_t$ for the vector at target position t (as in the toy, j counts source positions and t target positions),

$$\mathbf{h}_j = E_{\text{src}}(x_j) + \mathbf{p}_j, \qquad \mathbf{h}_t = E_{\text{tgt}}(y_{t-1}) + \mathbf{p}_t, \qquad \mathbf{q}_t = \mathbf{h}_t W_Q, \quad \mathbf{k}_j = \mathbf{h}_j W_K, \quad \mathbf{v}_j = \mathbf{h}_j W_V.$$

The query, the key, and the value are now learned linear maps of one vector per position, which carries both the word and its position (Figure 4). Without the position vectors this is the

toy with its tables factored, $E_K = E_{\text{src}} W_K$: the same kind of read, with fewer parameters. The scores are also divided by $\sqrt{d}$, $e_{tj} = \mathbf{q}_t \mathbf{k}_j^\top / \sqrt{d}$, which keeps them in a range where the softmax can still learn (§3). The output layer reads $[\mathbf{h}_t; \mathbf{c}_t]$, as before.

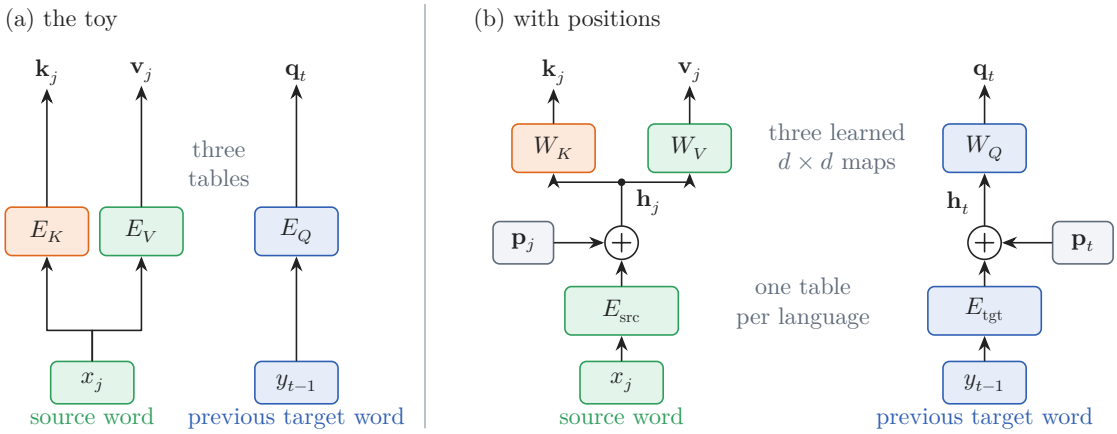


Figure 4. To give a word a position, give it one vector; its query, key, and value become maps of that vector. (a) The toy looks up a source word's key and value in two tables, E_K and E_V, and the previous target word's query in a third, E_Q. (b) With positions, each word has one vector, from one table per language, E_{src} or E_{tgt}, to which the vector of its position is added ($\oplus$); three learned $d \times d$ maps compute the key and the value of source position j from $\mathbf{h}_j$, and the query of target position t from $\mathbf{h}_t$.

What a position vector should be

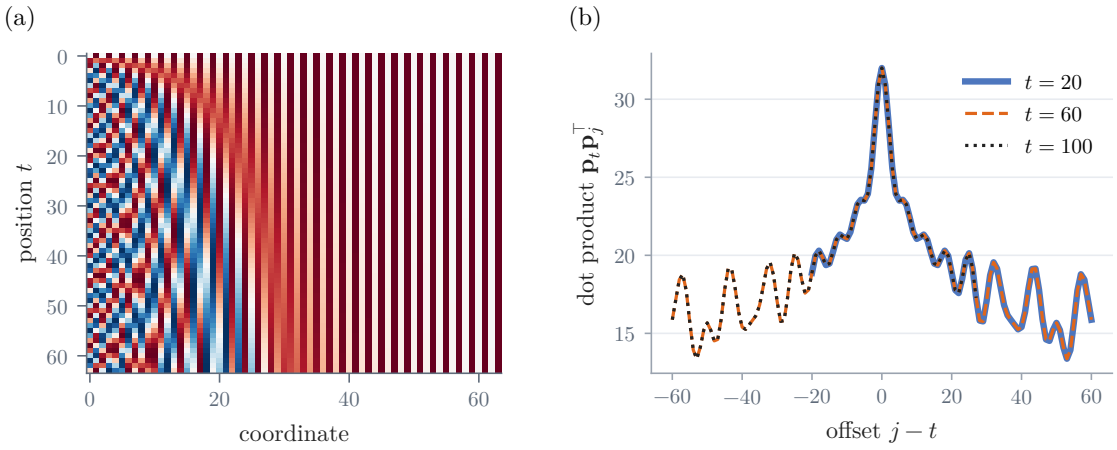


Figure 5. Sinusoidal positions encode offsets as rotations. (a) The position vectors of Vaswani et al. (2017) for the first 64 positions with $d = 64$: each row is one position and each column one coordinate, from -1 (blue) to 1 (red); the first coordinates oscillate quickly, the last ones slowly. (b) The dot product $\mathbf{p}_t \mathbf{p}_j^\top$ of the vectors of two positions, against their offset $j - t$, from three starting positions t: the curves coincide, because the dot product depends only on the offset, and it is largest at offset 0.

The same position vectors $\mathbf{p}_1, \mathbf{p}_2, \ldots$ serve both sides: source position j adds $\mathbf{p}_j$ to the vector of the English word x_j, and target position t adds $\mathbf{p}_t$ to the vector of the French word before it, y_{t-1}. They should be:

1. **distinct**: each position gets a different vector, so that a read of the source can tell the third English word from the fifth, and two target positions that follow the same French word ask different questions;
2. **fixed**: position 3 gets the same vector in every sentence, so that what the model learns about a third word carries over from caption to caption;
3. **bounded**: the vectors stay the same size at every position, so that at late positions they do not drown the word's vector;
4. **defined for every position**, on either side, including positions beyond the longest sentence seen in training;
5. **relative**: moving from position t to $t+u$ changes the vector in the same way for every t, so that "two words back" looks the same from every position of the source or of the target, and a map learned for it at one position works at all of them.

Vectors learned for each position, one row of a table per position up to the longest training sentence, meet the first three requirements but not the last two. Vaswani et al. (2017) chose fixed vectors of sines and cosines at geometrically spaced frequencies, for $i = 0, \ldots, d/2 - 1$:

$$p_{t,2i} = \sin(t\,\omega_i), \qquad p_{t,2i+1} = \cos(t\,\omega_i), \qquad \omega_i = 10000^{-2i/d}.$$

Each pair of coordinates is a point on a circle that turns at its own rate as t grows, from one radian per position for $i = 0$ to one turn in about 57,000 positions for the last pair when $d = 192$, like a clock with many hands (Figure 5a). The coordinates stay between -1 and 1 at every position. Moving from t to $t+u$ turns every pair by the angle $u\,\omega_i$, a rotation that depends on u but not on t: in the paper's words, "for any fixed offset k, PE_{pos+k} can be represented as a linear function of PE_{pos}" (their k is u here, and PE_{pos} is $\mathbf{p}_t$), which the authors hoped would let the model "easily learn to attend by relative positions". So the dot product of two position vectors, $\mathbf{p}_t \mathbf{p}_j^\top = \sum_i \cos((j-t)\,\omega_i)$, depends only on their offset (Figure 5b; Appendix A.2).[1]

The paper gives no deeper reason for this design than two hopes: that the offset property would let the model attend by relative positions, and that sinusoids would let it handle sentences longer than any seen in training. They also cost no parameters and serve sentences of any length, on the source side and the target side alike. The rest, sines and cosines at geometrically spaced frequencies with a base of 10,000, is a choice that meets the requirements rather than one that follows from them: the clock's fast hands tell neighboring positions apart, and its slow hands distant ones. The deeper reason appears when the positions are put into the scores instead of the vectors: asking a score to depend only on the offset forces rotations (§9).

What positions change

With positions and the learned maps, the validation cross-entropy falls from 3.04 to 1.42 and BLEU rises from 1.7 to 39.3. Both failures are gone (Figure 6): the two orders of the chase now get two translations, and the karate caption no longer loops.

[1] The token vectors of §§2–6 start with entries of standard deviation $1/\sqrt{d}$, about one unit long, while a sinusoidal vector is about $\sqrt{d/2} \approx 10$ units long. The translators of §§2–6 multiply the sinusoids by $1/\sqrt{d}$. The paper instead multiplies the token vectors by $\sqrt{d}$ (§7).

English *a dog is chasing a man .*
with positions *un chien court après un homme court .*
English *a man is chasing a dog .*
with positions *un homme court après un chien .*
two orders, two translations
English *a girl in karate uniform breaking a stick with a front kick .*
with positions *une fille en uniforme de karaté avec un bâton .*
uniforme, word for word; stops before the kick
reference *une fille en tenue de karaté brisant un bâton avec un coup de pied .*

Figure 6. With positions, two word orders get two translations, and the loop is gone. Greedy output of the translator with positions and the learned maps, for the sentences of Figure 3; the reference in gray. The karate caption ends early, and *uniforme* translates *uniform* word for word (red).

The reads show why (Figure 7). In the toy, the query depends only on the previous French word, so the rows that follow the same word are identical; with positions, each French position has a query of its own, and the reads move along the caption as the translation advances. Learned position vectors instead of sinusoids gave 42.9 BLEU in a single run, against 39.3; Vaswani et al. (2017) found nearly identical results, 25.7 against 25.8 BLEU, and preferred sinusoids for the second of their hopes.

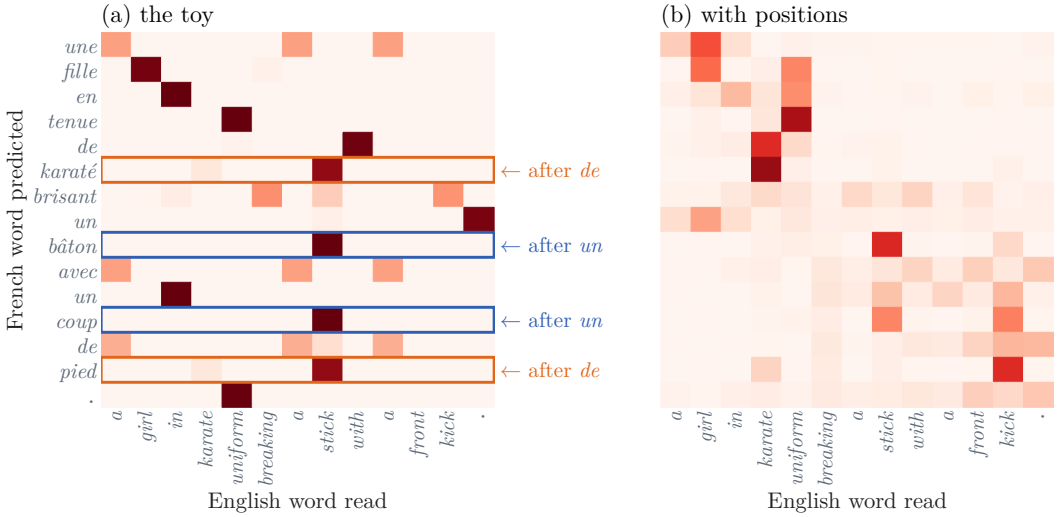


Figure 7. Without positions, rows that follow the same French word read the same; with positions, the reads follow the caption. The cross-attention weights on Lecture 3's karate caption, with the reference translation supplied word by word: each row is the read made to predict one French word, each column one English word, and each row sums to one (darker is more weight). (a) The toy of §1: its query depends only on the previous French word, so the rows predicted after *de* (*karaté*, *pied*) are identical, and so are those predicted after *un* (*bâton*, *coup*); each pair is outlined in its own color. (b) With positions and the learned maps.

The weights are now one table per language, 2,367,744 numbers, the three maps, $3d^2 =$ 110,592, and the output layer, 2,477,090: 4,955,426 in all, since sinusoidal positions have no weights.

Two gaps remain, and the karate translation shows both (Figure 6). *Uniforme* translates *uniform* word for word, where Lecture 3's translators wrote *tenue*, the French for a karate uniform: the model sees each English word without its neighbors. And the translation stops after *un bâton*, without *breaking* or *a front kick*: each French position knows its place and the previous word, but not what the earlier positions have read or written.

3. Context: self-attention

The toy's other two gaps have one remedy. Each English word should be represented in the context of its caption, and each French position should know what the translation has said so far. Both need the positions of one sentence to read one another: attention turned on a single sequence, introduced in Lecture 3 (Section 10). This section builds that read, works one head by hand and then in matrix form, explains the division by $\sqrt{d_k}$, and shows how a mask lets one pass compute every prediction of a sentence.

Reading one's own sentence

Self-attention lets every position of a sentence read the other positions of the same sentence. Write the sentence's vectors, one per position, as the rows $\mathbf{h}_1, \dots, \mathbf{h}_T$ of a matrix $X \in \mathbb{R}^{T\times d}$, where T is the length of the sentence. In the first layer each $\mathbf{h}_t$ is a word's vector plus its position's (§2). In the encoder, every English position may read every other. In the decoder, position t holds the French word written before it, y_{t-1} (with $y_0 = \text{BOS}$), and predicts y_t, so it may read only itself and the positions before it: the translation so far. From here on, t and j both count positions of the one sentence being read, t the position that reads and j a position it reads.

Each position's read $\mathbf{c}_t$ is a weighted average of the vectors it may read. The question is how to choose the weights, and two simple choices show what is needed.

Equal weights. Give every readable position the same weight: in the decoder, $\mathbf{c}_t = \frac{1}{t}\sum_{j\le t}\mathbf{h}_j$ (Figure 8a). In matrix form, $C = AX$ with $A = \text{softmax}(M)$, the softmax taken along each row, where the **mask** M has $M_{tj} = 0$ for $j \le t$ and $M_{tj} = -\infty$ for $j > t$: a score of $-\infty$ gets the weight $e^{-\infty} = 0$, and the other positions share the weight equally. The encoder leaves out M. This read ignores what the words are: every earlier word counts the same.

Weights from similarity. Score each readable position by the dot product of its vector with the reader's, $A = \text{softmax}(XX^\top + M)$ (Figure 8b). Now the weights depend on the words, but in the wrong way. A vector is usually most similar to itself, so each word mostly reads itself (0.82 of the weight on average). And the dot product is symmetric, while the relations worth reading are not: a pronoun looks for the noun it refers to, but the noun has no reason to look for the pronoun.

Both failures have the remedy of §2: let each position look for something with a query, be found through a key, and pass on a value, each computed from its vector by a learned map. **Self-attention** computes all three from the same sentence:

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V, \qquad C = \text{softmax}\Big(\frac{QK^\top}{\sqrt{d_k}} + M\Big)V,$$

with $W_Q, W_K \in \mathbb{R}^{d\times d_k}$ and $W_V \in \mathbb{R}^{d\times d_v}$: queries and keys have d_k coordinates and values d_v (in §2, $d_k = d_v = d$). Row t of Q is the query $\mathbf{q}_t$ of position t, row j of K the key $\mathbf{k}_j$, and row j of V the value $\mathbf{v}_j$. The score of position j for position t,

$$e_{tj} = \frac{\mathbf{q}_t\mathbf{k}_j^\top}{\sqrt{d_k}} = \frac{\mathbf{h}_t W_Q W_K^\top \mathbf{h}_j^\top}{\sqrt{d_k}},$$

is the bilinear score of Lecture 3 (Section 7) with its matrix factored as $W_QW_K^\top$. That matrix need not be symmetric, so e_{tj} and e_{jt} can differ: a pronoun can find its noun without the noun finding the pronoun. And what a position passes on, $\mathbf{v}_j$, can differ from what it is found by, $\mathbf{k}_j$.

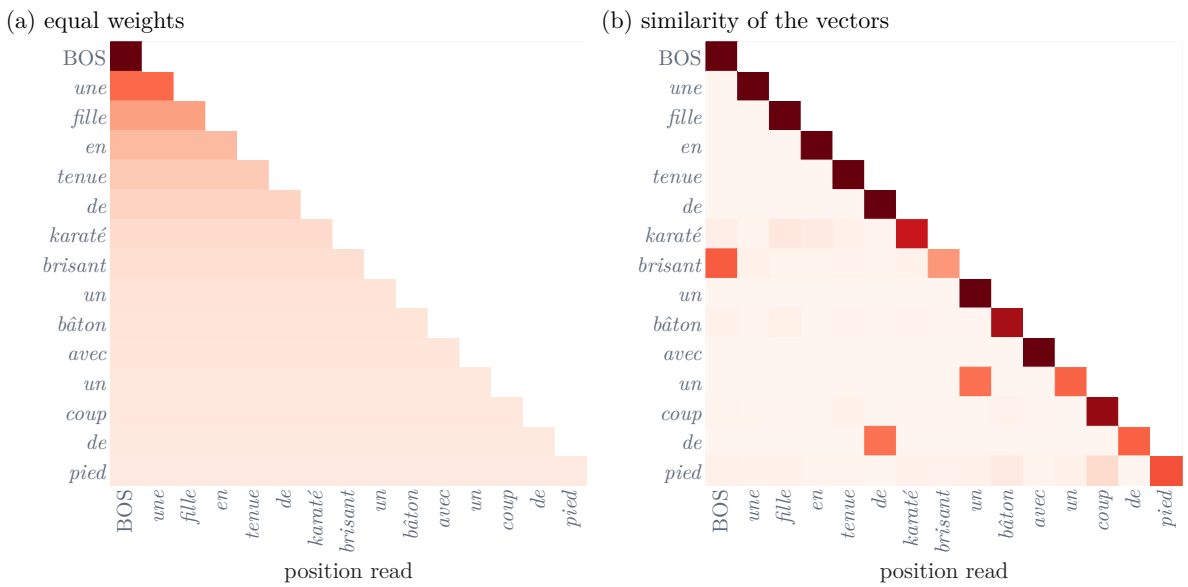


Figure 8. Equal weights ignore the words; raw similarity mostly reads the word itself. The first two ways for each position to read the positions up to it, on the reference translation of the karate caption as the decoder of §2's translator holds it: at each position, the vector of the French word before the one it predicts plus the position's vector, with BOS at the first position. Each row is one reading position, labeled with its word, and shows how it divides one unit of weight over the positions up to its own (columns), darker for more weight. (a) Equal weights: each position averages its prefix. (b) Weights from the dot products of the vectors: each word reads mostly itself, and a repeated word (*un*, *de*) also reads its earlier copy. Neither choice reads along the relations between words.

One head by hand

Take the five tokens *lily saw tom . she*, read by one masked head, with token vectors whose five coordinates are features chosen by hand: *name*, *female*, *action*, *end of sentence*, and *pronoun*. The maps to two dimensions are chosen by hand as well: a key says (*is a name*, *is female*); a query says (*wants a name*, *wants a female*), so that an action looks for a name and a pronoun for a female name; and a value says (*is a character*, *lily rather than tom*), with $+1$ for *lily* and -1 for *tom*.[2] The rows of Q, K, and V are then:

	lily	***saw***	***tom***	***.***	***she***
query **q**	$(0, 0)$	$(2, 0)$	$(0, 0)$	$(0, 0)$	$(2, 2)$
key **k**	$(1, 1)$	$(0, 0)$	$(1, 0)$	$(0, 0)$	$(0, 1)$
value **v**	$(1, 1)$	$(0, 0)$	$(1, -1)$	$(0, 0)$	$(0, 0)$

[2]The matrices have one row per feature. W_K has rows $(1, 0)$, $(0, 1)$, 0, 0, 0; W_Q has rows 0, 0, $(2, 0)$, 0, $(2, 2)$; and W_V has rows $(1, -1)$, $(0, 2)$, 0, 0, $(0, -2)$, so that the value of *lily* is $(1, -1) + (0, 2) = (1, 1)$ and the value of *she* is $(0, 2) + (0, -2) = 0$.

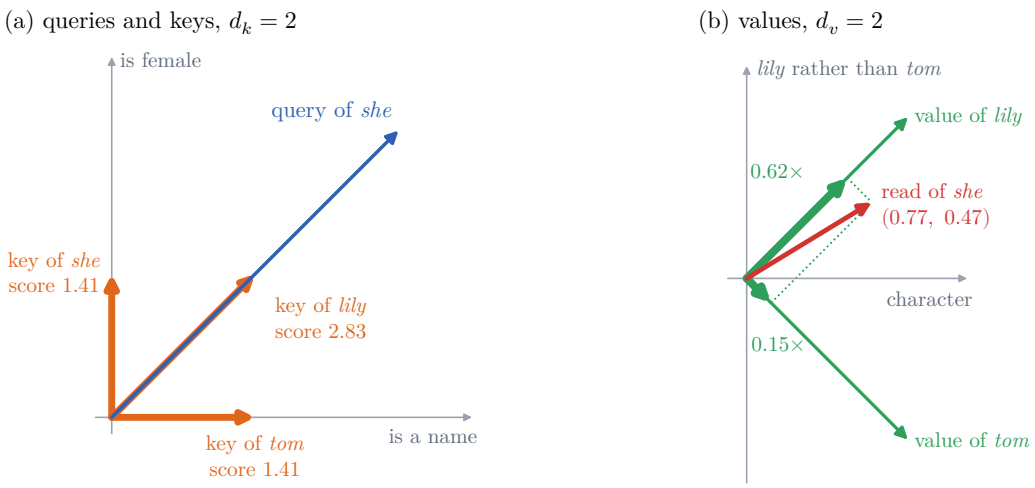


Figure 9. A query gives its highest score to the key that reaches furthest along its direction, and the read is a weighted sum of value arrows. The head of *lily saw tom . she*, drawn as arrows, as Sanderson (2024b) draws attention. (a) The space of queries and keys: the query of *she* (blue) and the three keys that are not zero (orange), each with its score, the dot product with the query divided by $\sqrt{2}$. (b) The space of values: the values of *lily* and *tom* (thin green), the same values multiplied by their weights in the row of *she* (thick green), and their sum, the read of *she* (red).

The query of *she*, $(2, 2)$, scores the five keys $(4, 0, 2, 0, 2)/\sqrt{2} = (2.83, 0, 1.41, 0, 1.41)$, and the softmax gives the weights $\alpha_{she} = (0.62, 0.04, 0.15, 0.04, 0.15)$ over *lily*, *saw*, *tom*, the full stop, and *she*. The read is the weighted sum of the values, $\mathbf{c}_{she} = 0.62\,(1, 1) + 0.15\,(1, -1) = (0.77, 0.47)$: mostly characters of the story, and pointing to *lily* rather than *tom* (Figure 9). *Tom* stands nearer to *she*, but *she* reads *lily*, because the key of *lily* matches both coordinates of its query: the read goes by content, not position. And *lily* is found by what kind of word it is but passes on which character it is; with raw similarity instead, *she* would put 0.56 of its weight on itself and 0.21 on *lily*. In a trained model the features are directions learned from data, spread over many coordinates, and rarely this clean.

The matrix form

Written for all positions at once, the read is

$$C = \text{softmax}\Big(\frac{QK^\top}{\sqrt{d_k}} + M\Big)V,$$

with the softmax taken along each row. One head computes it in five steps (Figure 10):

1. **Project.** Multiply X ($T \times d$) by the three matrices: Q and K are $T \times d_k$, V is $T \times d_v$.
2. **Score.** $S = QK^\top/\sqrt{d_k}$ is $T \times T$: entry (t, j) compares the query of position t with the key of position j.
3. **Mask.** Add M, which sets the entries above the diagonal to $-\infty$.
4. **Weigh.** Take the softmax of each row: A is $T \times T$, each row nonnegative and summing to one, with zeros above the diagonal.
5. **Read.** $C = AV$ is $T \times d_v$: row t is the weighted average of the values that position t may read.

Only steps 2 and 5 mix positions; the projections act on each row separately. Every step is a matrix product or an operation on each entry or each row, which parallel hardware does fastest; for this reason Vaswani et al. (2017) chose the dot product over the additive score of Bahdanau et al. (2015).

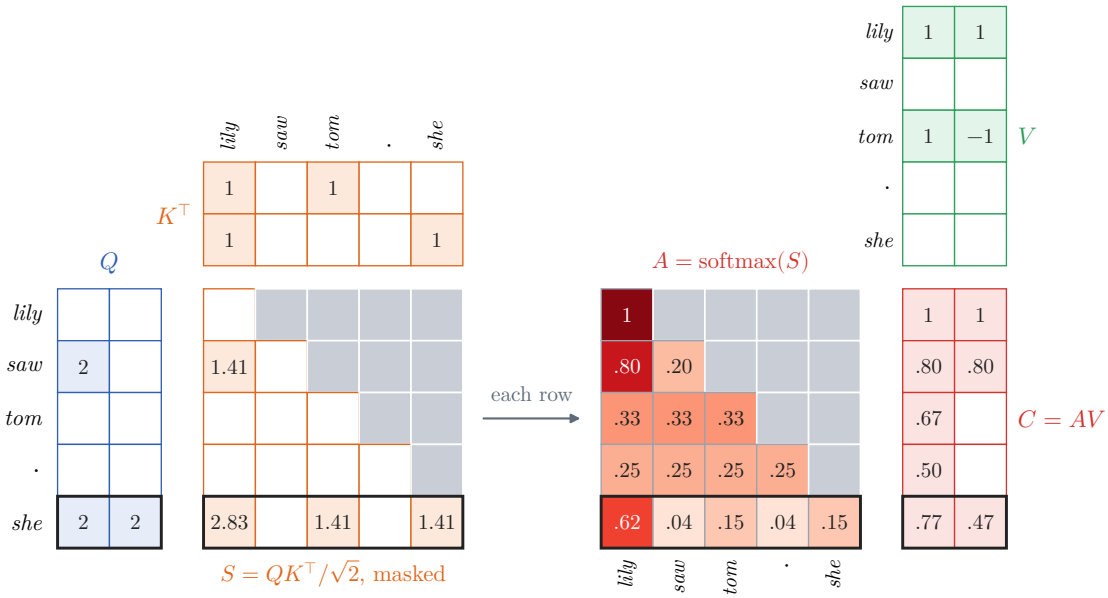


Figure 10. One head, as matrices: each query is scored against the keys up to it, and the weights average the values. The example above for all five positions at once, with $d_k = d_v = 2$. Each square is one number, its value printed in it; a blank square is zero. The queries (blue) and the keys (orange) give the scores $S = QK^\top/\sqrt{d_k}$, one row per reading position and one column per position read; the mask turns the entries above the diagonal into $-\infty$ (gray). The softmax of each row gives the weights A (red), and $C = AV$ averages the values (green) with them. The row of *she* is outlined.

Why divide by $\sqrt{d_k}$

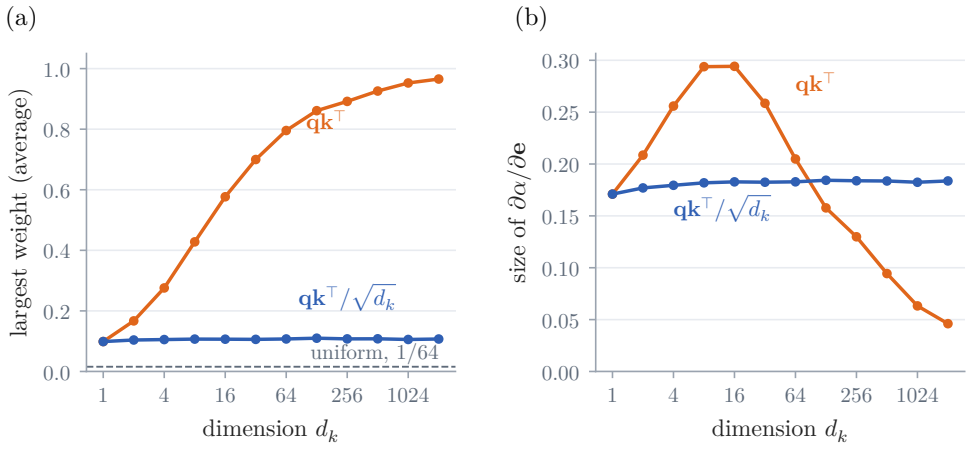


Figure 11. Without the division, attention collapses onto one key as d_k grows. Random queries and keys with independent standard normal coordinates: one query and 64 keys, 2,000 trials (500 for $d_k \geq 1{,}024$). (a) The largest of the 64 attention weights, on average, against the dimension d_k, with the scores divided by $\sqrt{d_k}$ (blue) and without (orange). (b) The size of the softmax's derivative (the Frobenius norm of the matrix $\partial\alpha/\partial\mathbf{e}$) on the same draws. Unscaled, the largest weight approaches 1, and the derivative peaks near $d_k = 16$ and then falls; scaled, both stay flat.

Without the division, a softmax over many keys puts almost all its weight on one key, and then stops learning. Suppose that the d_k coordinates of a query and a key are independent, with mean 0 and variance 1, as they roughly are in a freshly initialized network. Their dot product $\mathbf{qk}^\top = \sum_{i=1}^{d_k} q_i k_i$ is a sum of d_k independent terms, each with mean 0 and variance 1, so it has variance d_k. With $d_k = 256$, the scores of different keys spread with a standard deviation of 16, and a softmax over 64 such scores puts, on average, almost 90% of its weight on the largest (Figure 11a). When one weight is close to 1 and the others close to 0, the derivative of every weight with respect to every score is close to zero (Appendix A.1), and the gradient that would move the weights vanishes. Without the division, the softmax's derivative first grows with d_k and then falls; at $d_k = 2{,}048$ it is a quarter of its value with the division (Figure 11b).

Dividing the scores by $\sqrt{d_k}$ makes their variance 1 for every d_k, so the weights stay spread over several keys at the start of training, and the derivative is the same for every d_k. It is the temperature of Lecture 2 (Section 6), fixed at $\sqrt{d_k}$. The argument assumes independent coordinates of unit variance, as at initialization; training can grow the scores again (Appendix E).[3] Vaswani et al. (2017) gave the variance argument in a footnote and wrote that they "suspect" the saturation. The evidence they cited was the large comparison of translation models of Lecture 3 (Section 7), in which unscaled dot-product scores fell behind additive scores as the dimension grew, to 18.2 against 22.1 BLEU at 1,024 dimensions (Britz et al., 2017).

The causal mask: every prediction in one pass

In training, the decoder sees the whole reference translation at once (teacher forcing, Lecture 3, Section 2): its input at position t is the reference's previous word y_{t-1}, and its output there is a distribution over y_t. If position t could read position $t+1$, whose input is y_t, it would read the very word it must predict. The causal mask M forbids exactly this, and it is enough, for three reasons:

[3] Once training has aligned queries with the keys they should match, a dot product of d_k terms grows like d_k rather than $\sqrt{d_k}$, and the rules for scaling hyperparameters with width known as μP divide by d_k instead (Yang et al., 2021).

1. **Within a position.** The query, key, and value of position t are computed from $\mathbf{h}_t$ alone: each is $\mathbf{h}_t$ times a matrix.
2. **Within an attention layer.** The weight α_{tj} is zero for $j > t$, and for $j \le t$ it depends only on $\mathbf{q}_t$ and $\mathbf{k}_1, \dots, \mathbf{k}_t$, since the softmax normalizes over $j \le t$. So $\mathbf{c}_t = \sum_{j \le t} \alpha_{tj} \mathbf{v}_j$ depends only on $\mathbf{h}_1, \dots, \mathbf{h}_t$.
3. **Through the stack.** Every other operation of the model acts on each position separately, or reads the source, which is known in full; so by induction over the layers, the output at position t depends only on the source and on $y_0, \dots, y_{t-1}$.

So one forward pass over a caption pair predicts every French word at once, each from its own prefix, and one backward pass trains them all, where a recurrent decoder needed one step per word (Lecture 3, Section 5). The encoder needs no causal mask, since the whole English caption is known before the translation starts.

Sentences of any length

The length T is not built into the model. The maps W_Q, W_K, and W_V act on one position at a time, so the same weights serve a sentence of any length; only the $T \times T$ matrices of scores and weights change size with T. For the source, T is the caption's length n; for the target, it is the number m of positions BOS, $y_1, \dots, y_{m-1}$; and the decoder's read of the source compares m queries with n keys, an $m \times n$ matrix.

To train on many caption pairs at once, the captions of a batch are padded with a filler token to the longest one in the batch, so that they stack into one array. A **padding mask** adds $-\infty$ to the score of every padding position, so that no real word reads the filler, and the loss ignores the predictions at padding positions (Figure 12). In generation, the decoder still writes one word at a time (Lecture 3, Section 2), and its T grows by one with each word.

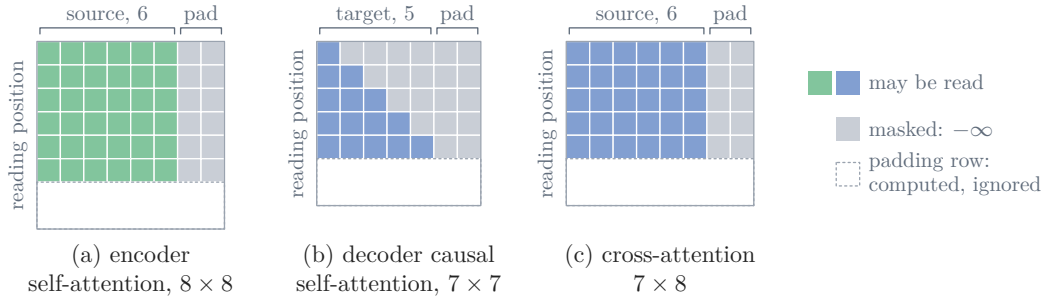


Figure 12. Padding and masks fit sentences of different lengths into fixed arrays. One caption pair from a batch whose longest source has 8 words and longest target 7 positions; this source has 6 words and this target 5 positions (BOS and four words), and the rest is padding. Each grid shows which positions (columns) a position (row) may read (colored) and which are masked with a score of $-\infty$ (gray). (a) Encoder self-attention, 8×8: every source word reads every source word, but not the padding. (b) Decoder self-attention, 7×7: each target position reads itself and the positions before it. (c) The decoder's read of the source, 7×8: target rows and source columns, so not square. The rows of padding positions are computed and then ignored.

Adding the read to the vector

The toy's output layer kept the previous word beside the read, $[E(y_{t-1}); \mathbf{c}_t]$, so that a read could add to what the position already had without replacing it. With self-attention the same holds inside the model: each read is **added** to the vector it was computed from,

$$\mathbf{h}_t \leftarrow \mathbf{h}_t + \mathbf{c}_t,$$

in the encoder's self-attention, the decoder's self-attention, and the decoder's read of the source. The output layer then computes the **logits** of the French words, one number per word, from the decoder's final vector at position t alone, $q(y_t \mid y_{<t}, \mathbf{x}) = \text{softmax}(\mathbf{h}_t U)$. Without the addition, a word's vector would be replaced by an average of values, and the word itself would survive only as far as it happened to read itself. The addition is a **residual connection**; in a deep stack it is indispensable (§6).

So the vector of a word now depends on the words around it. The word vectors of Lecture 2 (Section 5) gave *bank* one vector for the river and for the money. After self-attention, the read of *bank* in *the river bank* adds a change toward what *river* passes on, and in *money in the bank* toward what *money* passes on, so its two uses get different vectors (Figure 13).

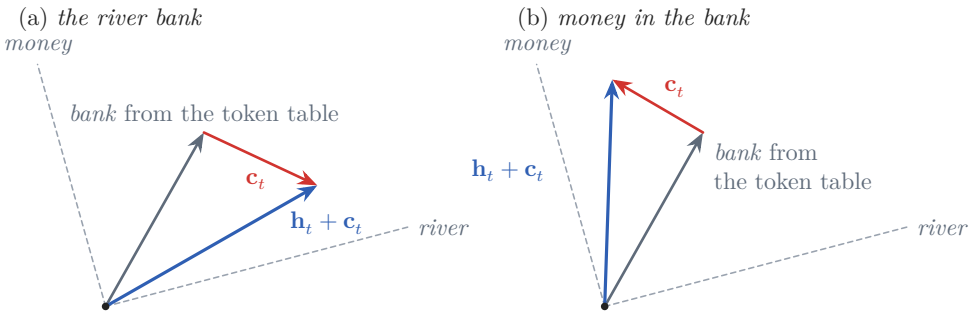


Figure 13. Self-attention moves a word's vector toward its context. A schematic, after Sanderson (2024b), in two of the d directions of the space. The token table gives *bank* one vector (gray), the same in every sentence. (a) In *the river bank*, the read $\mathbf{c}_t$ (red) adds a change toward *river*; (b) in *money in the bank*, toward *money*. The sums $\mathbf{h}_t + \mathbf{c}_t$ (blue) differ, so the two uses of *bank* get different vectors.

With one layer of self-attention in the encoder and one of causal self-attention in the decoder, the validation cross-entropy falls from 1.42 to 1.40 and BLEU rises from 39.3 to 41.5. The translator has fewer weights than in §2, 3,941,282, because the output layer now reads one vector instead of two: the three attention layers have $3d^2$ weights each, and U shrinks to $192 \times 6{,}434$ with its bias. The karate caption shows what the new read buys (Figure 14): *uniform* now becomes *tenue*, read together with *karate*, though the end of the caption is still garbled. The gain is smaller than that of positions. Each position still makes one read per layer, of one kind, and a read can only average what it reads.

English	*a girl in karate uniform breaking a stick with a front kick .*	
the toy (§1)	*un bâton dans un bâton dans un bâton dans …*	
positions (§2)	*une fille en uniforme de karaté avec un bâton .*	
self-attention (§3)	*une fille en tenue de karaté avec un ballon de pied devant un coup .*	*tenue*: *uniform* read with *karate*
reference	*une fille en tenue de karaté brisant un bâton avec un coup de pied .*	

Figure 14. With self-attention, *uniform* is read in its context. Greedy output for the karate caption from the toy (§1), with positions (§2), and with one layer of self-attention on each side (§3); the reference in gray. With self-attention, *uniform* becomes *tenue* (blue), the French for a karate uniform, but the rest of the caption is still wrong.

4. Heads, the MLP, and a stack of blocks

After §3, each position makes one read per attention layer, and a read is a weighted average of values that are linear maps of the vectors. Two limits follow: a position can gather only one kind of information per read, and nothing in the model transforms what it gathers, except the averaging itself. Several heads remove the first limit, and a small network applied at every position, the MLP, the second. Together with attention, they make the Transformer's **block**, and a stack of blocks lets later reads use what earlier ones found.

Several heads

One head computes one weighted average per position, so a position can gather only one kind of information in one read. A French position may need both the English word it translates and the word that word depends on, *uniform* and *karate* for *tenue*; one set of weights cannot pick out both without blurring them. **Multi-head attention** runs H reads side by side, each with its own projections to $d_k = d_v = d/H$ coordinates, and combines them (Figure 15).

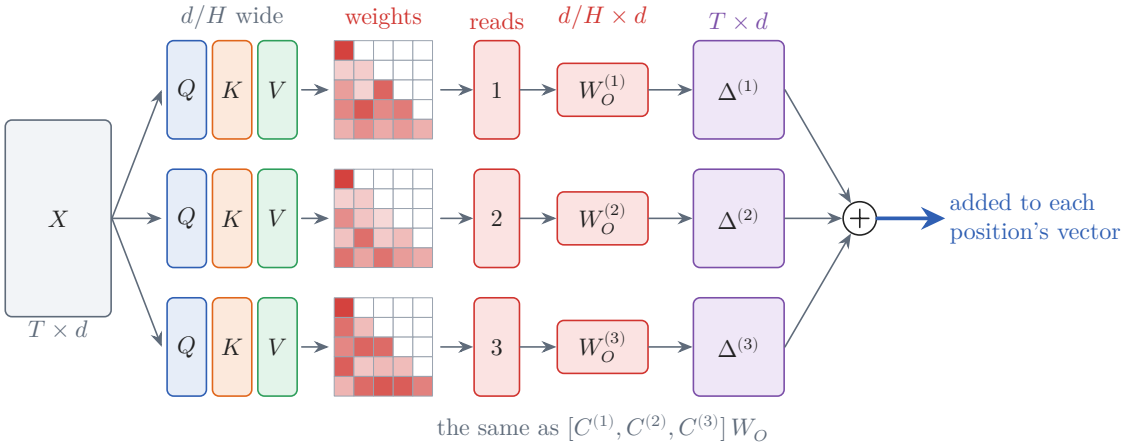


Figure 15. Each head computes its own weights and proposes its own change, and the changes are added. Multi-head attention for three heads, drawn schematically. Each head projects the T vectors of size d to queries (blue), keys (orange), and values (green) of size d/H, computes its own $T \times T$ matrix of weights (red), and averages its values with them. Its slice of the output matrix, $W_O^{(h)}$ (red), maps its read back to size d, and the three changes $\Delta^{(h)} = C^{(h)} W_O^{(h)}$ (purple) are added: the same as placing the reads side by side and multiplying by W_O. The sum is then added to the vector that each position came in with (§3).

For $h = 1, \dots, H$,

$$C^{(h)} = \text{softmax}\Big(\frac{(XW_Q^{(h)})(XW_K^{(h)})^\top}{\sqrt{d/H}} + M\Big) XW_V^{(h)}, \qquad \text{MultiHead}(X) = [C^{(1)}, \dots, C^{(H)}]\, W_O.$$

The H reads of d/H columns each are placed side by side, giving a $T \times d$ matrix, and the output matrix $W_O \in \mathbb{R}^{d \times d}$ mixes them. Two counts show that several heads cost no more than one head of full width with an output map:

1. **Parameters.** The projections of all the heads together are three $d \times d$ matrices, cut into H slices, and W_O adds a fourth: $4d^2$ weights, whatever H is, since more heads means narrower ones.
2. **Work.** The scores of one head cost $T^2\, d/H$ multiplications, and of all H heads $T^2 d$, the same as one head of full width.

What heads buy is several different weightings at the same cost; in the words of Vaswani et al. (2017), "with a single attention head, averaging inhibits this." In their ablations, one head of 512 dimensions scored 0.9 BLEU below eight heads of 64 on the development set, and 32 heads of 16 dimensions 0.4 below. In trained models heads specialize: in BERT, a large Transformer encoder of English text, some heads put most of their weight on the previous token, some on the next, and some on punctuation (Clark et al., 2019). In translation models many heads can be removed with little loss, 38 of the 48 of one encoder for 0.15 BLEU (Voita et al., 2019), but not those that read the source (Michel et al., 2019).

The heads' reads need not be placed side by side at all. Cut W_O into H slices of d/H rows, $W_O^{(h)}$, one for each head; then

$$[C^{(1)}, \dots, C^{(H)}]\, W_O = \sum_{h=1}^{H} C^{(h)} W_O^{(h)} = \sum_{h=1}^{H} A^{(h)} X\, W_V^{(h)} W_O^{(h)},$$

where $A^{(h)}$ is the matrix of weights of head h. Each head computes its own change to every position's vector, of size d, and the changes are added (Figure 15). A head chooses where to read with $W_Q^{(h)}$ and $W_K^{(h)}$; what it writes, from the vectors it reads, is fixed by the product $W_V^{(h)} W_O^{(h)}$, a $d \times d$ matrix of rank at most d/H: a map down to d/H coordinates and back up, so everything a head writes lies in a subspace of at most d/H dimensions. Elhage et al. (2021) study trained models by treating each head as a separate term of this sum.

Why an MLP: communication and computation

Attention gathers information; it does not compute with it. A read is a weighted average of values, and each value is a linear map of a vector, so what a position receives is a weighted sum of linear images of the vectors it read. The only nonlinearity in attention is in its weights, which decide where to read, not what to make of what is read. But a translator needs new features made from old ones. To write *tenue*, a French position needs a feature that marks the pair *karate uniform*: *uniform* alone is *uniforme*, and *karate* alone is *karaté*. Such a feature is an AND, present when both words are, and an AND is not a linear function of its inputs: a linear function that is 0 for neither word, 0 for *karate* alone, and 0 for *uniform* alone is also 0 for both. Making it takes a threshold, a nonlinearity.

So attention is **communication**, or message passing, the only place where information moves between positions, and what a position does with what has arrived needs **computation**: a nonlinear function of its vector, applied at the position. After the attention, each block applies a network with one hidden layer to every position separately, with the same weights at every position and no access to any other position (Figure 16):

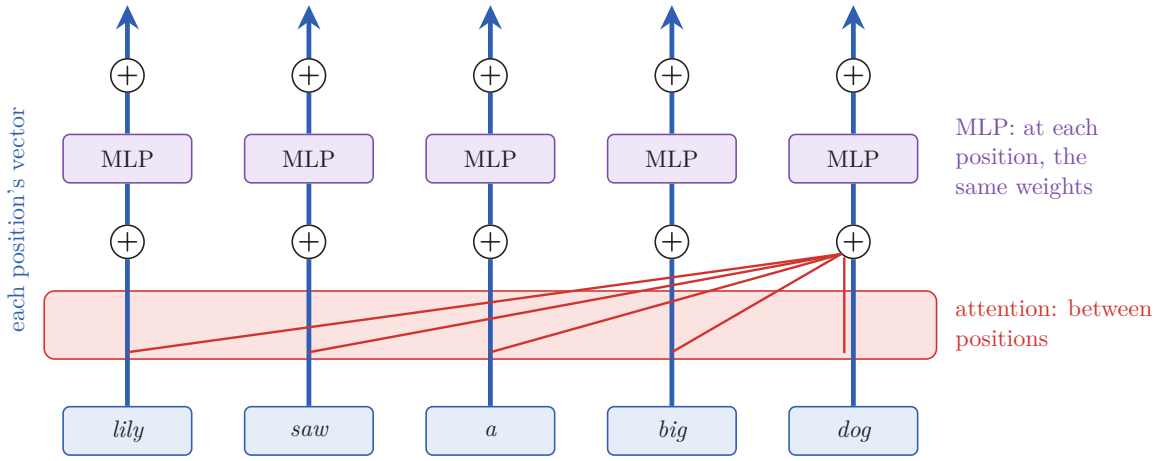


Figure 16. Attention moves information between positions; the MLP computes at each position. One block over five positions (columns). The attention sublayer (red) reads, at each position, the positions up to it, and adds what it reads to that position's vector; only the reads of the last position are drawn. The MLP (purple) is one small network, with the same weights, applied to each position separately: nothing passes between the columns. The blue lines carry each position's vector up through the block.

$$\mathrm{MLP}(\mathbf{h}) = \phi(\mathbf{h}W_1 + \mathbf{b}_1)\,W_2 + \mathbf{b}_2, \qquad W_1 \in \mathbb{R}^{d\times d_{\mathrm{ff}}}, \quad W_2 \in \mathbb{R}^{d_{\mathrm{ff}}\times d},$$

with a hidden layer four times as wide as the vector, $d_{\mathrm{ff}} = 4d$, and a nonlinearity ϕ: the ReLU, $\max(0,\cdot)$, in the model of Vaswani et al. (2017), and the GELU (Hendrycks and Gimpel, 2016), a smooth version of it, in later models. The paper calls it the position-wise feed-forward network; language-model code usually calls it the MLP. Its $8d^2$ weights are two thirds of an encoder block's.

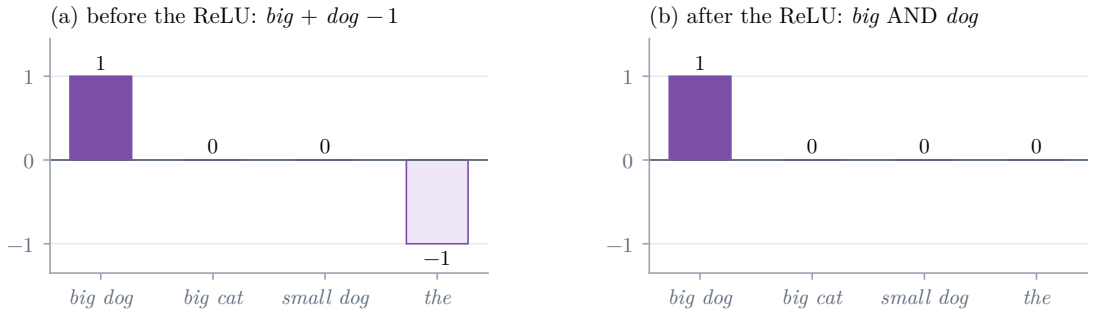


Figure 17. With a suitable negative bias, a hidden unit followed by a ReLU can compute an AND. A toy hidden unit j, after Sanderson (2024c): its column of W_1 has a 1 in a coordinate that marks *big* and a 1 in one that marks *dog*, and its bias is -1. The inputs have a 1 in the coordinates of the words they contain. (a) Before the ReLU, the unit's value is 1 for *big dog*, 0 when only one of the two words is present, and -1 when neither is. (b) After it, only *big dog* is positive.

The MLP can be read one hidden unit at a time. Write $\mathbf{w}_j$ for column j of W_1, as a row, and $\mathbf{u}_j$ for row j of W_2. Then

$$\mathrm{MLP}(\mathbf{h}) = \sum_{j=1}^{d_{\mathrm{ff}}} \phi(\mathbf{h}\mathbf{w}_j^\top + (\mathbf{b}_1)_j)\,\mathbf{u}_j + \mathbf{b}_2.$$

The dot product $\mathbf{h}\mathbf{w}_j^\top$ asks how far the vector points along $\mathbf{w}_j$, the bias sets a threshold, and the ReLU passes only what exceeds it; a column that points along two features, with a bias that only both together overcome, makes the unit an AND of the two (Figure 17), the kind of feature that *tenue* needs. The unit's value then scales $\mathbf{u}_j$, which is what the unit adds to the

vector (Sanderson, 2024c). The MLP thus behaves like a large key–value memory whose keys, the $\mathbf{w}_j$, and values, the $\mathbf{u}_j$, are parameters, and some individual hidden units of trained models respond to recognizable patterns in the text (Geva et al., 2021).[4] Sanderson (2024c) gives an example of the kind a large language model may compute: once attention has brought *Michael* to the position of *Jordan*, a unit whose column points along both *first name Michael* and *last name Jordan*, with a bias that only both together overcome, can add a row that points along *basketball*. The fact is stored in the weights and retrieved at the position.

The block, and a stack of blocks

A block is attention, then the MLP, each added to the vector it read (§3). With four heads in each attention layer and an MLP after the attention in the encoder and in the decoder, the validation cross-entropy falls from 1.40 to 1.26 and BLEU rises from 41.5 to 46.5, for two changes made together in one run. The model is now one block deep on each side. The weights added are an output map W_O for each of the three attentions, $3d^2 = 110{,}592$, and the two layers of the MLP in the encoder and in the decoder, 591,744: 4,643,618 in all.

With one block on each side, the encoder reads the caption once, and the decoder reads its translation once and the caption once. Later reads could use what earlier ones computed: a French position could read English vectors that already carry their context, and attend by what the layers below have found. Stacking three blocks on each side, each block reading the vectors the block below wrote, does this; in every decoder block, the read of the source takes its keys and values from the encoder's last block (Figure 18). The validation cross-entropy falls to 1.16 and BLEU rises to 49.3, narrowly past Lecture 3's LSTM with attention. Each further block adds, on the encoder's side, an attention and an MLP, and on the decoder's side two attentions and an MLP: 6,711,842 weights in all.

Nothing in a block controls the size of its vectors. Measure the size of a vector $\mathbf{h}$ by the root mean square of its d numbers, $\sqrt{\frac{1}{d}\sum_i h_i^2}$: a vector whose numbers are about ± 1 has size about 1, whatever d is. In the three-block stack, the decoder's vectors at the top had an average size of 6.7 after 50 training steps and 3.1 at the end, against 1.6 and 2.9 in the one-block translator, and the size changes as training changes the weights (Figure 19), so every block reads vectors whose size the blocks below have set.

[4] The $\mathbf{w}_j$ and the $\mathbf{u}_j$ are directions in a space of d dimensions, which has room for only d mutually perpendicular directions but for many more that are nearly perpendicular (Appendix A.4). A model can therefore represent more features than it has coordinates, at the price of some interference between features that are present together; Elhage et al. (2022) call this **superposition**. So a feature need not be a coordinate. Later work traced where trained language models store facts: in GPT-2, the MLPs of the middle layers, at the last word of a subject, matter most for recalling a fact about it (Meng et al., 2022), although editing those layers is not always the best way to change the fact (Hase et al., 2023).

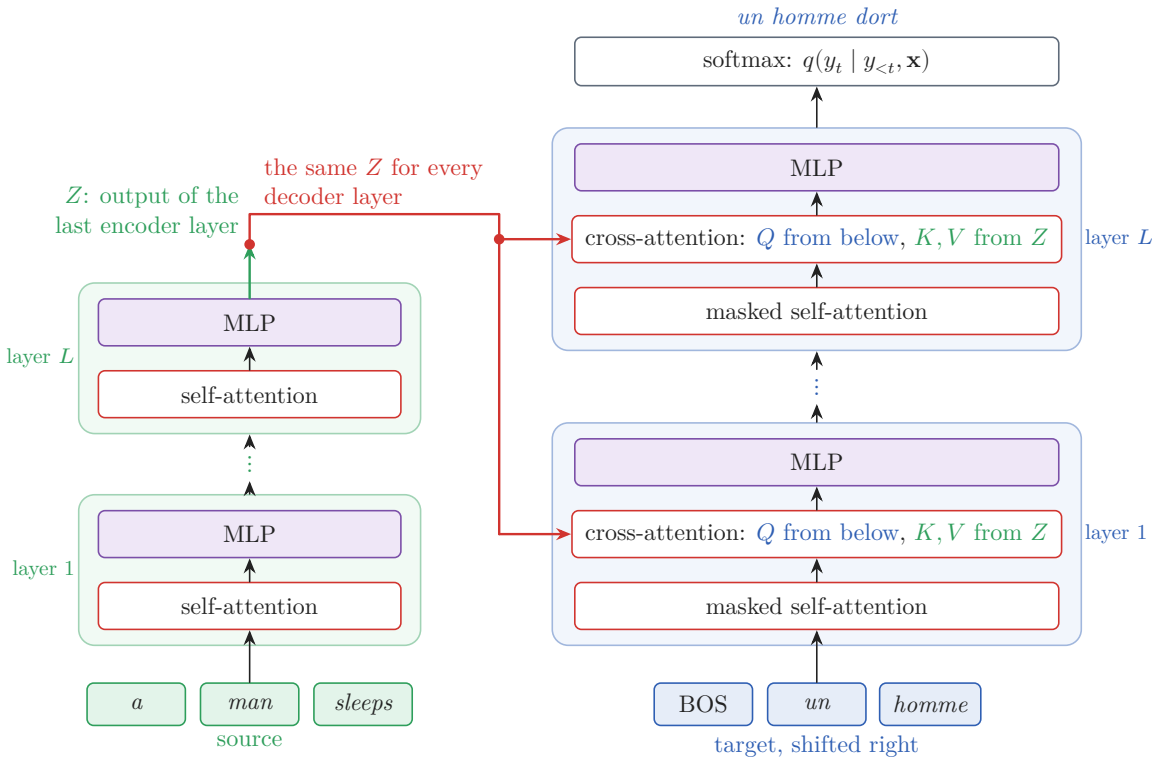


Figure 18. A stack of blocks on each side: three uses of attention, and where each takes its queries, keys, and values. The translator of this section, with $L = 3$, has the architecture of the Transformer of 2017 (§7). The encoder (green) applies L layers of self-attention and an MLP to the source; every source position reads every source position. The decoder (blue) applies L layers to the target shifted right, each with masked self-attention over the target positions up to its own, cross-attention, and an MLP. The output of the encoder's last layer, Z, is read by the cross-attention of every decoder layer (red): the same Z in layers 1 to L, which take their queries from the decoder layer below and their keys and values from Z. Normalizations and residual connections are not drawn.

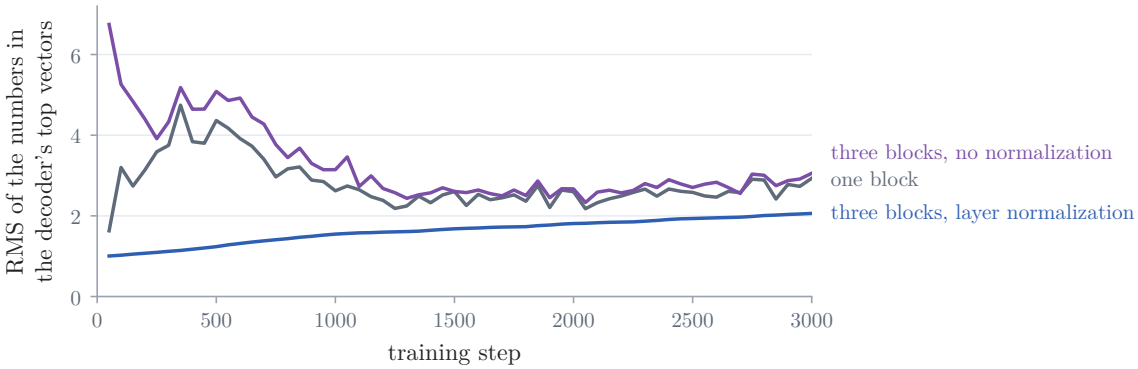


Figure 19. Without normalization, the size of the vectors is large and drifts; with layer normalization, it is set by the learned scale and grows slowly. The size of the decoder's vectors at the top (the root mean square of their numbers), every 50 steps of training, for one block (gray), three blocks without normalization (purple), and three blocks with the layer normalization of §5 (blue). With normalization, the top vector is the output of a normalization, so its size is set by the learned scale and shift.

5. Keeping the vectors in check: layer normalization

In the stack of §4, every sublayer adds its output to the vector it read, and nothing keeps the result at a fixed size. Its size (§4) depends on the weights, changes as training changes them, and differs from one position and one layer to the next. The size matters in three places. The scores are dot products of two maps of the vector, so they grow with the square of its size and push the softmax toward a single key, where it stops learning (§3). The hidden units of the MLP compare their inputs with thresholds, their biases, that do not scale with the input. And one learning rate has to suit every weight, while the effect of a step on a weight scales with the size of the vectors it multiplies.

Layer normalization

Layer normalization (Ba et al., 2016) rescales each vector by itself: it subtracts the mean of its d coordinates, divides by their standard deviation, and applies a learned scale γ and shift β to each coordinate,

$$\text{LayerNorm}(\mathbf{h}) = \gamma \odot \frac{\mathbf{h} - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta, \qquad \mu = \frac{1}{d}\sum_{i=1}^{d} h_i, \quad \sigma^2 = \frac{1}{d}\sum_{i=1}^{d}(h_i - \mu)^2,$$

with a small ϵ to avoid dividing by zero, and $\odot$ multiplying number by number. Before γ and β, the normalized vector has mean 0 and size 1, whatever the size of $\mathbf{h}$: the pattern of its numbers is kept, and only its center and scale change (Figure 20). The scale and shift then let the model choose another size and center for each coordinate, but as parameters that training sets, not as a side effect of the weights below. Because it uses only the vector itself, layer normalization computes the same thing in training and in use, for sentences of any length and batches of one, unlike batch normalization, which normalizes each coordinate over the examples of a batch and is the usual choice for images.

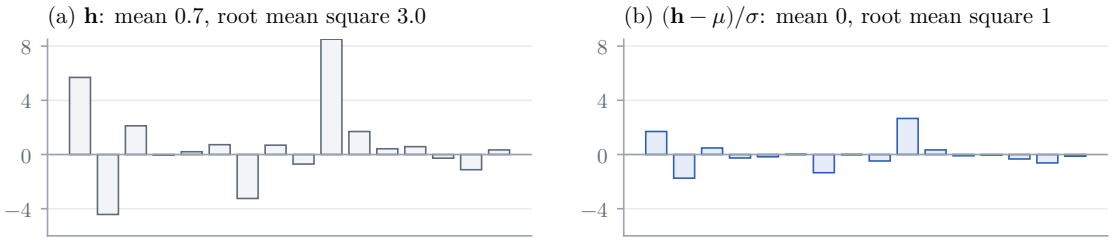


Figure 20. Layer normalization gives every vector the same mean and size, whatever it came in with. An illustrative vector of 16 numbers (a) before and (b) after the normalization, with $\gamma = 1$ and $\beta = 0$: its mean is subtracted and it is divided by its standard deviation, so the pattern of its numbers is kept and only its center and scale change.

Normalizing after each addition

The Transformer of 2017 normalized after each addition: every sublayer F, attention or MLP, updates the vector as

$$\mathbf{h} \leftarrow \text{LayerNorm}(\mathbf{h} + F(\mathbf{h})),$$

so every sublayer reads a vector whose size is set by γ and β, not by the weights below. With a normalization after each of the fifteen additions of the three-block translator, the validation cross-entropy falls from 1.16 to 1.08 and BLEU rises from 49.3 to 52.7, and the size of the vectors at the top grows smoothly from 1.0 to 2.1, set by the learned scale and shift (Figure 19).

Each normalization adds a scale and a shift of d numbers each, 5,760 parameters for the fifteen, for a total of 6,717,602.

Normalization does not explain why the stack trains at all; that depends on the additions of §3.

6. Depth: residual connections

The stack of §4 trains because every sublayer adds its output to its input, as the reads of §3 did. Trained with the additions of §3 removed, so that each sublayer's output replaces its input, three blocks on each side do not learn to translate, even with the normalizations of §5. Their validation cross-entropy ends at 4.64 and their BLEU at 0.0. They learn a little, since predicting every French word by its frequency alone gives 5.39, but they translate the chase as *chien chien chien …* and the karate caption as *un un un …*: one word, repeated.

Where residual connections came from

A deeper network can compute everything a shallower one does: its extra layers could pass their input on unchanged. Yet He et al. (2016a) found that plain deep networks for images trained worse as they grew deeper, and not by overfitting: "adding more layers to a suitably deep model leads to higher training error". Their networks normalized every layer (§5) and their gradients had healthy sizes, so they judged the difficulty "unlikely to be caused by vanishing gradients". Figure 21a repeats their comparison on scikit-learn's small set of handwritten digits, in single runs at one learning rate. The plain networks of 8 and 20 layers fit their training images, and deeper ones fit them worse: the one of 44 layers still misclassifies 60% of them after 3,000 steps, and 62.4% of the test images, against 2.2% for 8 layers: a failure far more severe than the few points He et al. measured on their image networks, but of the same kind.

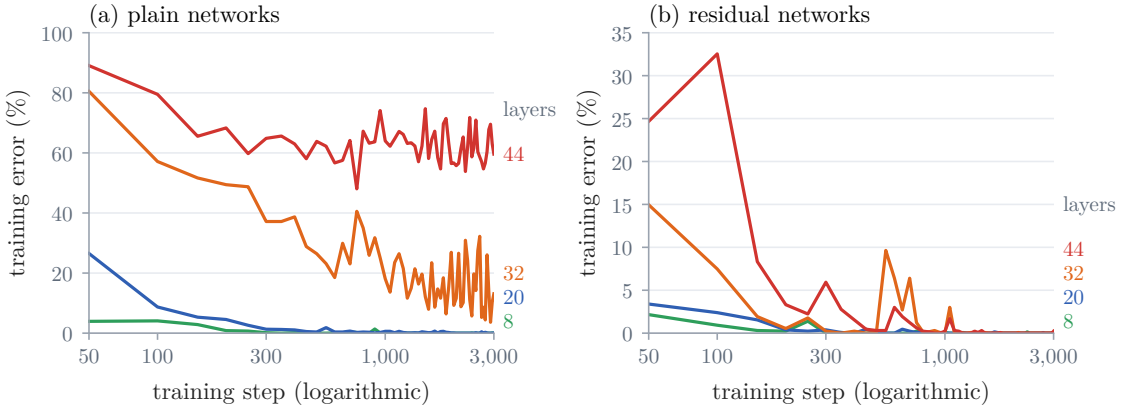


Figure 21. Deeper plain networks fit their training data worse; residual networks of every depth fit it. After He et al. (2016a, their Figs. 1 and 6), on scikit-learn's handwritten digits (8×8 pixels): networks of 8, 20, 32, and 44 fully connected layers (green, blue, orange, red), each hidden layer of 64 units followed by batch normalization and a ReLU, trained for 3,000 steps of 64 images by gradient descent with momentum. (a) Plain networks, each layer replacing its input. (b) Residual networks, in which every two layers add their output to their input, before the ReLU. The training error is the share of all 1,297 training images classified wrongly; the step axis is logarithmic, and panel b's error axis stops at 35%.

The addition of §3 comes from He et al.'s remedy: each block adds its output to its input,

$$\mathbf{h} \leftarrow \mathbf{h} + F(\mathbf{h}),$$

where F is the block's computation, so that it learns a change to its input instead of a replacement for it. If the best a block can do is nothing, it only has to drive F to zero. With this change, the networks of every depth fit their training images (Figure 21b); the residual network of 44 layers reaches a test error of 3.2%. In a stack of such blocks with nothing else between them, write $\mathbf{h}^{(i)}$ for a position's vector after i blocks. The vector after the last block is the vector after block ℓ plus everything the later blocks added, $\mathbf{h}^{(L)} = \mathbf{h}^{(\ell)} + \sum_{i=\ell}^{L-1} F_i(\mathbf{h}^{(i)})$, and the gradient of the loss $\mathcal{L}$ with respect to the earlier vector is

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(\ell)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(L)}} \Big(I + \frac{\partial}{\partial \mathbf{h}^{(\ell)}} \sum_{i=\ell}^{L-1} F_i(\mathbf{h}^{(i)}) \Big),$$

whose first term, the identity, passes the gradient at the top to every block unchanged (He et al., 2016b). This is the additive path of the LSTM's memory cell in Lecture 2 (Section 8), turned from the time axis to the depth axis.[5]

Residual connections in a stack of attention layers

Attention gives a second reason. A read is a weighted average over positions, and averages of averages become alike. For stacks of attention layers alone, without MLPs or residual connections, Dong et al. (2021) proved that the output converges, doubly exponentially fast in the depth, to a matrix whose rows are all the same, and that skip connections prevent this. Every position then carries nearly the same vector and predicts nearly the same distribution. The three-block translator without residual connections fits this account: in its last decoder layer, every head gives every word of the karate caption exactly the same weight, 1/13, and it writes one word over and over.

A smaller language model shows the same collapse. It predicts the next character of the children's stories of Lecture 2, with eight blocks of width 64, four heads each, and a context of 128 characters, and with its positions entered by rotating the queries and keys, a scheme described in §9. It was trained for 600 steps of 32 chunks of 128 characters, at a peak learning rate of 0.005 reached after a warmup of 200 steps, and scored in bits per character on the first 100,000 characters of held-out stories. Predicting each character by its frequency alone gives 4.22 bits (Lecture 2, Section 4). Without residual connections, the vectors at two different positions of a text have an average cosine similarity of 0.92 after the first block and 0.99 after the eighth at initialization, and the model never does better than those 4.22 bits; with them, it reaches 1.76 (Figure 22). The residual connection adds back to each position its own vector, which the averages would wash out.

[5] A residual network can also be read as many networks at once. Expanding the sum, a stack of n residual blocks contains 2^n paths from the input to the output, each skipping some blocks and passing through the others. Veit et al. (2016) found that in residual networks for images, most of the gradient flows along the short paths, and that deleting a single block at test time barely changed the error, while deleting a layer of an ordinary deep network made its accuracy fall to chance.

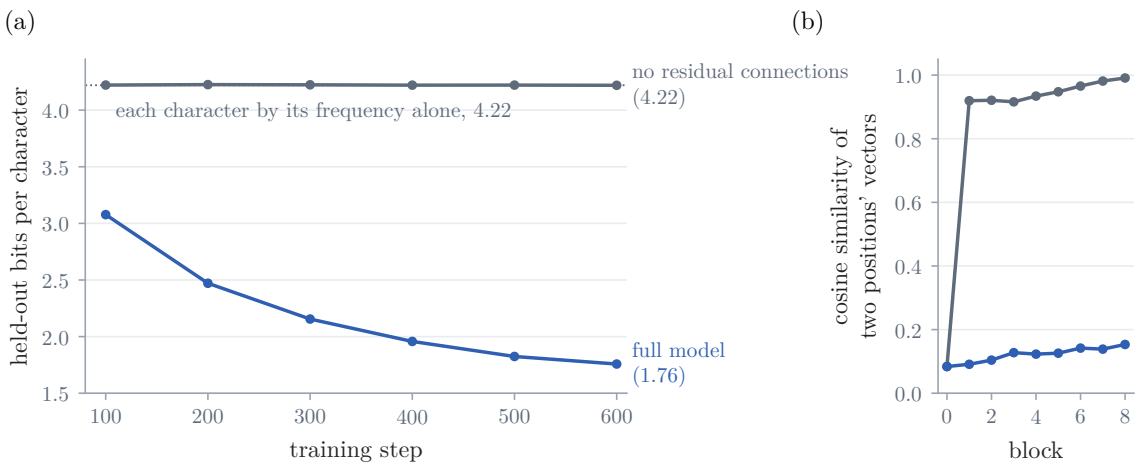


Figure 22. Without residual connections, the vectors at different positions become alike, and the model learns nothing beyond the frequencies of the characters. The eight-block character model, with residual connections (blue) and without (gray). (a) Held-out bits per character during training: with them, the model reaches 1.76; without them, it stays at 4.22, the cost of predicting each character by its frequency alone (dashed). (b) The average cosine similarity between the vectors at two different positions of a text, at initialization, after the token table (0) and after each block: without residual connections it is 0.92 after the first block and 0.99 after the eighth; with them it stays below 0.2.

The Transformer wraps each sublayer, attention or MLP, in a residual connection, and every sublayer and the token table produce vectors of the same size d, in the words of Vaswani et al. (2017), "to facilitate these residual connections". The additions of §3 are these connections: with them and the normalizations of §5, the three-block translator reaches 52.7 BLEU; without them, 0.0.

Pre-norm and post-norm

The normalization of §5 sits after each residual addition, $\mathbf{h} \leftarrow \text{LayerNorm}(\mathbf{h} + F(\mathbf{h}))$, as in the Transformer paper, an arrangement now called **post-norm**. GPT-2 and nearly every model since normalize the input of each sublayer instead (Figure 23), an arrangement called **pre-norm**, with one more normalization after the last block:

$$\mathbf{h} \leftarrow \mathbf{h} + F(\text{LayerNorm}(\mathbf{h})).$$

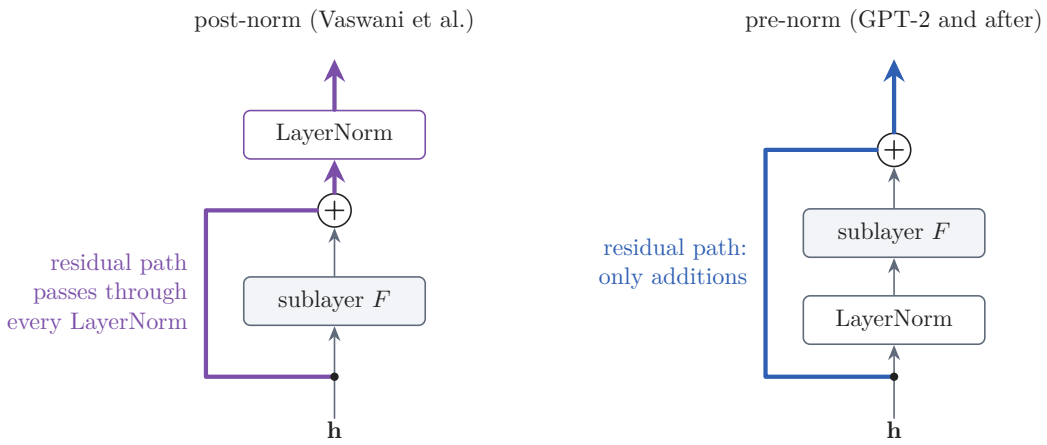


Figure 23. Post-norm puts a normalization on the residual path; pre-norm keeps it free. One sublayer F, attention or MLP, in each arrangement. In post-norm, as in the Transformer paper, the sum of the input and the sublayer's output is normalized, so the residual path (purple) passes through a normalization in every sublayer. In pre-norm, the sublayer reads a normalized copy of its input, and the residual path (blue) carries only additions, from the token vectors to the last block.

In pre-norm nothing but additions lies on the residual path, so the expansion of $\mathbf{h}^{(L)}$ above holds exactly and the identity carries the gradient from the top to every sublayer unchanged. In post-norm the gradient on the residual path is rescaled by every normalization it crosses.

In practice post-norm needs care at the start of training. Under simplifying assumptions, Xiong et al. (2020) showed that at initialization the gradients of post-norm's parameters near the output are large whatever the depth, while in pre-norm they shrink as $1/\sqrt{L}$, so a full learning rate from the first step makes post-norm's first updates too large. The Transformer paper used a **warmup**, a learning rate that rises from zero over the first steps of training (§7), and Xiong et al. showed that post-norm needs it: in their experiments, post-norm trained by Adam (§7) without warmup reached 8.5 BLEU on a translation task, against about 34 with warmup, while pre-norm trained without warmup about as well as post-norm with it. When post-norm does train, it can end slightly ahead: 27.6 against 26.8 BLEU on English–German in one comparison (Nguyen and Salazar, 2019).

The character model shows the same instability. The run above, with 1.76 bits, is pre-norm with the warmup at a peak learning rate of 0.005; the model was trained once more in each other combination of arrangement, warmup or none, and peak learning rate, 0.005 or 0.001. Figure 24b gives all eight results:

1. With the larger peak and no warmup, post-norm learns nothing beyond the frequencies of the characters: within a few steps its loss reaches the 4.22 bits of the frequencies and stays there (Figure 24a). Pre-norm trains at that rate without warmup, though better with it.
2. With warmup, post-norm trains too, and ends slightly ahead of pre-norm (1.70 against 1.76 bits per character).
3. At the smaller peak, both train with or without warmup, post-norm more slowly.

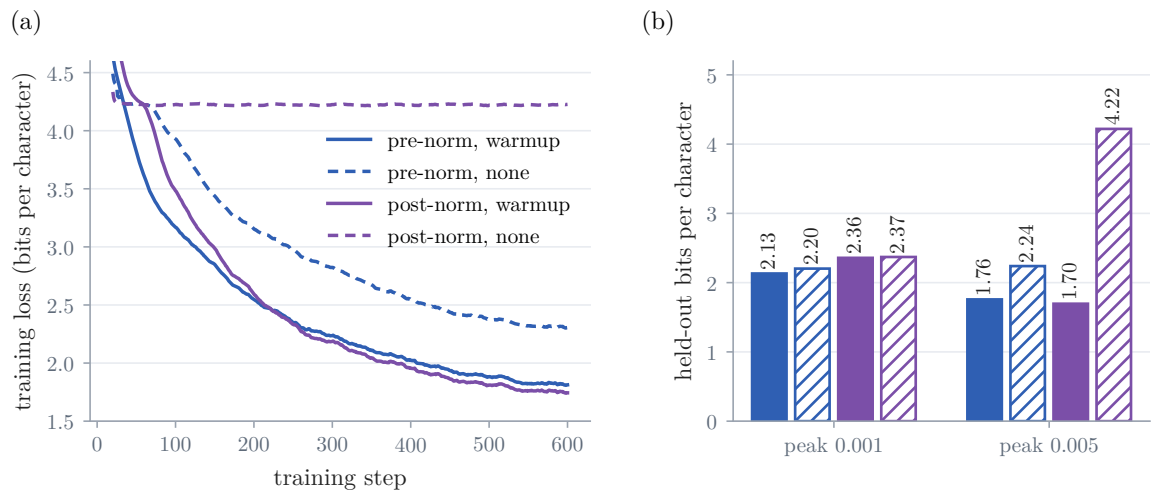


Figure 24. Without warmup, a large learning rate stops post-norm from learning. The character model of this section, trained for 600 steps in the two arrangements, with a warmup of 200 steps or none, then a cosine decay. (a) Training loss with a peak learning rate of 0.005, averaged over 20 steps. (b) Held-out bits per character at the end of training, for both peak learning rates: pre-norm blue, post-norm purple, with warmup filled, without hatched.

These single short runs show the instability, not which arrangement ends better after long training.

The residual stream

With residual connections, the vector at a position has a simple history. It starts as its token's vector, plus its position's if position vectors are added, and every sublayer reads it and adds something to it. Elhage et al. (2021) call this vector the **residual stream**: a shared workspace of d numbers at each position, the blue lines of Figure 16, which attention updates with information from other positions and the MLP with computation at the position. In pre-norm the vector at the top is exactly its starting vector plus the changes (Figure 25): for an attention sublayer, one change per head (§4) plus a bias; for an MLP, a weighted sum of the rows of W_2 (§4) plus a bias. Since only attention moves information between positions (§4), a model with L blocks moves it at most L times on any path to an output.

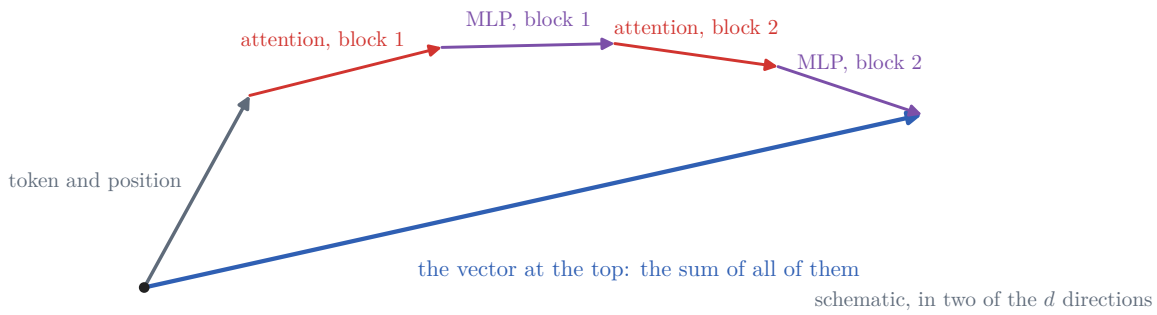


Figure 25. The vector at a position is the sum of what every sublayer added to it. A schematic, after Sanderson (2024a), in two of the d directions: the token's vector plus its position's (gray), then the change each attention (red) and each MLP (purple) adds, head to tail. In pre-norm nothing else acts on the stream, so the vector at the top (blue) is exactly their sum.

The stack has begun to fit its 29,000 training captions better than new ones. Over its last 250 steps, its validation cross-entropy stopped falling, from 1.07 to 1.08, while the loss on its training batches was below 0.5. And more than a third of its parameters sit in two tables

that each hold a vector for every French word, the input table and the output layer, learned separately.

7. The Transformer of 2017

With three blocks on each side, every part of Figure 2 is in place: the translator of §§4–6 has the architecture of Vaswani et al. (2017), at a smaller size. The paper adds two things that address the problems left at the end of §6: one table shared between the input and the output, and a training recipe, two parts of which keep a model from fitting its training data too closely.

One table for the input and the output

The output layer is a table of word vectors too. The logit of French word v is the dot product of the decoder's last vector at position t, $\mathbf{h}_t$, with column v of U, so U holds one vector for each word, and the decoder's last vector is pushed toward the vector of the word that should come next. The target table E_{tgt} also holds one vector for each word, and similar words get similar vectors in both. Inan et al. (2017) and Press and Wolf (2017) used one table for both, which made language models better as well as smaller: tied, a recurrent model of 51 million parameters scored a test perplexity of 74.3 on a standard benchmark, against 78.4 untied with 66 million (Press and Wolf).

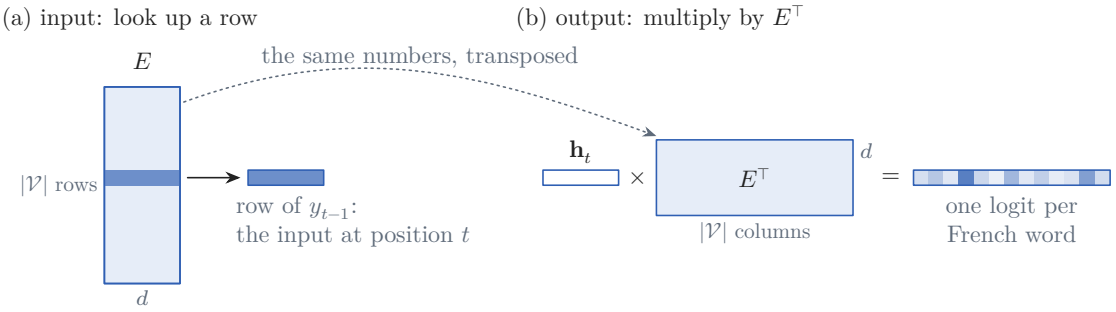


Figure 26. One table serves as the input and, transposed, as the output layer. The French table E has one row of d numbers for each word. (a) At the input, the row of y_{t-1} is the word's vector, to which the vector of position t is added. (b) At the output, the decoder's last vector $\mathbf{h}_t$ times $E^\top$ gives one logit for each French word.

Vaswani et al. shared one table three ways: as the encoder's input, as the decoder's input, and, transposed, as the decoder's output layer, $q(y_t \mid y_{<t}, \mathbf{x}) = \text{softmax}(\mathbf{h}_t E^\top)$ (Figure 26). That needs one vocabulary for both languages, which they had: for English–German, both were split by byte-pair encoding into pieces from one shared vocabulary of about 37,000 tokens. The translator here has two vocabularies, so only the French table can be shared, with the output layer, which removes U and its bias: 1,241,762 parameters. At the input the paper multiplies the table's vectors by $\sqrt{d}$, without giving a reason. Shazeer and Stern (2018, Section 8.1) explain the same factor in their Transformers. The table is stored with small entries, of size about $1/\sqrt{d}$ like those of the other weight matrices, so that one step size suits all of them. The factor then brings the vectors it supplies to entries of about size one, like the sinusoids'.

Three attentions

The translator has used attention in three ways since §3, and the paper's names for them are now standard (Figure 18, in §4): **encoder self-attention**, in which every source position reads every source position; **masked decoder self-attention**, in which every target position reads

the target positions up to its own; and **cross-attention**, whose queries come from the decoder and whose keys and values come from the encoder's last layer, the read of the toy, now made from vectors that carry their context.

Write MultiHead(Y, Z) for multi-head attention whose queries are computed from the rows of Y, the target vectors, and whose keys and values are computed from the rows of Z, the source vectors, each head with its own maps. The encoder applies L layers (the paper writes N) to the source vectors Z, each with two sublayers in the post-norm arrangement of §§5–6:

$$Z \leftarrow \text{LayerNorm}(Z + \text{MultiHead}(Z, Z)), \qquad Z \leftarrow \text{LayerNorm}(Z + \text{MLP}(Z)).$$

Its output Z, the result of the encoder's last layer, holds one vector per source position. The decoder applies L layers to the target vectors Y, computed from BOS and the target shifted right by one position, each with three sublayers:

$$\begin{aligned} Y &\leftarrow \text{LayerNorm}(Y + \text{MultiHead}_{\text{causal}}(Y, Y)), \\ Y &\leftarrow \text{LayerNorm}(Y + \text{MultiHead}(Y, Z)), \\ Y &\leftarrow \text{LayerNorm}(Y + \text{MLP}(Y)). \end{aligned}$$

Every decoder layer reads the same Z: the read of the source in decoder layer ℓ takes its keys and values from the encoder's last layer, not from encoder layer ℓ, and its queries from decoder layer ℓ. With n source words and m target positions, this read's matrix of weights is $m \times n$. The decoder's last vector at position t, $\mathbf{h}_t$, gives the logits of the next word, through the shared table.

The training recipe

The paper's recipe has four parts. The first two have trained every translator of §§1–6:

1. **Adam** (Kingma and Ba, 2015). For each parameter θ, with gradient g at the current step, Adam keeps running averages of the gradient and of its square, and moves the parameter by the ratio:

   $$m \leftarrow \beta_1 m + (1 - \beta_1)\, g, \qquad s \leftarrow \beta_2 s + (1 - \beta_2)\, g^2, \qquad \theta \leftarrow \theta - \eta\, \frac{m}{\sqrt{s} + \epsilon},$$

   with small corrections in the first steps, while the averages start from zero. Dividing by $\sqrt{s}$ makes every parameter move by about η per step, whatever the scale of its gradient. The paper used $\beta_1 = 0.9$, $\beta_2 = 0.98$, and $\epsilon = 10^{-9}$.
2. **A warmup, then a decay.** The learning rate rises linearly for the first w steps and then decays as the inverse square root of the step number n:

   $$\eta_n = d^{-1/2} \min(n^{-1/2},\ n\, w^{-3/2}),$$

   with $w = 4{,}000$ in the paper and $w = 1{,}000$ here, a peak of 0.0023. §6 gave one reason for the warmup, post-norm's large gradients at the start; another is that Adam's estimate s rests on very few gradients in the first steps, which makes its steps erratic (Liu et al., 2020).
3. **Dropout** of 0.1 on the output of every sublayer, before it is added to the sublayer's input, and on the sums of the token and position vectors. In training, each number is set to zero with probability 0.1 and the others are divided by 0.9, so the network cannot rely on any single one; in use nothing is dropped. Without dropout, the paper's base model scored 24.6 BLEU on its development set, against 25.8.

4. **Label smoothing** of 0.1, introduced for image classifiers by Szegedy et al. (2016). Each prediction is trained toward a target that puts 0.9 of its probability on the correct word and spreads 0.1 evenly over the vocabulary, which penalizes giving any word a probability near zero. In the paper's words, "this hurts perplexity, as the model learns to be more unsure, but improves accuracy and BLEU score": 25.3 BLEU without it, 25.8 with it.

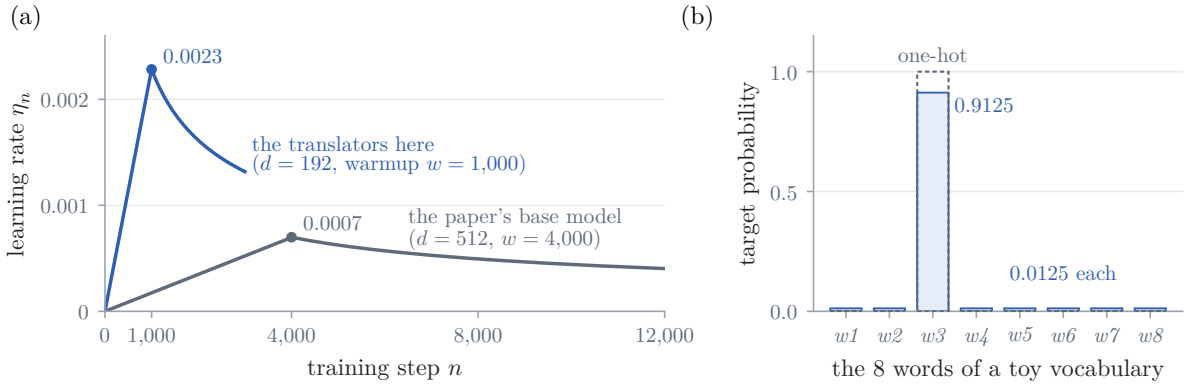


Figure 27. The learning rate rises during the warmup and then decays; label smoothing moves a little probability off the correct word. (a) The schedule $\eta_n = d^{-1/2} \min(n^{-1/2}, n\,w^{-3/2})$: for the translators here ($d = 192$, $w = 1{,}000$, 3,000 steps) it peaks at 0.0023; for the paper's base model ($d = 512$, $w = 4{,}000$, drawn to 12,000 of its 100,000 steps) at 0.0007. (b) The training target over a vocabulary of 8 words: one-hot (dashed) against label smoothing with 0.1, which gives the correct word $0.9 + 0.1/8 = 0.9125$ and each other word 0.0125. With the translator's 6,434 French words, each other word gets $0.1/6{,}434$.

The paper's result

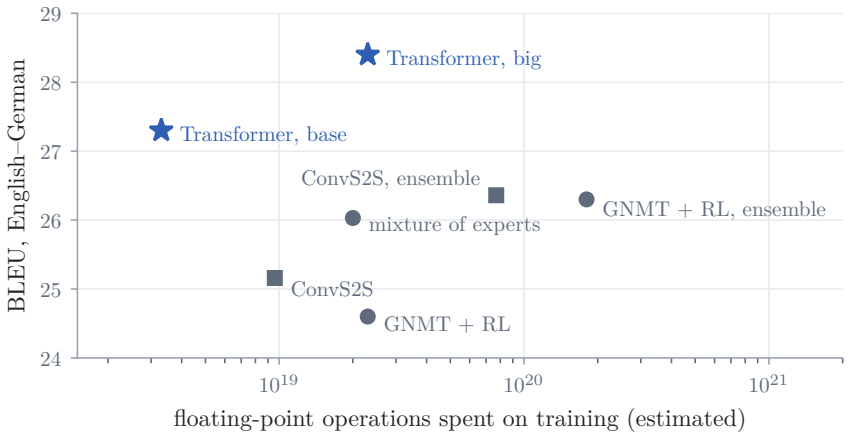


Figure 28. Better translations for a fraction of the training cost. The Transformer paper's main result, from its Table 2: BLEU on the English–German test set of 2014 against the estimated floating-point operations spent on training, for the best earlier systems (gray; single models and ensembles of recurrent, convolutional, and mixture-of-experts models) and the two Transformers (blue). Higher and further left is better.

The paper's base model has six layers on each side, $d = 512$, eight heads of 64 dimensions, and MLPs of width 2,048: about 65 million parameters. The big model has $d = 1{,}024$, sixteen heads, MLPs of width 4,096, and 213 million parameters. Both were trained on the 4.5 million sentence pairs of the 2014 English–German task of the Workshop on Machine Translation (36 million pairs for English–French), on one machine with eight GPUs: the base model for twelve hours, the big model for three and a half days.

The main result was as much about cost as about quality (Figure 28). On English–German the base model scored 27.3 BLEU, above every earlier system, ensembles included, at about a third of the training cost of the cheapest of them. The big model scored 28.4, two points above the best ensemble. On English–French the big model scored 41.8 in the paper's table, slightly above the best ensemble.[6]

What each part contributed

With the recipe, the three-block translator of §§4–6 becomes a small Transformer of 2017, with 5,475,840 parameters once the French table is shared. Its BLEU rises from 52.7 to 53.2, ahead of Lecture 3's LSTM with attention (49.2), and its cross-entropy from 1.08 to 1.12, as label smoothing is expected to do; Appendix C compares it with Lecture 3's translators in more detail.

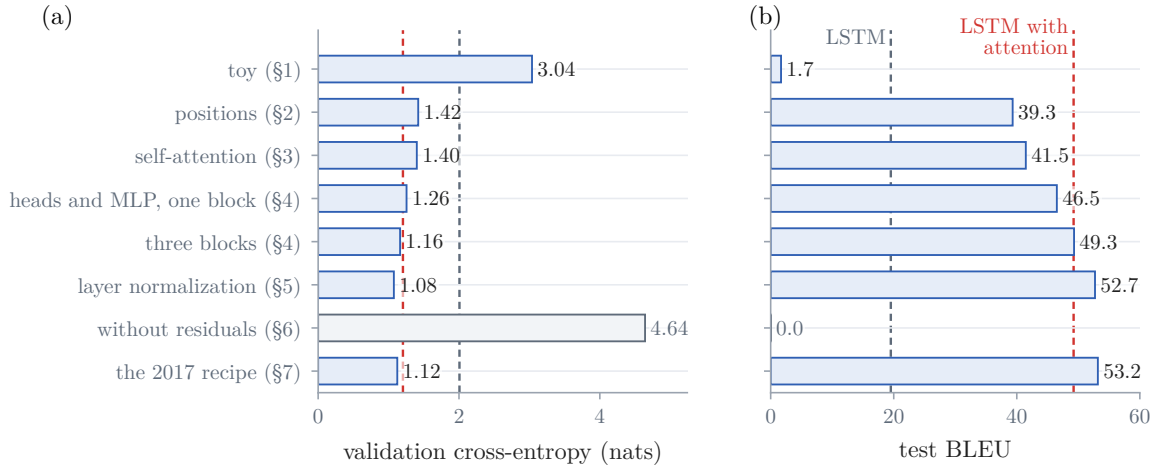


Figure 29. Each part of the Transformer, added in turn, and what it changes. The translators of §§1–7, each trained once in the same way on Multi30k. (a) Validation cross-entropy (nats per French word). (b) Test BLEU. The dashed lines are Lecture 3's LSTM encoder–decoders, without attention (gray) and with it (red). The gray bar is the three-block stack of §6 without residual connections.

Figure 29 collects the steps. The step of §2, positions with the learned maps, made the largest difference: without positions the toy could neither tell word orders apart nor keep its place in its translation. Each later part lowered the cross-entropy further, up to the recipe, which traded a little of it for BLEU, while the three-block stack without residual connections did not learn to translate at all. The steps are single runs taken in one order, and several of them make more than one change (§2 adds positions together with the learned maps, §4 heads and the MLP, and §7 the shared table, dropout, and label smoothing), so the figure records what each step of this build did, not what each part is worth alone.

One caption need not improve at every step, and the karate caption does not (Figure 30): positions end the loop, self-attention finds *tenue*, and later steps trade one error for another, switching between *tenue* and *uniforme* and never getting the kick right, while BLEU over the 1,000 test captions rises at every step.

[6]Versions 1–4 of the paper gave 41.0, slightly below the best ensemble's 41.29, in the abstract, Table 2, and Section 6.1; from version 5 (December 2017) on, the abstract and Table 2 give 41.8, while Section 6.1 still gives 41.0. The paper gives one training cost for each Transformer, for both language pairs.

English	*a girl in karate uniform breaking a stick with a front kick .*	test BLEU
the toy (§1)	*un bâton dans un bâton dans un bâton dans …*	1.7
positions (§2)	*une fille en uniforme de karaté avec un bâton .*	39.3
self-attention (§3)	*une fille en tenue de karaté avec un ballon de pied devant un coup .*	41.5
heads and the MLP (§4)	*une fille en uniforme de karaté fait un bâton devant un coup .*	46.5
three blocks (§4)	*une fille en tenue de karaté faisant un grand saut avec un coup .*	49.3
layer normalization (§5)	*une fille en uniforme de karaté milieu un bâton avec un ballon .*	52.7
the 2017 recipe (§7)	*une fille en tenue de karaté fait un bâton avec un ballon .*	53.2
reference	*une fille en tenue de karaté brisant un bâton avec un coup de pied .*	

Figure 30. The karate caption at every step of the build. Greedy output of each translator of §§1–7, with its test BLEU; *uniforme* in red, *tenue* in blue, the reference in gray. A single caption improves unevenly; the average over the test captions does not.

Removing one part at a time

Removing one part at a time from a complete model, with everything else unchanged, measures each part in the presence of all the others. Figure 31 does this for the eight-block character model of §6: removing the residual connections costs the most, then attention, layer normalization, and the MLP.

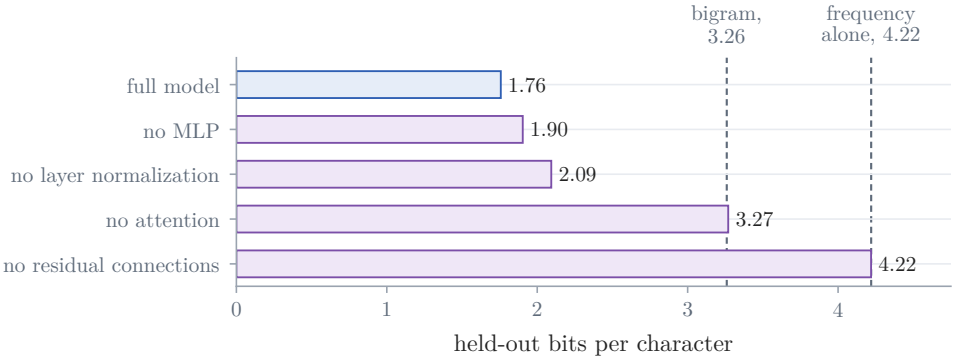


Figure 31. Every part matters, by very different amounts. Held-out bits per character of the eight-block character model of §6 after 600 steps, complete and with one part removed. The dashed lines are the costs of predicting each character by its frequency alone (4.22) and of the character bigram model of Lecture 2 (3.26).

Without attention, each position sees only its own character, not even where it stands, since the model's positions enter only through the attention scores: the model can do no better than a bigram model, and it scores about what the character bigram model of Lecture 2 (Section 6) does, 3.26 bits. Without the MLPs, which hold two thirds of the parameters (264,704 of 402,176), it loses least: at this scale, what attention brings to a position matters more than what the MLP computes there. Without layer normalization it trains, more slowly and less well. These are single runs of a small model, and the ranking of the parts may differ at scale: in large models the MLP holds most of the parameters (§10) and, by the findings cited in §4, much of what the model knows.

Every weight of the translator

The finished translator has these weights:

Matrix	Shape	Role	Parameters
E_{src}	$5{,}898 \times 192$	English word vectors	1,132,416
E_{tgt}	$6{,}434 \times 192$	French word vectors; the output layer	1,235,328
W_Q, W_K, W_V, W_O, 9 attentions	192×192 each	four heads and their combination	147,456 each
$W_1, \mathbf{b}_1, W_2, \mathbf{b}_2$, 6 MLPs	192×768, 768×192	the network at each position	295,872 each
γ, β, 15 normalizations	192 each	scale and shift	384 each
total (sinusoidal positions have none)			5,475,840

More than four in ten of the parameters are the two word tables, a share that falls as models grow (§10).

The finished translator learns only from sentences paired with their translations. Most text comes without one, and a model that learned from text alone could use far more of it.

8. The decoder alone: a language model

The decoder of §7 predicts each French word from the English caption and the French words before it, $q(y_t \mid y_{<t}, \mathbf{x})$ at every position at once. Remove the encoder and the cross-attention, and what remains, a stack of blocks of masked self-attention and MLPs, computes $q(x_t \mid x_{<t})$ at every position of a text: a language model in the sense of Lecture 2, trained on the next token of plain text.

Decoders, encoders, and their uses

A decoder alone can still translate, by writing the source, a separator, and the target as one sequence and predicting it token by token, the reading of transduction as language modeling in Lecture 3 (Section 1). An early decoder of this kind wrote the first section of Wikipedia articles from the documents they cite (Liu et al., 2018). The decoder became the standard language model through OpenAI's GPT (Radford et al., 2018): twelve blocks with $d = 768$ and twelve heads, MLPs with the GELU, learned position vectors, and a context of 512 tokens. GPT was trained on about 7,000 unpublished books, then adapted to twelve tasks of language understanding, and was the best published model on nine of them. GPT-2 (Radford et al., 2019) changed three things:

1. it used pre-norm (§6);
2. it scaled down the initial weights of the layers that write into the residual stream by $1/\sqrt{2L}$, so that the stream does not grow with depth at the start of training;
3. it grew to 48 blocks and 1.5 billion parameters, with a context of 1,024 tokens.

The same block also works without the causal mask (Figure 32).

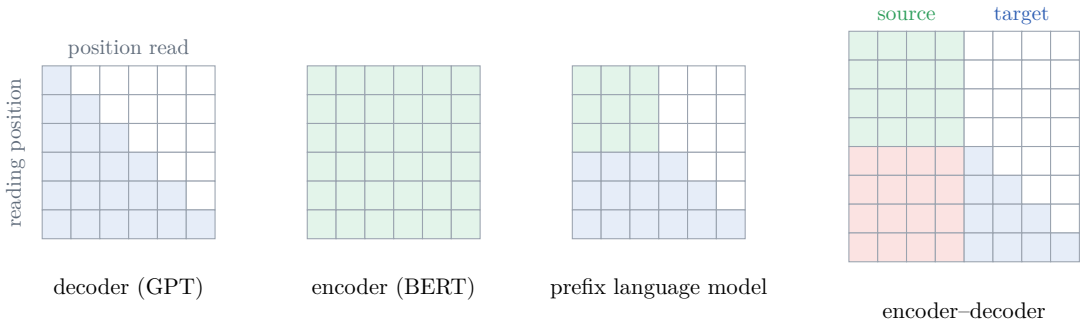


Figure 32. Decoders, encoders, prefix language models, and encoder–decoders differ in their masks. Each grid shows which positions (columns) a position (row) may read, shaded where allowed. From the left: a decoder (GPT), where each position reads itself and the positions before it; an encoder (BERT), where every position reads every position; a prefix language model, where the positions of an input (here the first three) read each other and every later position reads everything before it; and an encoder–decoder (the Transformer of 2017; T5, Raffel et al., 2020), where the encoder's positions read each other and the decoder's positions read the decoder positions up to their own and, through cross-attention, all of the encoder's.

Every position then reads the whole text, which makes predicting the next token trivial, so such **encoders** are trained to fill in blanks instead: BERT (Devlin et al., 2019) hides 15% of the tokens of a text and learns to predict them. For several years fine-tuned encoders were the best models for classifying and tagging text. But an encoder is not a generative model of text: its predictions of blanks do not combine by the chain rule into a distribution over texts. Between the two lies the **prefix language model**, a decoder in which the positions of a given input read each other in both directions while the output is predicted causally. Since about 2020 nearly every large language model has been a decoder.

One batch, step by step

Figure 33 draws the decoder-only model in the pre-norm arrangement of §6. The **word model** of this section is such a decoder, with GPT-2's layout, trained on the children's stories of Lecture 2; the table below follows one of its training batches through it, shape by shape.

The model has $L = 4$ blocks of width $d = 128$, $H = 4$ heads of $d_k = 32$ dimensions, MLPs of width $d_{\text{ff}} = 512$, positions entered by rotating each query and key by angles proportional to its position (rotary positions, §9), and an input vocabulary of $|\mathcal{V}| = 8{,}682$ tokens: the 8,681 words of Lecture 2's vocabulary and BOS. A batch holds $B = 16$ chunks of $T = 256$ tokens, and all of them, at all positions, are processed in parallel. As in the decoder of §3, position t holds the input x_{t-1} and predicts x_t, and each story starts with BOS. X holds the residual stream, one vector $\mathbf{h}_t$ per position, as in §3, and $\hat{X}$ its normalized copy.

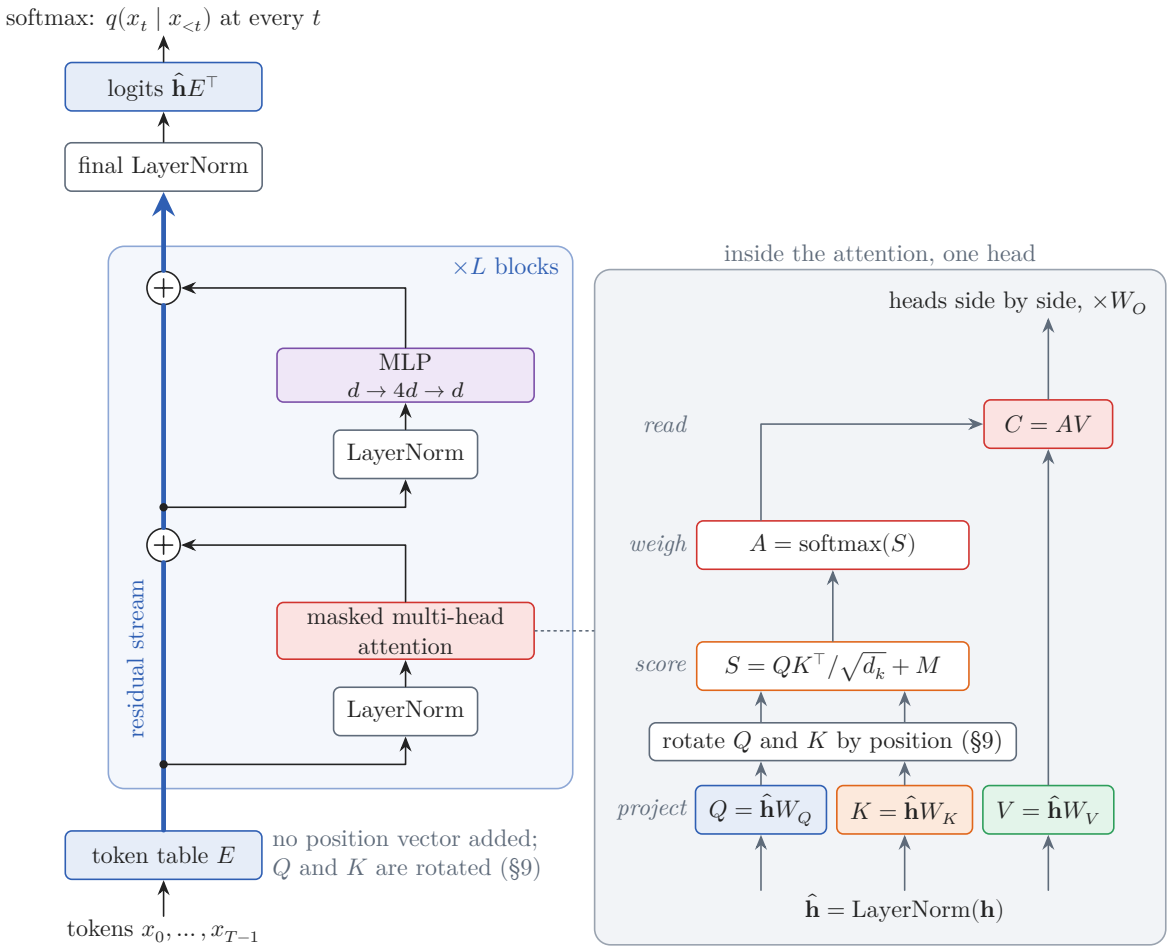


Figure 33. A decoder-only Transformer: a residual stream that every sublayer reads and writes. Left: each token's vector enters the residual stream (the vertical line, blue); each of the L blocks normalizes the stream, computes masked multi-head attention (red), and adds the result to the stream, then does the same with the MLP (purple); at the top, a final normalization and the token table, used in reverse, give logits for the next token at every position. Right: inside the attention, one head projects the normalized vectors to queries, keys, and values, rotates the queries and keys by their positions, scores, weighs, and reads. Positions enter only as these rotations (§9): no position vector is added to the token vectors.

Step	Operation	Shape
Embed		
tokens	I, integer token ids, the inputs $x_0, \ldots, x_{T-1}$ of each chunk	16×256
embed	$X = E[I]$, rows of the token table	$16 \times 256 \times 128$
Attention sublayer, in each of the L blocks		
normalize	$\hat{X} = \text{LayerNorm}(X)$, each vector separately	$16 \times 256 \times 128$
project	$Q = \hat{X}W_Q + \mathbf{b}_Q$, $K = \hat{X}W_K + \mathbf{b}_K$, $V = \hat{X}W_V + \mathbf{b}_V$, each split into 4 heads; rotate Q and K by their positions	$16 \times 4 \times 256 \times 32$
score	$S = QK^\top/\sqrt{32} + M$, one 256×256 matrix per head	$16 \times 4 \times 256 \times 256$
weigh	$A = \text{softmax}(S)$, along each row	$16 \times 4 \times 256 \times 256$
read	$C = AV$, then the heads side by side	$16 \times 256 \times 128$
write	$X \leftarrow X + CW_O + \mathbf{b}_O$	$16 \times 256 \times 128$
MLP sublayer, in each of the L blocks		

Step	Operation	Shape
expand	$G = \text{GELU}(\text{LayerNorm}(X)W_1 + \mathbf{b}_1)$	$16 \times 256 \times 512$
write	$X \leftarrow X + GW_2 + \mathbf{b}_2$	$16 \times 256 \times 128$
Output, after the blocks		
logits	$Z = \text{LayerNorm}(X)\, E^\top$, the token table used in reverse	$16 \times 256 \times 8{,}682$
loss	the average of $-\log \text{softmax}(Z)$ at the targets $x_1, \ldots, x_T$	scalar

Every step except *score* and *read* acts on each of the 16×256 vectors separately, with weights shared by all of them (§3). The logit of BOS is set to $-\infty$, so that the distribution is over the 8,681 words, which include the end-of-story token. The loss is the average log loss of the $16 \times 256 = 4{,}096$ next-token predictions of the batch, the objective of Lecture 2, computed by one pass.

Training

As in the translator's decoder (§3), one pass over a chunk of text computes the prediction of every next token, each from its own prefix. Figure 34 shows this on a sentence in the style of the stories, *lily saw a big dog in the park . she gave it a bone and it was happy .*, the **running sentence** of this section.

A training step does the following:

1. **Sample a batch.** Draw B chunks of T consecutive positions from the training stories, joined one after another. The inputs are the stories, each preceded by BOS; the targets are the same stories, each followed by the end-of-story token, so that the target at each position is the next input.
2. **Forward.** Compute the logits at every position of every chunk, and the average log loss of the BT targets.
3. **Backward.** Compute the gradient of the loss with respect to every parameter by back-propagation, as in Lecture 1 (Section 8).
4. **Update.** Move each parameter against its gradient with Adam (§7), with a learning rate that rises during a warmup and then decays.

Repeat for a fixed number of steps, and keep the parameters that do best on held-out text.

The word model is the decoder followed through one batch above, with a context of 256 tokens, longer than most stories. It uses the data and vocabulary of the word models of Lecture 2 (Sections 6–8), with every story starting from BOS. It has 1,904,640 parameters, 1,111,296 of them in the token table, which is tied to the output layer (§7). It was trained for 4,000 steps of 16 chunks of 256 words, the 16 million words the word LSTM of Lecture 2 read, with Adam at a learning rate that rises over 200 steps to 0.003 and then decays along a cosine to a tenth of that, and with gradients clipped at a norm of 1, in 54 minutes on two processor cores.

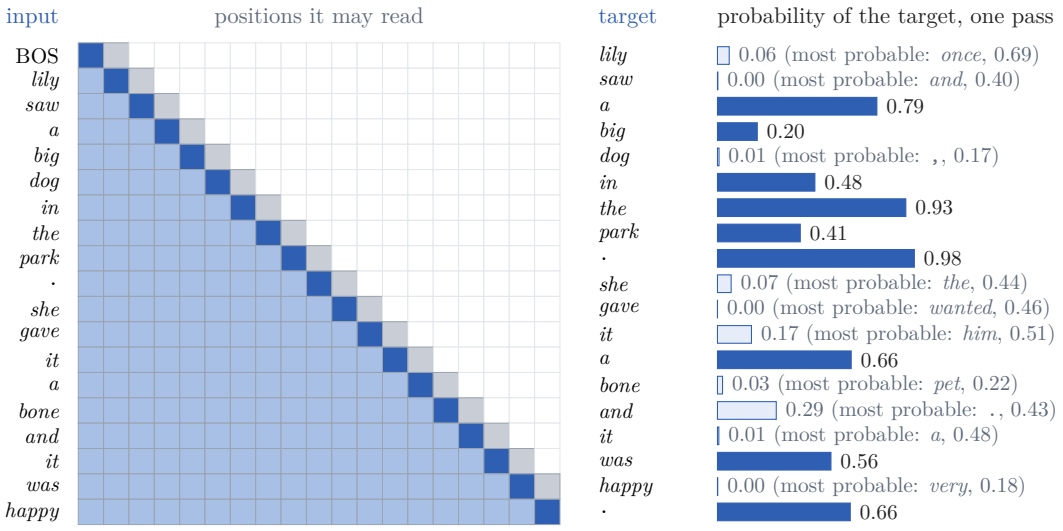


Figure 34. One pass computes the prediction at every position, and each prediction reads only its prefix. The running sentence as a training example for the word model. Left: the inputs, one per row. Middle: the causal mask; a position may read the blue squares, the dark one its own input, but not the gray one, the token it must predict. Right: the target of each position and the probability that the trained word model gives it in one pass (pale bars: another token is the most probable, given in parentheses).

Against Lecture 2's models

On the test stories, with the prediction events of Lecture 2 (every word and every end-of-story token, each story read from its beginning), it and Lecture 2's models reach these perplexities:

Model	Trained on	Depth	Parameters	Test perplexity
Kneser–Ney 4-gram	counts of up to four words	none	counts	24.6
Fixed-window network	windows of four words	one hidden layer	1.7 million	26.9
LSTM (Lecture 2)	chunks of 64 words	one layer of 128 units	1.8 million	23.1
Transformer	chunks of 256 words	four blocks	1.9 million	9.9

Its probabilities sum to one over the vocabulary, and it is scored on the same 442,736 prediction events of the test stories on which Lecture 2's code gives the LSTM its 23.1, so the perplexities can be compared. The Transformer and the LSTM differ in more than the architecture, though: the Transformer has four blocks against one recurrent layer, and it was trained on chunks of 256 words against 64, which reach back over most of a story. The comparison is of two setups, not of attention against recurrence. Appendix D shows how the two compare during training.

Generation

Generation is the loop of Lecture 2 (Section 7), run one token at a time:

1. Feed the text so far, starting with BOS, through the model.
2. Take the distribution at the last position, and choose a token from it, here by sampling with a temperature (Lecture 2, Section 6).
3. Append the token to the text, and repeat until the end-of-story token, or a length limit.

Every step runs the whole prefix through the model again, and only the last position's output is used. The model's next-word distributions look like those of Lecture 2's models, only sharper (Figure 35). A story sampled from BOS at temperature 0.7:

> once upon a time, there was a little girl named lily. she loved to play outside in the sun. one day, she went to the park with her mommy and daddy. they were having so much fun! suddenly, lily's mommy and daddy came running up and down the slide. they were very sad and didn't know what to do. they wanted to leave the slide but they were afraid. lily's mommy said, " let's go back to the slide and have a good day. " they went home and put on their coats, left the slide, and mommy was happy. …

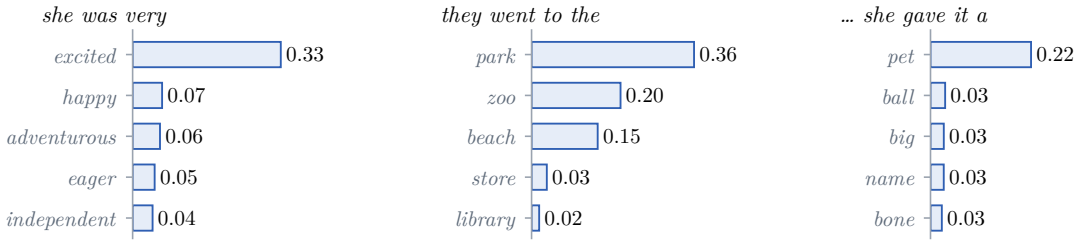


Figure 35. The word model's next-word distributions are sharp where the story allows few words. The five most probable next words, with their probabilities, after three prompts: *she was very* (*excited* 0.33), *they went to the* (*park* 0.36, *zoo* 0.20, *beach* 0.15), and the running sentence up to *she gave it a* (*pet* 0.22).

Checking the mask

The argument of §3 shows that a correct implementation is causal. An implementation can still be wrong: a mask shifted by one position, padding that leaks, an operation that mixes positions in the wrong place. A leak lets the prediction at position t read the input at position $t + 1$, which is the token it must predict, and the model learns to copy it. Its training loss and its held-out loss, both computed by one teacher-forced pass, then become far lower than any honest model could reach. A separate test set does not reveal the leak, because the test text is fed in the same way.

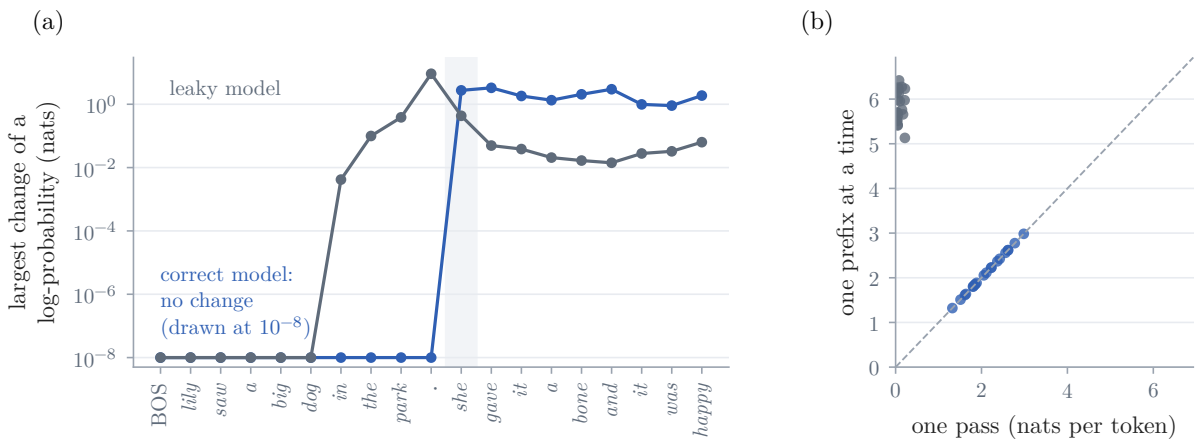


Figure 36. Two tests catch a mask that lets a prediction see its answer. The trained word model (blue) and a model trained with a leaky mask, which lets each position also read the next position (gray). (a) The running sentence with the input at one position (*she*, shaded) replaced by *he*: the largest change of any log-probability at each position (a change of exactly zero is drawn at 10^{-8}). In the correct model nothing changes before the replaced position. (b) The log loss of short held-out stories computed in one pass and computed one prefix at a time, keeping only the last position's prediction: for the correct model the two agree, and for the leaky model the one-pass loss is far too low.

A word model trained for 300 steps with a mask that lets each position also read the next one, and otherwise the same settings, has a teacher-forced perplexity of 1.08 on the first 1,000 validation stories, far below the 10.0 of the honest model on the same stories. Two tests catch it:

1. **Change the future.** Replace the input at one position and verify that the outputs at all earlier positions are unchanged. In the honest model nothing before the replaced position changes at all; in the leaky model, the position just before it changes by 9.2 nats, and since each of the four blocks lets a position read one position further, the leak reaches four positions back (Figure 36a).
2. **Compare with one prefix at a time.** Compute the log loss of a text in one pass, and again by feeding each prefix alone and keeping only the prediction at its last position, where there is no next input to read. The two must agree up to rounding. For the honest model they agree within 3×10^{-7} nats per token on 20 short test stories; for the leaky model the one-pass loss is 0.09 nats per token and the prefix-by-prefix loss 5.85, worse than the honest model's 2.11 (Figure 36b).

The second test is also the definition of what the model should compute, since a model in use only ever sees prefixes. The word model passes both.

Appendix D looks inside the trained word model: what one head reads, what removing it costs, and how self-attention gives one word different vectors in different contexts.

One part of the word model differs from the translator of §7: its positions. Position vectors added to the word vectors, as in §2, mix where a word stands with what it is, while what a read most often needs is how far back another word stands.

9. Positions revisited: rotations

The position vectors of §2 meet the fifth requirement only half way. The dot product of two sinusoidal vectors depends only on their offset, but a score compares two maps of word-plus-position vectors,

$$e_{tj} = \left(E(x_{t-1}) + \mathbf{p}_t\right) W_Q W_K^\top \left(E(x_{j-1}) + \mathbf{p}_j\right)^\top,$$

where $E(x_{t-1})$ is the vector of the token at position t, as in §8. Expanded, this has a term in the two words, a term in the two positions, and two terms that mix one word with the other's position, and nothing makes the position terms depend on the offset alone once $W_Q W_K^\top$ is learned. And the hope that sinusoids would let a model read texts longer than any in training was not borne out: Press et al. (2022) found that language models with sinusoidal positions have "very limited extrapolation capabilities". Most language models since about 2022 put the positions where the offset matters, into the scores, and leave the word vectors alone.

What a relative position must be

Ask for the score of a query at position t and a key at position j to depend on the two positions only through their offset. Su et al. (2021) posed this requirement and found one solution. A short argument, from a blog post by Puranik (2026), shows that under four assumptions it has essentially no others; the post concedes that the first, linearity, is a choice rather than a necessity:

1. **Linearity.** Position t acts on a query by a matrix, $\mathbf{q}_t \mapsto \mathbf{q}_t F_t$, and position j on a key by a matrix, $\mathbf{k}_j \mapsto \mathbf{k}_j G_j$.
2. **Only offsets are visible.** The score $\mathbf{q}_t F_t G_j^\top \mathbf{k}_j^\top$ depends on the positions only through $j - t$: $F_t G_j^\top = B(j-t)$ for some matrices $B(u)$.
3. **Nothing happens at offset zero.** $B(0) = I$; any other $B(0)$ can be folded into the key map W_K.
4. **Continuity.** $B(u)$ changes continuously with u.

From the first three, $B(a+b) = B(a)\,B(b)$ for all offsets a and b (Appendix A.2): the matrices $B(u)$ form a **one-parameter group**, like the rotations of a clock hand by u steps. Every continuous one-parameter group of matrices has the form $B(u) = \exp(uX)$ for one fixed matrix X, and, setting aside matrices X that cannot be diagonalized, the eigenvalues of X list the possibilities (Figure 37):

- an eigenvalue 0: the score does not depend on the positions at all, a model with **no position information**;
- a negative or positive real eigenvalue: the score shrinks or grows exponentially with the offset; a shrinking factor, a decay with distance, is used in some recurrent variants of attention (§11), and a growing one is useless;
- a pair of complex eigenvalues $a \pm \mathrm{i}\omega$: a two-dimensional block that **rotates** by the angle $u\omega$ and scales by e^{ua}. With $a = 0$, this is a pure rotation.

So an encoding that is linear, sees only offsets, and is continuous rotates pairs of coordinates by angles proportional to the offset, possibly with a decay; what remains to choose is the frequencies. This is the form that the most widely used encoding takes.

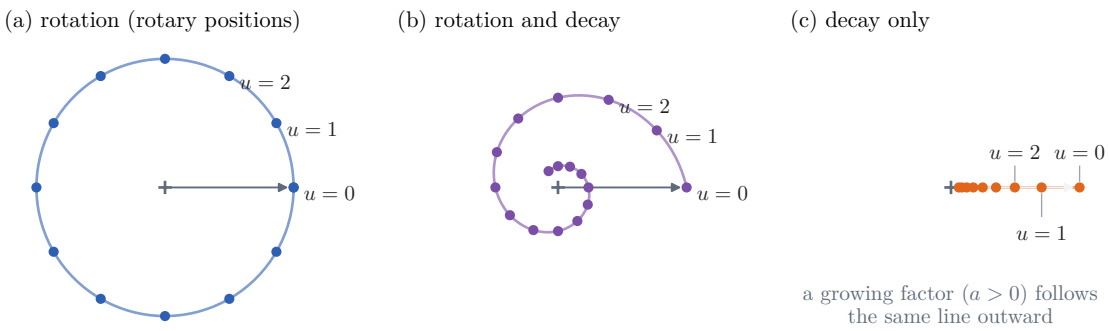


Figure 37. A continuous group of matrices that depends only on the offset rotates, decays, or both. The path of a two-dimensional vector under $B(u) = \exp(uX)$ for $u = 0, 1, 2, \ldots$ (dots). (a) Eigenvalues $\pm i\omega$: a rotation by $u\omega$, here with $\omega = 30°$, the form of rotary positions. (b) Eigenvalues $a \pm i\omega$ with $a < 0$: rotation and decay. (c) A negative real eigenvalue: decay along a line; a positive one grows along the same line.

Rotary position embeddings

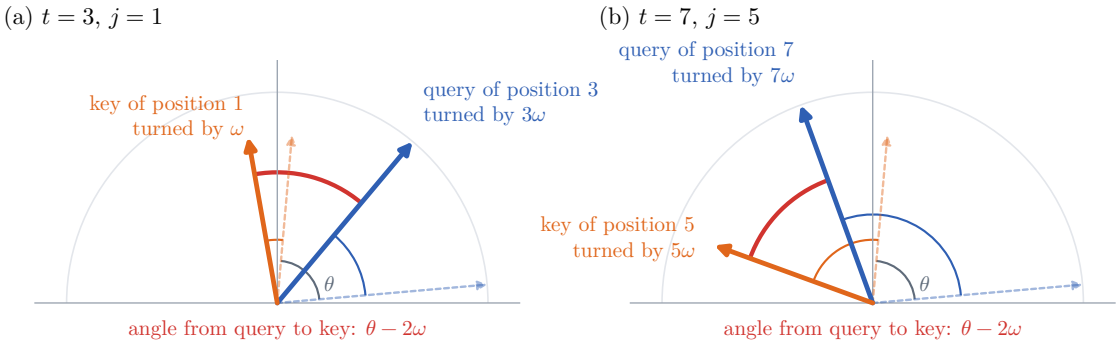


Figure 38. Rotary positions turn the query and the key by their positions, so the angle between them depends only on the offset. One pair of coordinates of a query and a key, before turning (dashed) and after (solid); θ is the angle from the query to the key before turning (gray). A vector at position t is turned by $t\omega$, here with $\omega = 15°$. (a) The query at position 3 and the key at position 1. (b) The same query and key at positions 7 and 5. Both are turned further in (b), but the angle between them (red) is the same, and so is their dot product.

Rotary position embeddings (Su et al., 2021) add nothing to the word vectors. In every head, they rotate each query and each key, pair of coordinates by pair of coordinates, by an angle proportional to its position: the query of position t by $t\,\omega_i$, the key of position j by $j\,\omega_i$, with the frequencies ω_i of the sinusoids. A dot product does not change when both vectors are turned by the same angle, so the score depends on the positions only through their offset (Figure 38):

$$(\mathbf{q}_t R_t)(\mathbf{k}_j R_j)^\top = \mathbf{q}_t\, R_t R_j^\top\, \mathbf{k}_j^\top = \mathbf{q}_t\, R_{t-j}\, \mathbf{k}_j^\top,$$

where R_t rotates every pair i by the angle $t\,\omega_i$, and $R_t R_j^\top = R_{t-j}$ because rotations compose by adding angles. A head can then express "the token two positions back" in the same way at every position. The positions enter only through the scores, and the values, which carry what a position passes on, carry no position at all.

Without positions, and beyond the training length

A small experiment compares three choices for a language model. A character-level model of the children's stories, with four blocks and a context of 256 characters,[7] was trained for 750 steps once with no position information, once with learned position vectors, and once with rotary positions. On held-out stories it scored 2.19 bits per character with none, 1.93 with learned vectors, and 1.57 with rotary positions (Figure 39). Even without positions it beats the 3.26 bits of the character bigram model of Lecture 2 (Section 6), since the causal mask lets it recover some order. For characters, what matters most is the few characters just before, wherever they stand, and rotary positions make "just before" the same at every position. These are short single runs.

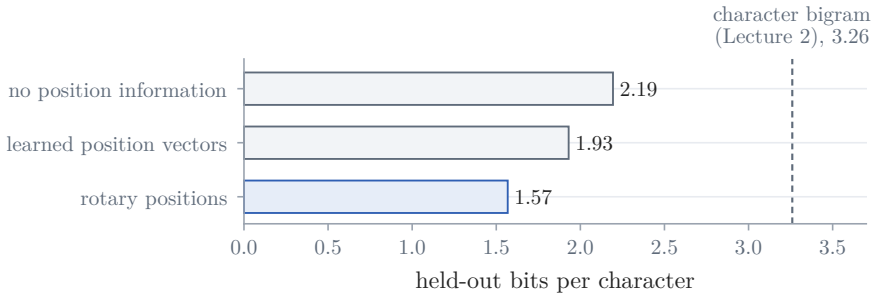


Figure 39. Rotary positions did best in a small language model, and even a model without positions beat the bigram model. Held-out bits per character of the four-block character model of the children's stories after 750 steps, with no position information, learned position vectors, or rotary positions; single short runs. The dashed line is the character bigram model of Lecture 2.

A decoder can do without any position information better than an encoder. With a causal mask, position t reads exactly t positions, so a head that spreads its weight evenly over its prefix, BOS included, reads a share $1/t$ of the value of BOS, a number that gives away the position. Language models of up to 1.3 billion parameters trained without position information came close to those with it, while encoders without positions failed to train (Haviv et al., 2022). A single read still cannot see the order within its prefix: the last position of *dog bites man* and of *man bites dog* reads the same three keys and values, unless the layers below have already written order into the vectors.

Rotations do not by themselves let a model read beyond its training length either: angles it never saw in training produce scores it never learned to use. Models trained on short texts are therefore extended by rescaling the rotations, so that a long text produces the angles of a shorter one, and training briefly on longer texts (Chen et al., 2023; Peng et al., 2024).

Models of 2026 have hundreds of billions of parameters and read hundreds of thousands of tokens, so the next question is what a Transformer costs: in parameters, in computation, and in memory as its text grows. The changes that such scale brought to the block of 2017 are collected in Appendix E.

[7] Width 80, four heads per block, 314,240 parameters (334,720 with learned position vectors), trained on the 6,000 training stories of Lecture 2's character models.

10. Every weight, and what it costs

A trained language model is a set of weight matrices. Their shapes fix the model's size and the computation that trains it, and, with the length of the text, the memory it needs to write.

Every weight matrix

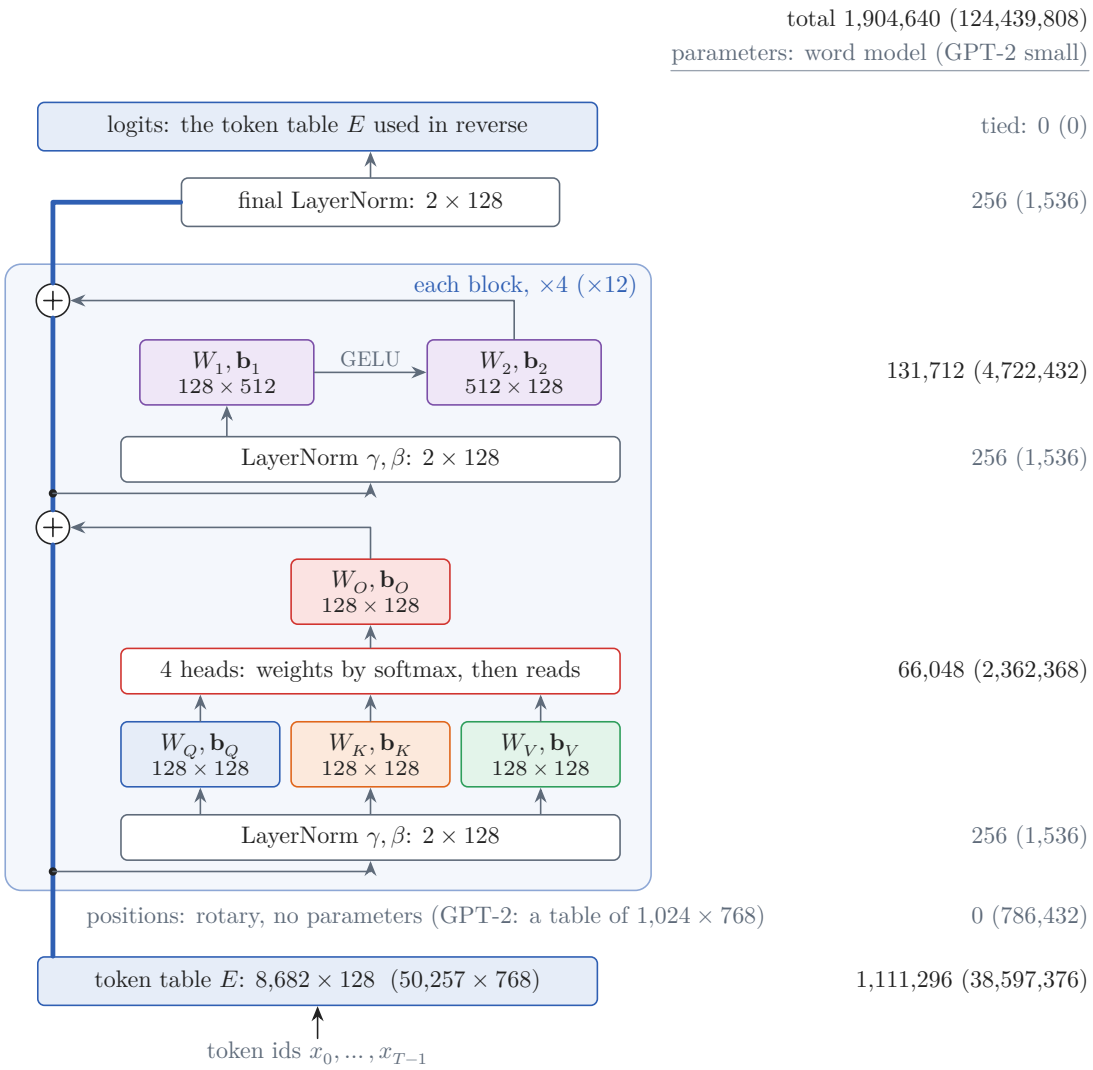


Figure 40. Every weight matrix of a decoder-only Transformer, with its shape and what it does. The word model of §8 ($d = 128$, $L = 4$, $|\mathcal{V}| = 8{,}682$), from the token ids at the bottom to the logits at the top. Each box with a shape is a matrix of the model with its bias, or a normalization's scale and shift; the outlined red box between W_Q, W_K, W_V and W_O is the heads' computation, which has no parameters. On the right, the number of parameters of each row, and in parentheses the number in GPT-2 small ($d = 768$, $L = 12$). Everything inside the frame is repeated L times with its own weights. Colors as in the other figures: queries and the stream blue, keys orange, values green, attention red, the MLP purple, normalizations gray.

The model's weights, from the input to the output (Figure 40):

Weights	**Shape**	**What they do**	**Where**
token table E	$\|\mathcal{V}\| \times d$	a vector for each token; used in reverse, the logit of each next token	§§2, 7
position table (GPT-2)	$T_{\max} \times d$	a vector for each position; none with rotary positions	§§2, 9
W_Q, W_K, with biases, in each block	$d \times d$, cut into H heads	where each head reads: its queries and keys	§3
W_V, W_O, with biases, in each block	$d \times d$ each	what each head writes: down to d/H numbers and back up	§§3–4
W_1, $\mathbf{b}_1$, in each block	$d \times 4d$, $4d$	the questions of the MLP's hidden units	§4
W_2, $\mathbf{b}_2$, in each block	$4d \times d$, d	what each hidden unit adds to the vector	§4
γ, β, two per block and one at the end	d each	rescale and shift each normalized vector	§§5–6

Nothing else in the model is learned: the softmax, the mask, the rotations, the normalization's means and variances, and the residual additions have no parameters.

Parameters

A block has $4d^2$ weights in its attention (W_Q, W_K, W_V, W_O, whatever the number of heads, §4) and $8d^2$ in its MLP (W_1 and W_2, with $d_{\text{ff}} = 4d$), plus biases and normalization parameters, which are fewer by a factor of about d. A decoder with L blocks therefore has about

$$N \approx 12Ld^2$$

parameters in its blocks, plus $|\mathcal{V}|d$ in its token table (shared with the output layer when tied) and, with learned positions, $T_{\max}d$ in its position table, where $T_{\max}$ is the longest context. For GPT-2 small, with $L = 12$, $d = 768$, $|\mathcal{V}| = 50{,}257$, and $T_{\max} = 1{,}024$:

Component	**Count**	**Parameters**
blocks, weights	$12 \times 12 \times 768^2$	84,934,656
blocks, biases and normalizations	$12 \times 13 \times 768$	119,808
token table	$50{,}257 \times 768$	38,597,376
position table	$1{,}024 \times 768$	786,432
final normalization	2×768	1,536
total		124,439,808

The decoder of §8, built with these sizes and learned positions, has exactly this many. (The GPT-2 paper reported 117 million; the repository of the released code later explained that the original counts were wrong.)

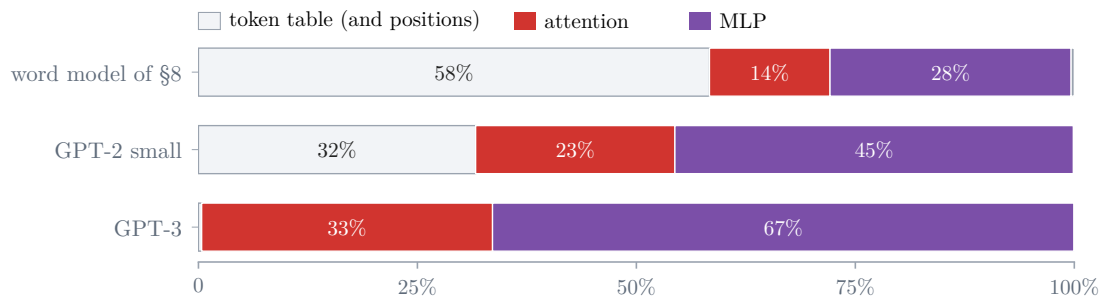


Figure 41. As models grow, the blocks take over from the token table. The share of the parameters in the token table (with the position table, if any), in attention (W_Q, W_K, W_V, W_O), and in the MLPs (W_1, W_2), for the word model of §8 (1,904,640 parameters), GPT-2 small (124,439,808), and GPT-3 (about 175 billion, counted with its token table shared with the output layer). Biases and normalizations, at most 0.4%, make up the rest. After the running tally of GPT-3's weights in Sanderson (2024a, b, c).

A third of GPT-2 small is its token table (Figure 41). As models grow, the blocks dominate: GPT-3, with $L = 96$ and $d = 12{,}288$, has about 1.74×10^{11} parameters in its blocks and 0.6 billion in its token table, of 175 billion in all (Brown et al., 2020). The word model of §8 is the opposite case: $12 \times 4 \times 128^2 = 786{,}432$ weights in its blocks against 1,111,296 in its token table. An encoder–decoder's decoder layers have a cross-attention as well, $4d^2$ more, which is why the base Transformer's decoder (25 million parameters) is larger than its encoder (19 million).

Computation

A product of a vector with a weight matrix of n entries costs n multiplications and n additions. Every weight of the blocks is used once per token in this way, so a forward pass costs about $2N$ floating-point operations (FLOPs) per token for the weights. The backward pass costs about twice as much: each matrix product of the forward pass gives rise to two of the same size, one for the gradient with respect to its input and one for the gradient with respect to its weights (Appendix A.3). Training therefore costs about

$$C \approx 6N \text{ FLOPs per token}$$

for the weights, where N counts every weight used in a matrix product, including the token table when it computes the logits. This is the rule that Kaplan et al. (2020) used for their scaling laws.

The rule checks the Transformer paper's estimate of its own cost. In each batch of the base model, the encoder's 19 million weights act on each of about 25,000 source tokens, and the decoder's 25 million and the shared output table's 19 million on each of about 25,000 target tokens: about $6 \times (19 + 44) \times 10^6 \times 25{,}000 \approx 9.5 \times 10^{12}$ FLOPs, and 9.5×10^{17} over its 100,000 steps. The paper estimated 3.3×10^{18}, from the training time and an assumed speed of the GPUs, so the GPUs ran at about 30% of the assumed speed.

By the same rule, training GPT-2 small on ten billion tokens costs about $6 \times 1.24 \times 10^8 \times 10^{10} \approx 7 \times 10^{18}$ FLOPs, which eight A100 GPUs did in an hour and a half in 2024 (Karpathy, 2024).

The quadratic term

Attention adds work that the parameter count does not see. With T positions in view, a token's scores cost Td multiplications in each block, summed over the heads, and its weighted average another Td: $4Td$ FLOPs per token and block, forward. A straightforward implementation, like PyTorch's reference formulation of attention and the word model of §8, computes the full $T \times T$

matrix of scores, masks it, and multiplies the full matrix of weights by the values (PyTorch documentation), so every position costs $4Td$, although the mask zeroes half of the weights. An implementation that skips the masked triangle reads $T/2$ positions on average. Over a sequence of T tokens and L blocks, against $24LTd^2$ FLOPs for the weights:

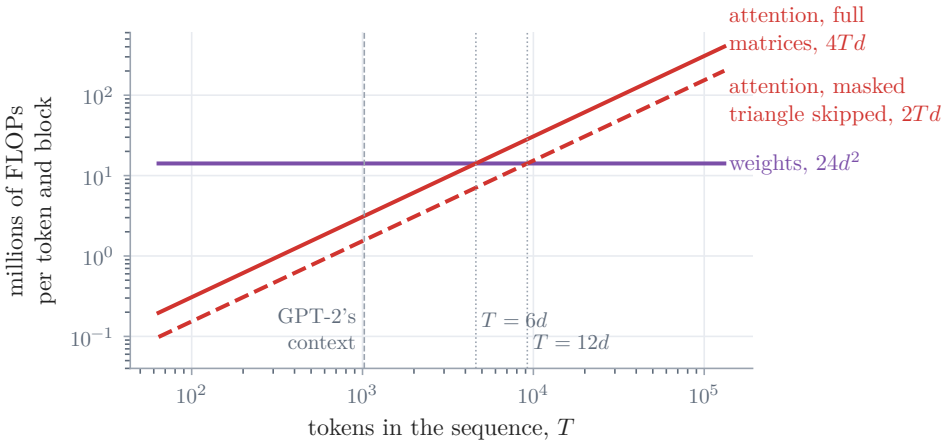


Figure 42. Attention's cost per token grows with the context; the weights' cost does not. Forward FLOPs per token in one block of a model of width $d = 768$ (GPT-2 small), against the number T of tokens in the sequence: about $24d^2$ for the weights, whatever the context (purple), and for attention $4Td$ if the full matrices are computed (red) or $2Td$ if the masked triangle is skipped (red, dashed). The weights' cost is reached at $T = 6d$ or $T = 12d$. Logarithmic scales.

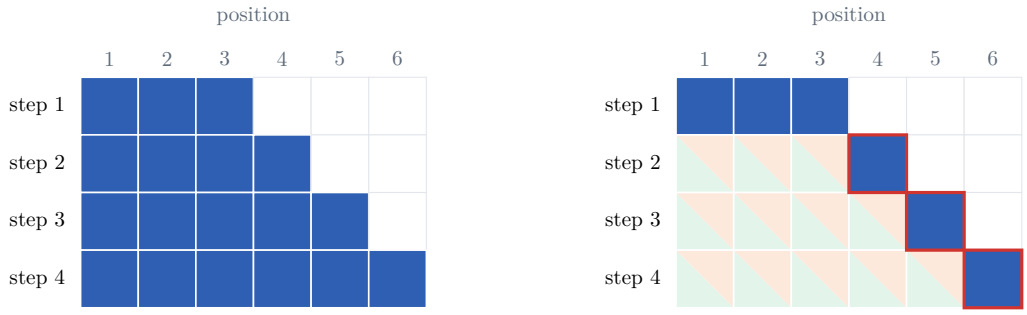
Attention implementation	**Forward pairwise work, all blocks**	**Equal to the weights' work at**
Computes the full matrices	$4LT^2d$	$T = 6d$
Skips the masked triangle	$2LT^2d$	$T = 12d$

Both expressions leave out the softmax, the normalizations, and other terms that grow with the number of activations rather than the number of weights. For GPT-2 small the crossover is at about 4,600 or 9,200 tokens, far beyond its context of 1,024 (Figure 42): at that scale the weights dominate. For a model with $d = 4{,}096$ and a context of 128,000 tokens, attention dominates. A straightforward implementation also stores the $T \times T$ matrix of weights of every head of every block for the backward pass: at 8,192 tokens, 67 million numbers per head. That memory, as much as the arithmetic, limited context lengths in practice. FlashAttention (Dao et al., 2022) computes exact attention in blocks, without ever storing the full matrix.

Arithmetic is not time. The time also depends on how much data each operation moves and on how many computations must wait for one another: in training, a pass over a text takes L dependent block computations however long the text, while in generation each new token needs the one before it, so r generated tokens take rL (Lecture 3, Section 10).

The key–value cache

Generation as described in §8 recomputes the whole prefix at every step and uses only the last position's output. But by the argument of §3, nothing at position t depends on the tokens after it: when a token is appended, the keys and values of the earlier positions, in every block and head, are exactly what they were at the step before. A running model therefore keeps them, in the **key–value cache** (Figure 43).



(a) without a cache (b) with a cache

Figure 43. With a cache, each generation step computes one new column. Generating four tokens after a prompt of three, in one attention layer. Each row is one generation step, each column one position; a filled square is a position whose query, key, and value that step computes. (a) Without a cache, every step recomputes every position of the prefix. (b) With a cache, the keys and values of earlier positions are kept (pale green and orange), and each step computes only the new position, whose query reads the cached keys and values (red).

At each generation step, the model:

1. computes the query, key, and value of the new position only, in each block, from the new token and what the blocks below computed for it;
2. appends the new key and value to the cache of that block;
3. reads the cache with the new query, and continues to the next block.

Each step then costs the weights' $2N$ FLOPs once, plus a read over the cache, instead of a pass over the whole prefix (Figure 43). Generation becomes one step per token, like a recurrent network's, but its "state", the cache, grows with every token instead of having a fixed size. The cache holds, for each token and block, a key and a value of d numbers each (summed over the heads), so a sequence of T tokens needs

$$2LdT \text{ numbers.}$$

For GPT-2 small at its full context of 1,024 tokens this is $2 \times 12 \times 768 \times 1{,}024 \approx 18.9$ million numbers, 38 MB at two bytes each. For a model of the Llama family with 7 billion parameters ($L = 32$, $d = 4{,}096$) at 4,096 tokens it is 1.07 billion numbers, 2.1 GB per sequence, beside 13.5 GB for the weights. A server answering many requests at once therefore runs out of memory for caches before it runs out of arithmetic.

The cache can be made smaller by letting heads share keys and values. In **multi-query attention** all the query heads of a block share one key head and one value head (Shazeer, 2019), and in **grouped-query attention** groups of query heads share one (Ainslie et al., 2023). The largest model of Llama 2 (Touvron et al., 2023), with 80 blocks of width 8,192, has 64 query heads of 128 dimensions and 8 key–value heads. It caches $2 \times 80 \times 8 \times 128 = 163{,}840$ numbers per token instead of $2 \times 80 \times 8{,}192 = 1{,}310{,}720$, an eightfold saving: 1.3 GB instead of 10.7 GB for a sequence of 4,096 tokens.

Most changes to attention since 2019 make the cache smaller still (Figure 44). **Multi-head latent attention**, introduced in DeepSeek-V2 (DeepSeek-AI, 2024a), computes the keys and values of all heads from one vector of 512 numbers per token and block, and caches only that vector, with 64 more numbers for positions. A **sliding window** lets a block read only the last W tokens, so that its cache stops growing at W. And Qwen3.5 replaces three of every four

attention layers by a recurrence whose state is a matrix of fixed size (Yang et al., 2025), a form of the linear attention of §11. For a sequence of 131,072 tokens, at two bytes per number, Llama 3.1 405B keeps 68 GB of cache, against 1.1 TB with full multi-head attention.

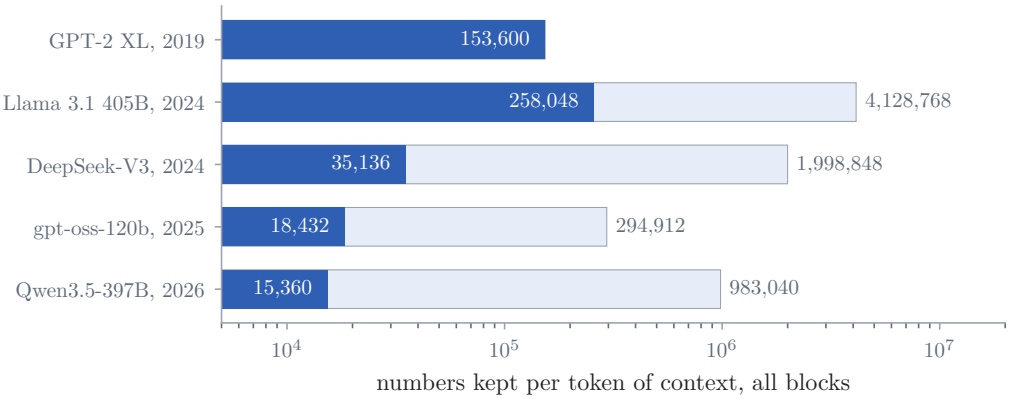


Figure 44. Grouped, compressed, windowed, and recurrent attention shrink the cache by one to two orders of magnitude. Numbers kept in memory per token of context, summed over all blocks, computed from each model's published configuration: keys and values for full attention, the compressed vector for DeepSeek-V3, and only the blocks with full attention for gpt-oss-120b (its windowed blocks keep at most 128 tokens) and Qwen3.5 (its recurrent blocks keep a state of fixed size). Light bars: what full multi-head attention with the same heads would keep. Logarithmic scale.

Except in windowed and recurrent layers, the cache still grows with the text. A model that reads every earlier position pays for it with work and memory that grow with the text, and it spends the same computation on every token, however hard the token is to predict.

11. Strengths, limitations, and variants

Reading every earlier position in one step is what let the Transformer replace recurrent networks, and it is also the source of its limits. Recent variants give the model a state of fixed size that it carries forward, as a recurrent network does, or apply one block several times in a loop.

Strengths

1. **Training is parallel over positions.** One pass computes every prediction of a text (§3), with a fixed number of dependent steps whatever the text's length, as matrix products that parallel hardware computes fast. The 2017 translators trained in a fraction of the time of the recurrent and convolutional systems before them (§7), and this is what made training on trillions of tokens practical.
2. **Any position reads any other in one step, by content.** No information has to pass through a chain of states or a stack of windows (Lecture 3, Section 10). Copying and retrieving from the context come easily: a two-layer Transformer can copy strings of a length exponential in its size, which models with a fixed-size state cannot, and trained Transformers copy and retrieve far better than models of the same size that keep a state of fixed size (Jelassi et al., 2024).
3. **Every token keeps its own vector, and the context is the memory.** Nothing is compressed into a summary of fixed size, the limit of Lecture 3's first encoder–decoders. The key–value cache (§10) is the model's memory of the text so far, and it grows with the text.

Limitations

1. **Attention's work grows with the square of the context, and its cache with the context** (§10). Long contexts are expensive in arithmetic and, above all, in memory.
2. **A fixed amount of computation per token.** One pass applies L blocks, however hard the next token is to predict. Some problems need more sequential steps than a fixed depth provides: for a Transformer of fixed depth and limited numerical precision, one pass can be simulated by circuits of constant depth, which are widely believed unable to evaluate Boolean formulas or to track a long sequence of state changes (Merrill and Sabharwal, 2023); Appendix B states this and related results. On state tracking, trained Transformers find shortcuts that work for the lengths they were trained on and fail beyond them (Liu et al., 2023). A model that writes intermediate tokens before its answer escapes the fixed depth, since each token is a further pass.
3. **Lengths beyond training.** Position encodings meet angles or offsets they never saw (§9), and models trained on short examples of a task often fail on longer ones, at any scale (Anil et al., 2022).
4. **Using a long context well.** Being able to read every position is not the same as using it: on question answering over many documents, models did best when the relevant document came first or last, and worst when it was in the middle (Liu et al., 2024).
5. **No memory beyond the context.** What the model knows comes from its weights, fixed after training, and from its context, which ends; nothing carries over from one context to the next.

Variants

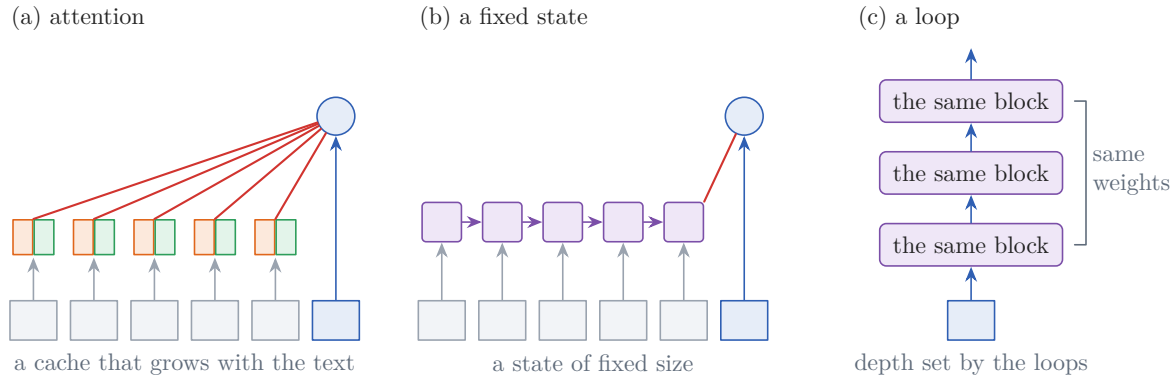


Figure 45. Attention keeps every past position; a recurrence keeps a state of fixed size; a loop applies one block several times. Computing the newest token's vector (top right). (a) Attention reads every earlier position's key and value: a cache that grows with the text. (b) A linear-attention or state-space layer reads and updates a state of fixed size (purple). (c) A looped Transformer reapplies one block with the same weights.

Two families of variants each answer one of the first two limits (Figure 45). The first keeps a state of fixed size, as recurrent networks did. Replace the exponential in the attention weights by a product of feature vectors, $\exp(\mathbf{q}\mathbf{k}^\top) \approx \varphi(\mathbf{q})\varphi(\mathbf{k})^\top$, for a map φ with positive values; then $\varphi(\mathbf{q}_t)$ moves out of the sums of causal attention,

$$\mathbf{c}_t = \frac{\varphi(\mathbf{q}_t)\, S_t}{\varphi(\mathbf{q}_t)\, \mathbf{z}_t^\top}, \qquad S_t = S_{t-1} + \varphi(\mathbf{k}_t)^\top \mathbf{v}_t, \quad \mathbf{z}_t = \mathbf{z}_{t-1} + \varphi(\mathbf{k}_t),$$

and attention becomes a recurrent network with constant memory and work per generated token, still trained in parallel over positions (Katharopoulos et al., 2020). Later layers of this kind let

the state decay with distance, the real eigenvalues of §9 (RetNet, Sun et al., 2023), or let the input decide what to keep and forget, as an LSTM's gates did (Mamba, Gu and Dao, 2024; the gated delta rule, Yang et al., 2025). A fixed state brings back the bottleneck of Lecture 3 (Section 5): a state of b bits can reliably copy at most about $b/\log_2|\mathcal{V}|$ tokens (Jelassi et al., 2024). So recent large models mix the two: in Jamba one block in eight is attention (Lieber et al., 2024), in Qwen3.5 one in four (Qwen Team, 2026).

The second applies one block several times, which lifts the limit of a fixed depth. The **Universal Transformer** (Dehghani et al., 2019) loops one block with a learned rule that decides at each position when to stop. Trained looped models do well where depth matters more than parameters: on reasoning tasks, a block of k layers looped L times nearly matches kL distinct layers (Saunshi et al., 2025), and a 3.5-billion-parameter language model did better with more loops at test time (Geiping et al., 2025). Whether loops, recurrence, or longer written reasoning is the better way to spend computation at test time is open.

12. Summary, and next week

- **The toy and its gaps** (§1). Lecture 3's translator built from attention alone reaches 1.7 BLEU on Multi30k: it cannot tell word orders apart, and it repeats itself because it does not know where it is in its translation.
- **Positions** (§2): a vector for each position, added to each word's vector, from which learned maps W_Q, W_K, W_V compute queries, keys, and values. Sinusoids make an offset a fixed rotation. Together with the learned maps, positions made the largest step of the build, to 39.3 BLEU.
- **Self-attention** (§3), $\mathrm{softmax}(QK^\top/\sqrt{d_k}+M)\,V$: the positions of one sentence read one another, all at once; the causal mask lets one pass train every prediction without any of them seeing its answer, and each read is added to the vector it came from.
- **The block, and a stack of blocks** (§§4–6): several heads read in different ways at the cost of one; attention communicates between positions and the MLP computes at each; stacked blocks let later reads use what earlier ones found; layer normalization keeps the vectors' size in check, and residual connections make the stack trainable. With its training recipe (§7), the translator reaches 53.2 BLEU, against Lecture 3's 49.2 for an LSTM with attention.
- **The decoder alone** (§8) is a language model; a correct implementation passes two tests: changing the future changes nothing earlier, and one pass agrees with one prefix at a time. **Rotary positions** (§9) follow from asking scores to depend only on offsets.
- **Costs** (§10): about $12Ld^2$ parameters in the blocks; $6N$ FLOPs per token in training; attention's work grows with the square of the context, and the key–value cache with the context.
- **Strengths, limitations, and variants** (§11): one pass trains every prediction, and any position reads any other directly; but every token gets the same fixed depth, long contexts are costly, and lengths beyond training generalize poorly. Recurrent layers with a fixed state and looped blocks address some of these, at a price of their own.

Next week. Lecture 5 turns the decoder of §8 into a **base model** trained on a large corpus: how text becomes tokens (byte-pair encoding, which produced the word pieces of §7), the data, the optimizer and its schedule, how large a model should be for a compute budget $C \approx 6ND$ (scaling laws), and what a base model can already do without further training, beginning with the induction heads of Appendix B. A base model continues text; it does not answer. So we will

also turn it into a small chat assistant, by **supervised fine-tuning** on conversations between a user and an assistant.

Background reading

The basics of this lecture are developed in full elsewhere: in the chapters of the Background Notes on Machine Learning and AI, standard textbooks, and courses and videos:

Topic	Background Notes	Textbook, course, or video
The Transformer	DL 9	Jurafsky and Martin, *Transformers*; Prince, *Understanding Deep Learning*, Chapter 12; *Dive into Deep Learning*, Chapter 11; 3Blue1Brown, *Deep Learning*, Chapters 5, 6, and 7 (videos)
Residual connections and normalization	DL 4	Prince, *Understanding Deep Learning*, Chapter 11
Adam, learning-rate schedules, initialization	DL 2, DL 3	Goodfellow, Bengio, and Courville, Chapter 8
Transformer language models	NLP & LLMs 4	Karpathy, *Let's build GPT* (video)
Architectures since 2017, state-space models		Raschka, *The Big LLM Architecture Comparison*
Circuit complexity (Appendix B)		Arora and Barak, *Computational Complexity: A Modern Approach*, Chapters 6 and 14

Vaswani et al. (2017) is worth reading in full, and the Annotated Transformer goes through it line by line in code. Phuong and Hutter (2022) state every variant of §§3–8 as precise pseudocode. Elhage et al. (2021) is the source of the residual-stream view of §6, and Puranik (2026) of the argument for rotations in §9. Sanderson (2024a, b, c) pictures the Transformer geometrically, and Figures 9, 13, 17, 25, 41, and 46 follow him; he writes vectors as columns, so his matrices are the transposes of the ones here. On why normalization helps, see Santurkar et al. (2018), on batch normalization (Ioffe and Szegedy, 2015), and Xu et al. (2019); on residual networks as ensembles of shallower ones, Veit et al. (2016); on features stored as nearly perpendicular directions, Elhage et al. (2022).

References and further reading

The Transformer and its parts

- **Ashish Vaswani et al. (2017), Attention Is All You Need.** Paper. Version 5 (December 2017) added the contribution note and the 41.8 of §7. The authors' code: tensor2tensor.
- **Denny Britz, Anna Goldie, Minh-Thang Luong, and Quoc Le (2017), Massive Exploration of Neural Machine Translation Architectures.** Paper. Additive against unscaled multiplicative attention.
- **Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun (2016a), Deep Residual Learning for Image Recognition.** Paper. And **(2016b), Identity Mappings in Deep Residual Networks.** Paper.
- **Andreas Veit, Michael Wilber, and Serge Belongie (2016), Residual Networks Behave Like Ensembles of Relatively Shallow Networks.** Paper.
- **Yihe Dong, Jean-Baptiste Cordonnier, and Andreas Loukas (2021), Attention is Not All You Need: Pure Attention Loses Rank Doubly Exponentially with Depth.** Paper.
- **Sergey Ioffe and Christian Szegedy (2015), Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift.** Paper.
- **Shibani Santurkar, Dimitris Tsipras, Andrew Ilyas, and Aleksander Madry (2018), How Does Batch Normalization Help Optimization?** Paper.
- **Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton (2016), Layer Normalization.** Paper.
- **Jingjing Xu, Xu Sun, Zhiyuan Zhang, Guangxiang Zhao, and Junyang Lin (2019), Understanding and Improving Layer Normalization.** Paper.
- **Shaked Brody, Uri Alon, and Eran Yahav (2023), On the Expressivity Role of LayerNorm in Transformers' Attention.** Paper.
- **Sheng Shen, Zhewei Yao, Amir Gholami, Michael W. Mahoney, and Kurt Keutzer (2020), PowerNorm: Rethinking Batch Normalization in Transformers.** Paper.
- **Jiachen Zhu, Xinlei Chen, Kaiming He, Yann LeCun, and Zhuang Liu (2025), Transformers without Normalization.** Paper.
- **Ruibin Xiong et al. (2020), On Layer Normalization in the Transformer Architecture.** Paper. Pre-norm and post-norm.
- **Toan Q. Nguyen and Julian Salazar (2019), Transformers without Tears: Improving the Normalization of Self-Attention.** Paper.
- **Ofir Press and Lior Wolf (2017), Using the Output Embedding to Improve Language Models.** Paper.
- **Hakan Inan, Khashayar Khosravi, and Richard Socher (2017), Tying Word Vectors and Word Classifiers: A Loss Framework for Language Modeling.** Paper.
- **Noam Shazeer and Mitchell Stern (2018), Adafactor: Adaptive Learning Rates with Sublinear Memory Cost.** Paper. Section 8.1 explains the factor $\sqrt{d}$ on the token vectors.
- **Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy (2021), Transformer Feed-Forward Layers Are Key-Value Memories.** Paper.
- **Dan Hendrycks and Kevin Gimpel (2016), Gaussian Error Linear Units (GELUs).** Paper.

- **Diederik P. Kingma and Jimmy Ba (2015), Adam: A Method for Stochastic Optimization.** Paper.
- **Liyuan Liu, Haoming Jiang, Pengcheng He, Weizhu Chen, Xiaodong Liu, Jianfeng Gao, and Jiawei Han (2020), On the Variance of the Adaptive Learning Rate and Beyond.** Paper.
- **Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna (2016), Rethinking the Inception Architecture for Computer Vision.** Paper. Label smoothing, Section 7.
- **Greg Yang et al. (2021), Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer.** Paper. μP; dividing scores by d_k, Appendix B.

Positions

- **Jianlin Su et al. (2021), RoFormer: Enhanced Transformer with Rotary Position Embedding.** Paper. In *Neurocomputing* in 2024.
- **Alok Puranik (2026), Using Group Theory to Explore the Space of Positional Encodings for Attention.** Jane Street Tech Blog, 22 April 2026. Post, accessed 20 October 2026. The one-parameter group of §9.
- **Ofir Press, Noah A. Smith, and Mike Lewis (2022), Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation.** Paper.
- **Adi Haviv, Ori Ram, Ofir Press, Peter Izsak, and Omer Levy (2022), Transformer Language Models without Positional Encodings Still Learn Positional Information.** Paper.
- **Shouyuan Chen, Sherman Wong, Liangjian Chen, and Yuandong Tian (2023), Extending Context Window of Large Language Models via Positional Interpolation.** Paper.
- **Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole (2024), YaRN: Efficient Context Window Extension of Large Language Models.** Paper.

The decoder alone, and encoders

- **Peter J. Liu et al. (2018), Generating Wikipedia by Summarizing Long Sequences.** Paper. An early decoder-only Transformer, for summarization.
- **Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever (2018), Improving Language Understanding by Generative Pre-Training.** Report. GPT.
- **Alec Radford et al. (2019), Language Models are Unsupervised Multitask Learners.** Report. GPT-2; the corrected parameter counts are in the code repository.
- **Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova (2019), BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.** Paper.
- **Colin Raffel et al. (2020), Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer.** Paper. T5; Section 3.2 compares the arrangements of Figure 32.
- **Tom B. Brown et al. (2020), Language Models are Few-Shot Learners.** Paper. GPT-3.
- **Jared Kaplan et al. (2020), Scaling Laws for Neural Language Models.** Paper. Parameter and FLOP counts, Section 2.1.

Inside trained Transformers

- **Kevin Clark, Urvashi Khandelwal, Omer Levy, and Christopher D. Manning (2019), What Does BERT Look At? An Analysis of BERT's Attention.** Paper.
- **Elena Voita, David Talbot, Fedor Moiseev, Rico Sennrich, and Ivan Titov (2019), Analyzing Multi-Head Self-Attention: Specialized Heads Do the Heavy Lifting, the Rest Can Be Pruned.** Paper.
- **Paul Michel, Omer Levy, and Graham Neubig (2019), Are Sixteen Heads Really Better than One?** Paper.
- **Kawin Ethayarajh (2019), How Contextual are Contextualized Word Representations? Comparing the Geometry of BERT, ELMo, and GPT-2 Embeddings.** Paper.
- **Nelson Elhage et al. (2021), A Mathematical Framework for Transformer Circuits.** Article. The residual stream; induction heads in two-layer models.
- **Nelson Elhage et al. (2022), Toy Models of Superposition.** Article.
- **Catherine Olsson et al. (2022), In-context Learning and Induction Heads.** Article.
- **Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov (2022), Locating and Editing Factual Associations in GPT.** Paper.
- **Peter Hase, Mohit Bansal, Been Kim, and Asma Ghandeharioun (2023), Does Localization Inform Editing? Surprising Differences in Causality-Based Localization vs. Knowledge Editing in Language Models.** Paper.

Costs

- **Noam Shazeer (2019), Fast Transformer Decoding: One Write-Head is All You Need.** Paper. Multi-query attention.
- **Joshua Ainslie et al. (2023), GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints.** Paper.
- **Hugo Touvron et al. (2023), Llama 2: Open Foundation and Fine-Tuned Chat Models.** Paper.
- **Andrej Karpathy (2024), Reproducing GPT-2 (124M) in llm.c in 90 Minutes for $20.** Post.
- **PyTorch documentation, torch.nn.functional.scaled_dot_product_attention.** Documentation. Its reference formulation computes the full matrix of scores, masks it, and multiplies the full matrix of weights by the values.
- **Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré (2022), FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.** Paper.

Since 2017

- **Biao Zhang and Rico Sennrich (2019), Root Mean Square Layer Normalization.** Paper.
- **Noam Shazeer (2020), GLU Variants Improve Transformer.** Paper.
- **Noam Shazeer et al. (2017), Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer.** Paper.

- **Aakanksha Chowdhery et al. (2023), PaLM: Scaling Language Modeling with Pathways.** Paper.
- **Mostafa Dehghani et al. (2023), Scaling Vision Transformers to 22 Billion Parameters.** Paper. Normalizing queries and keys.
- **Mitchell Wortsman et al. (2024), Small-Scale Proxies for Large-Scale Transformer Training Instabilities.** Paper.
- **DeepSeek-AI (2024a), DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model.** Paper. Multi-head latent attention.
- **Llama Team, AI @ Meta (2024), The Llama 3 Herd of Models.** Paper.
- **DeepSeek-AI (2024b), DeepSeek-V3 Technical Report.** Paper.
- **Qwen Team (2025), Qwen3 Technical Report.** Paper.
- **OLMo Team (2025), 2 OLMo 2 Furious.** Paper.
- **OpenAI (2025), gpt-oss-120b & gpt-oss-20b Model Card.** Paper.
- **Qwen Team (2026), Qwen3.5-397B-A17B.** Model card and configuration, accessed 20 October 2026.
- **Sebastian Raschka (2025, updated 2026), The Big LLM Architecture Comparison.** Article. And A Dream of Spring for Open-Weight LLMs: 10 Architectures from Jan-Feb 2026 (25 February 2026). Both accessed 20 October 2026.

Limitations and variants (§11 and Appendix B)

- **Gail Weiss, Yoav Goldberg, and Eran Yahav (2021), Thinking Like Transformers.** Paper. RASP.
- **Michael Hahn (2020), Theoretical Limitations of Self-Attention in Neural Sequence Models.** Paper.
- **David Chiang and Peter Cholak (2022), Overcoming a Theoretical Limitation of Self-Attention.** Paper.
- **William Merrill and Ashish Sabharwal (2023), The Parallelism Tradeoff: Limitations of Log-Precision Transformers.** Paper.
- **David A. Mix Barrington (1989), Bounded-Width Polynomial-Size Branching Programs Recognize Exactly Those Languages in NC^1.** Paper.
- **Lena Strobl, William Merrill, Gail Weiss, David Chiang, and Dana Angluin (2024), What Formal Languages Can Transformers Express? A Survey.** Paper.
- **Bingbin Liu, Jordan T. Ash, Surbhi Goel, Akshay Krishnamurthy, and Cyril Zhang (2023), Transformers Learn Shortcuts to Automata.** Paper.
- **Cem Anil et al. (2022), Exploring Length Generalization in Large Language Models.** Paper.
- **Nelson F. Liu et al. (2024), Lost in the Middle: How Language Models Use Long Contexts.** Paper.
- **Samy Jelassi, David Brandfonbrener, Sham M. Kakade, and Eran Malach (2024), Repeat After Me: Transformers are Better than State Space Models at Copying.** Paper.
- **Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret (2020), Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention.** Paper.

- **Yutao Sun et al. (2023), Retentive Network: A Successor to Transformer for Large Language Models.** Paper.
- **Albert Gu and Tri Dao (2024), Mamba: Linear-Time Sequence Modeling with Selective State Spaces.** Paper. On arXiv in December 2023.
- **Songlin Yang, Jan Kautz, and Ali Hatamizadeh (2025), Gated Delta Networks: Improving Mamba2 with Delta Rule.** Paper.
- **Opher Lieber et al. (2024), Jamba: A Hybrid Transformer-Mamba Language Model.** Paper.
- **Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser (2019), Universal Transformers.** Paper.
- **Angeliki Giannou, Shashank Rajput, Jy-yong Sohn, Kangwook Lee, Jason D. Lee, and Dimitris Papailiopoulos (2023), Looped Transformers as Programmable Computers.** Paper.
- **Nikunj Saunshi, Nishanth Dikkala, Zhiyuan Li, Sanjiv Kumar, and Sashank J. Reddi (2025), Reasoning with Latent Thoughts: On the Power of Looped Transformers.** Paper.
- **Jonas Geiping et al. (2025), Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach.** Paper.

Code, data, and explanations

- **Andrej Karpathy (2023), Let's build GPT: from scratch, in code, spelled out.** Video. And the code, nanoGPT.
- **Grant Sanderson (2024a), Transformers, the Tech behind LLMs.** 3Blue1Brown, *Deep Learning*, Chapter 5. Lesson and video. The token's vector moved by the blocks; the running tally of GPT-3's weights.
- **Grant Sanderson (2024b), Attention in Transformers, Step-by-Step.** Chapter 6. Lesson and video. Queries and keys as arrows; the value map as a map down and back up.
- **Grant Sanderson (2024c), How Might LLMs Store Facts.** Chapter 7. Lesson and video. The MLP one hidden unit at a time; nearly perpendicular directions and superposition.
- **Alexander Rush (2018), The Annotated Transformer.** Article, updated in 2022 by Austin Huang et al.
- **Mary Phuong and Marcus Hutter (2022), Formal Algorithms for Transformers.** Paper.
- **Desmond Elliott, Stella Frank, Khalil Sima'an, and Lucia Specia (2016), Multi30K: Multilingual English-German Image Descriptions.** Paper. The captions of the translators of §§1–7.
- **Ronen Eldan and Yuanzhi Li (2023), TinyStories: How Small Can Language Models Be and Still Speak Coherent English?** Paper. The corpus of the language models in this lecture.

Appendix A: Derivations

A.1 Dot products and the softmax

The variance of a dot product. Let $q_1, \dots, q_d$ and $k_1, \dots, k_d$ be independent, each with mean 0 and variance 1. Each product $q_i k_i$ has mean $\mathbb{E}[q_i]\,\mathbb{E}[k_i] = 0$ and variance $\mathbb{E}[q_i^2]\,\mathbb{E}[k_i^2] = 1$, and the products are independent, so $\mathbf{q}\mathbf{k}^\top$ has mean 0 and variance d. For the scores of one query against many keys, the same holds for each score, and the scores are uncorrelated.

The softmax's derivative. With $\alpha_i = e^{e_i} / \sum_j e^{e_j}$,

$$\frac{\partial \alpha_i}{\partial e_j} = \alpha_i \delta_{ij} - \alpha_i \alpha_j, \qquad \text{that is,} \qquad \frac{\partial \alpha}{\partial \mathbf{e}} = \operatorname{diag}(\alpha) - \alpha^\top \alpha$$

for a row α. This matrix is symmetric and positive semidefinite, its rows sum to zero (adding a constant to all scores changes nothing), and its diagonal entries are $\alpha_i(1-\alpha_i) \le \frac{1}{4}$. If one weight is $1-\varepsilon$, every entry is $O(\varepsilon)$: a saturated softmax passes back almost no gradient to the scores, and so to the queries and keys. Its largest entry, $\frac{1}{4}$, occurs for two equal weights of $\frac{1}{2}$, where the matrix's Frobenius norm, the size plotted in Figure 11b, is $\frac{1}{2}$; for uniform weights over m keys the norm is about $1/\sqrt{m}$, so it is small at both extremes.

A.2 Sinusoidal and rotary positions

Offsets are rotations. For one frequency ω, write the pair of coordinates of position t as $\mathbf{r}_t = (\sin t\omega,\ \cos t\omega)$. By the addition formulas,

$$\mathbf{r}_{t+u} = (\sin(t\omega + u\omega),\ \cos(t\omega + u\omega)) = \mathbf{r}_t \begin{pmatrix} \cos u\omega & -\sin u\omega \\ \sin u\omega & \cos u\omega \end{pmatrix},$$

a rotation by $u\omega$ that does not depend on t. Applying it to every pair, $\mathbf{p}_{t+u} = \mathbf{p}_t R(u)$ for a block-diagonal rotation $R(u)$, so a linear map in the model can move from a position to the one u places away. The dot product of two pairs is $\sin t\omega \sin j\omega + \cos t\omega \cos j\omega = \cos((t-j)\omega)$, so $\mathbf{p}_t \mathbf{p}_j^\top = \sum_i \cos((t-j)\omega_i)$, a function of $t-j$ alone, equal to $d/2$ at $t = j$ (Figure 5b).

Rotary embeddings. Treat each pair of coordinates (x_{2i}, x_{2i+1}) of a query or key as a complex number $x_{2i} + \mathrm{i}\, x_{2i+1}$. Rotary embeddings multiply the pair of a query at position t by $e^{\mathrm{i}t\omega_i}$ and of a key at position j by $e^{\mathrm{i}j\omega_i}$. The dot product of two pairs is the real part of one times the conjugate of the other, so the pair's contribution to the score is

$$\operatorname{Re}(q\, e^{\mathrm{i}t\omega_i}\, \overline{k\, e^{\mathrm{i}j\omega_i}}) = \operatorname{Re}(q\, \bar{k}\, e^{\mathrm{i}(t-j)\omega_i}),$$

which depends on the positions only through $t-j$. For a query and a key with equal pairs, the score is $\sum_i |q_i|^2 \cos((t-j)\omega_i)$, which oscillates and, averaged over the frequencies, tends to fall with distance: a mild preference for nearby tokens that the model can override.

The one-parameter group of §9. Let position t act on queries by an invertible matrix F_t and position j on keys by an invertible matrix G_j, and suppose that $F_t G_j^\top = B(j-t)$ for every t and j, with $B(0) = I$. Taking $j = 0$ gives $F_t = B(-t)\, G_0^{-\top}$, and taking $t = 0$ gives $G_j^\top = F_0^{-1} B(j)$. Substituting both,

$$B(j-t) = F_t G_j^\top = B(-t)\, G_0^{-\top} F_0^{-1}\, B(j) = B(-t)\, (F_0 G_0^\top)^{-1} B(j) = B(-t)\, B(j),$$

since $F_0 G_0^\top = B(0) = I$. With $a = -t$ and $b = j$, this is $B(a+b) = B(a)B(b)$ for all integers a and b: the matrices form a one-parameter group. If B extends continuously to real offsets, then $B(u) = \exp(uX)$ with $X = \frac{d}{du} B(u)\big|_{u=0}$, a standard fact about such groups. When X can be

diagonalized over the complex numbers, its eigenvalues give the cases of §9; a pair of complex eigenvalues $a \pm \mathrm{i}\omega$ contributes, on the plane spanned by the real and imaginary parts of its eigenvectors, a two-dimensional block $e^{ua}\begin{pmatrix}\cos u\omega & -\sin u\omega \\ \sin u\omega & \cos u\omega\end{pmatrix}$, a rotation by $u\omega$ scaled by e^{ua}. Rotary embeddings take $F_t = G_t = R_t$, so that $F_t G_j^\top = R_{t-j}$, a pure rotation of every pair. Matrices X that cannot be diagonalized give factors polynomial in u; Puranik (2026) found no use of them, and shows that ALiBi, a bias proportional to the offset (Press et al., 2022), arises from one of them after extending the queries and keys by one coordinate each.

A.3 Counting operations

Let a layer compute $Y = XW$ with $X \in \mathbb{R}^{n\times a}$ (one row per token) and $W \in \mathbb{R}^{a\times b}$: nab multiplications and about as many additions, $2nab$ FLOPs, which is 2 FLOPs per weight per token. In the backward pass, given $G = \partial\mathcal{L}/\partial Y$, the gradient with respect to the input is $GW^\top$ and with respect to the weights $X^\top G$, two products of the same size, $4nab$ FLOPs. Per token and weight, forward and backward together cost $2 + 4 = 6$ FLOPs, hence $C \approx 6N$ per token. Attention adds, per block and token, $2Td$ FLOPs for the scores and $2Td$ for the weighted average when T positions are computed, and twice that in the backward pass. A straightforward implementation computes all T positions of a sequence for every token and masks the scores afterwards; one that skips the masked triangle computes t positions for the token at position t, $T/2$ on average (§10). The softmax, the normalizations, and the additions cost operations in proportion to the activations, not the weights, and are usually left out of the count.

A.4 Nearly perpendicular directions

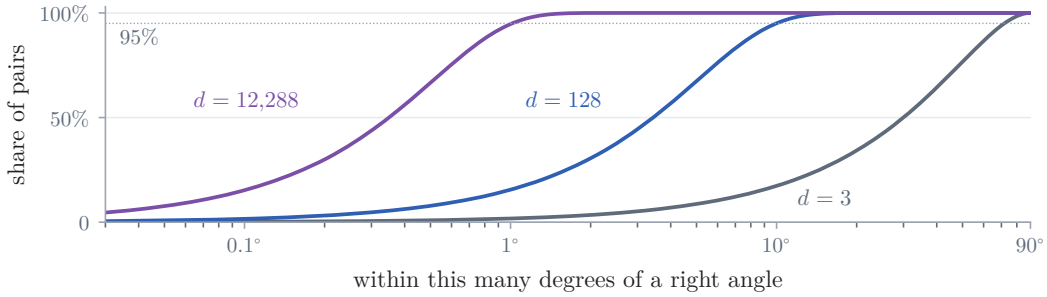


Figure 46. In many dimensions, almost every pair of random directions is nearly perpendicular. For 1,000 random directions in d dimensions, the share of their 499,500 pairs whose angle is within a given number of degrees of a right angle, for $d = 3$, $d = 128$ (the word model of §8), and $d = 12{,}288$ (GPT-3). The horizontal axis is logarithmic. After Sanderson (2024c), who makes this point about the MLP.

Draw two directions at random in d dimensions, as unit vectors $\mathbf{u}$ and $\mathbf{v}$ obtained by normalizing vectors with independent standard normal coordinates. By symmetry, the cosine of the angle between them, $\mathbf{u}\mathbf{v}^\top$, is distributed like one coordinate of a random unit vector: its mean is 0, and since the squares of the d coordinates sum to 1 and are identically distributed, its variance is $1/d$. The angle therefore differs from a right angle by about $1/\sqrt{d}$ radians, or $57.3^\circ/\sqrt{d}$: about 5° for $d = 128$ and 0.5° for $d = 12{,}288$. Of the measured pairs, 95.1% are within 10°, two standard deviations, of a right angle for $d = 128$, and 94.7% within 1° for $d = 12{,}288$ (Figure 46).

For a fixed tolerance, the number of directions that can be pairwise within it of a right angle grows exponentially with d: among 70 billion random directions in 12,288 dimensions, fewer than one pair is expected to be further than 5° from a right angle.

Appendix B: What a fixed depth can compute (optional)

A Transformer reads by content, and it applies a fixed number of blocks to all positions at once; both shape what it can compute. The results below answer different questions, which are easy to conflate: whether some setting of the weights computes a function correctly for inputs of every length (**expressibility**); whether such a model can also be confident, that is, reach a low log loss; whether training finds such weights (**learnability**); and whether weights trained on short inputs work on longer ones. Mathematical results mostly answer the first two, for idealized models whose assumptions matter; experiments answer the last two.

B.1 Programs written in attention

Weiss et al. (2021) defined a small programming language, RASP, whose operations correspond to the parts of a Transformer. A program manipulates sequences as long as the input: the tokens, the positions (`indices`), and sequences computed from them. Elementwise operations, such as `length` $-$ `indices` $-\,1$, apply a function at every position, as the MLP does. Two operations correspond to attention:

- `select(keys, queries, predicate)` builds a selection matrix, whose entry (t, j) is true when *predicate*(key at j, query at t) holds: an attention pattern;
- `aggregate(selection, values)` gives each position t the average of the values at the positions it selects: the weighted average of attention, with equal weights on the selected positions.

A third, `selector_width(selection)`, gives each position the number of positions it selects. The reversal task of Lecture 3 (Section 5), which an encoder–decoder without attention failed on long strings, is four lines:

$$\texttt{length} = \texttt{selector_width}(\texttt{select}(\texttt{tokens}, \texttt{tokens}, \mathit{true}))$$
$$\texttt{opposite} = \texttt{length} - \texttt{indices} - 1$$
$$\texttt{flip} = \texttt{select}(\texttt{indices}, \texttt{opposite}, =)$$
$$\texttt{reverse} = \texttt{aggregate}(\texttt{flip}, \texttt{tokens})$$

Each `aggregate` that depends on the result of another needs a further layer, so a RASP program also predicts the number of layers a Transformer needs. Weiss et al. (2021) trained Transformers of the predicted sizes on such tasks and reached nearly perfect accuracy. A program shows that weights exist; that training found weights of the predicted size answers the question of learnability, for these tasks and lengths. One such program, two heads in two layers that copy what followed an earlier occurrence of the current token, has been found in trained language models (Elhage et al., 2021; Olsson et al., 2022); Lecture 5 takes it up.

B.2 What one pass cannot do

A Transformer applies L blocks to all positions in parallel, and nothing in one forward pass is iterated more than L times. That is what removed the sequential computation of recurrence; it is also a limit, made precise by two different arguments.

Confidence. The **parity** of a string of bits is 1 if it contains an odd number of ones; flipping any single bit flips it. Hahn (2020) showed that for a Transformer with fixed weights, soft attention with bounded scores, and Lipschitz layers, whose outputs change at most in proportion to their inputs (which needs normalizations with $\epsilon > 0$), changing one bit of an input of length n changes the output of one pass by $O(1/n)$, up to logarithmic factors with

a causal mask: no position can claim more than a constant multiple of $1/n$ of a read. So on uniformly random strings its cross-entropy on parity is at least about one bit, minus a term that vanishes as n grows. This bounds confidence, not correctness: Chiang and Cholak (2022) built a Transformer with fixed weights that gets parity right at every length, with probabilities that approach one half, or as high as wished if layer normalization has $\epsilon = 0$.

Exact computation. A **circuit family** computes a function of inputs of each length n with a circuit of gates; its **depth** is the number of steps that must happen in sequence even with unlimited parallel hardware. TC^0 is the class of functions computed by circuits of constant depth and polynomial size with AND, OR, NOT, and majority gates that may take any number of inputs; NC^1 allows depth $O(\log n)$ with gates of two inputs. $\mathrm{TC}^0 \subseteq \mathrm{NC}^1$, and whether they are equal is open; most researchers believe they are not. Merrill and Sabharwal (2023) proved that a Transformer of fixed depth whose numbers have $O(\log n)$ bits of precision can be simulated, in one pass, by uniform TC^0 circuits (circuits that a simple procedure can write out for every length): each block at each position is a constant-depth circuit, since sums of n numbers, products, divisions, and exponentials to that precision all are. So, unless $\mathrm{TC}^0 = \mathrm{NC}^1$, such Transformers cannot in one pass evaluate Boolean formulas, or compose a sequence of permutations of five objects, a problem that Barrington (1989) showed to be as hard as anything in NC^1; tracking the state of a finite automaton through a sequence of moves is in general a problem of this kind. The argument says nothing against parity, which is in TC^0: it is a different obstacle from Hahn's, about exact computation rather than confidence. Both are statements about the worst case and about growing inputs.

Both arguments concern one pass; intermediate tokens and loops escape that assumption (§11).

Appendix C: Our small Transformer, against the LSTMs

With the recipe of §7, the three-block translator of §§4–6 becomes a small Transformer of 2017, with 5,475,840 parameters once the French table is shared. Its validation cross-entropy rises from 1.08 to 1.12, as label smoothing is expected to do (in Vaswani et al.'s ablation, perplexity 4.92 with it and 4.67 without), and its BLEU rises from 52.7 to 53.2, a small difference for single runs. Against Lecture 3's translators, trained on the same data for the same number of steps and decoded in the same way:

Translator	Parameters	Training time	Validation cross-entropy	Test BLEU
LSTM (Lecture 3)	4.0 million	9 min	2.01	19.5
LSTM with attention (Lecture 3)	4.4 million	18 min	1.20	49.2
Transformer, 3 blocks a side	5.5 million	17 min	1.12	53.2

Training times were measured on two processor cores, shared with other work during the Transformer's run. The Transformer trails the LSTM with attention in cross-entropy for most of training, passes it between 2,250 and 2,500 steps (Figure 47a), and ends ahead in both measures. Its cross-attention finds alignments of its own: one head of the last decoder layer reads *uniform* when predicting *tenue* (0.96, against 0.66 for the LSTM with attention) and goes back to *karate* for *karaté*, the reordering of Lecture 3 (Figure 47b). Better on average is not better on every caption: greedily, it translates the karate caption as *une fille en tenue de karaté fait un bâton avec un ballon* ., "a girl in karate uniform makes a stick with a ball", and loses the kick.

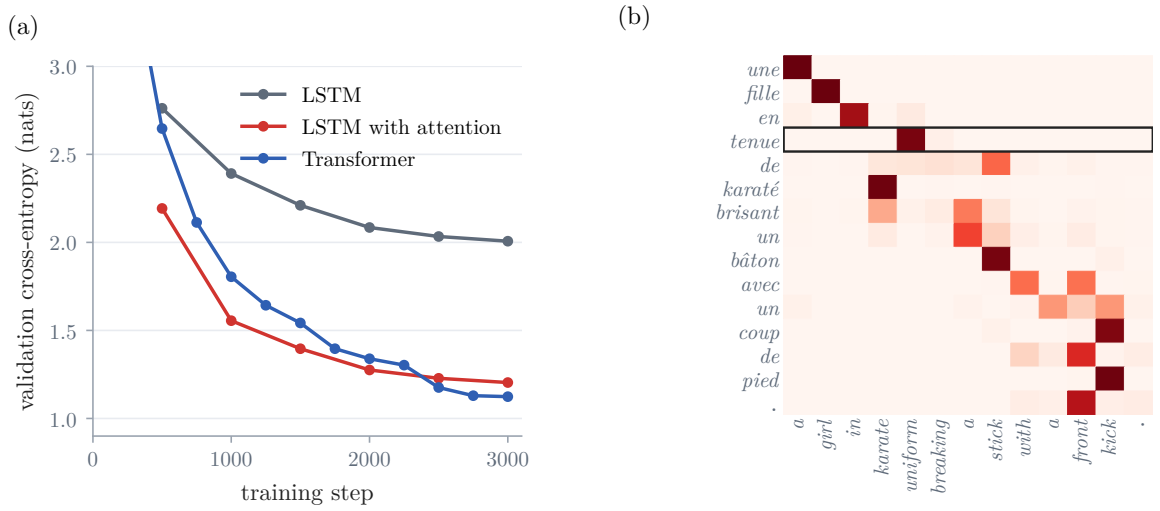


Figure 47. The Transformer translates Multi30k better than the LSTMs, and its cross-attention aligns. (a) Cross-entropy per French word on the validation captions during training (nats, without label smoothing), for the LSTM encoder–decoder of Lecture 3 (gray), the same network with attention (red), and the Transformer of this section (blue). (b) Cross-attention weights of one head in the last decoder layer of the Transformer on the karate caption, with the reference translation supplied word by word: each row is one French word, each column one English word, and each row sums to one (darker is more weight). The head is the one whose largest weights agree most often with the attention of Lecture 3's LSTM on this caption; the row of *tenue* is outlined.

Appendix D: Inside the word model

Three looks inside the word model of §8: how its training compares with Lecture 2's LSTM, what one of its heads reads and what removing the head costs, and whether self-attention gives a word different vectors in different contexts.

D.1 Training, against the LSTM

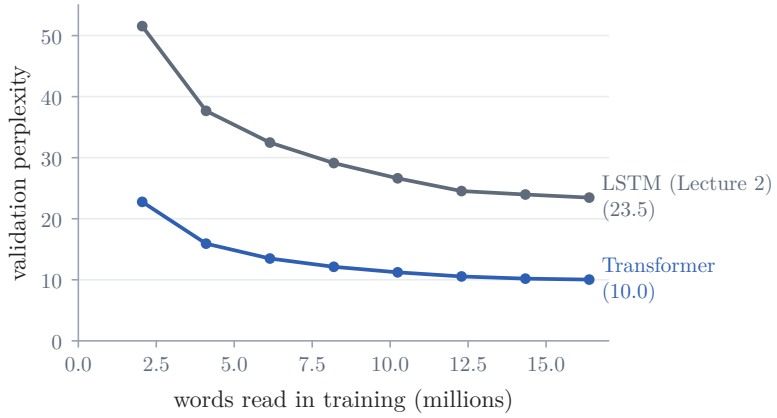


Figure 48. The word Transformer passes the LSTM within an eighth of its training. Validation perplexity of the word LSTM of Lecture 2 (gray) and of the Transformer (blue), on the first 1,000 validation stories every 500 steps, against the number of words read in training; both read 4,096 words per step. The language models to compare against are Lecture 2's, trained on the same stories; Lecture 3's models are translators, compared in Figure 47.

The Transformer passes the LSTM's final validation perplexity after 500 steps, an eighth of the training, and is still improving at the end (Figure 48).

D.2 One head, and what removing it costs

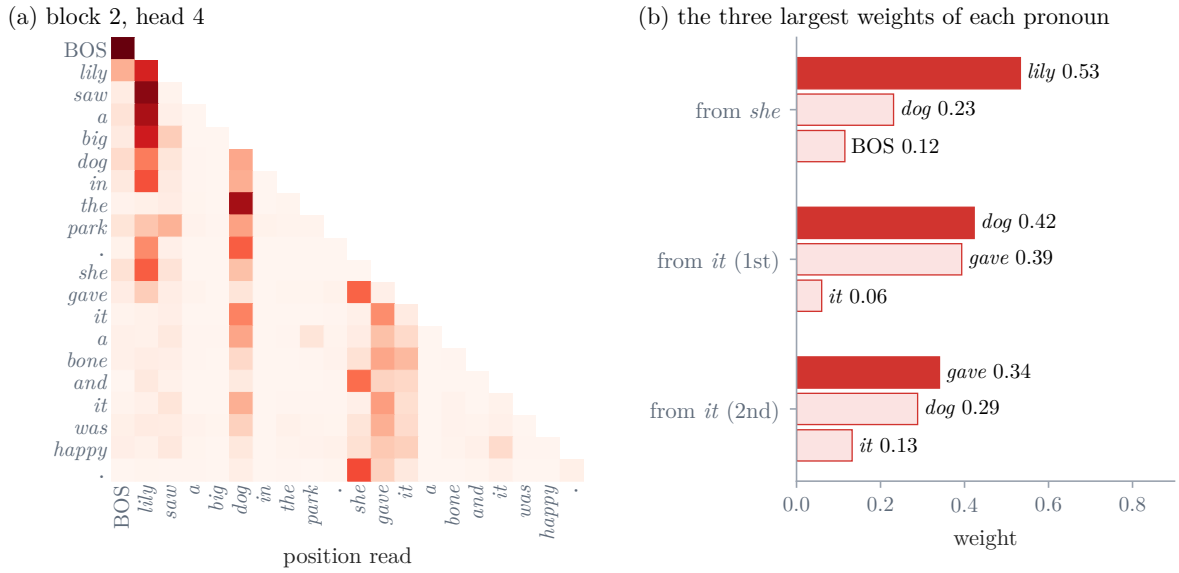


Figure 49. A trained head reads from *she* to *lily*, and from each *it* to *dog* and to *gave*. One head of the word model on the running sentence: of its sixteen heads, the one that puts the most weight on *lily* from *she* and on *dog* from the two occurrences of *it*. (a) Its weights, drawn as in Figure 8. (b) The three largest weights in the rows of *she* and of the two occurrences of *it*.

Figure 49 shows one head of the word model on the running sentence (block 2, head 4). From *she* it puts more than half of its weight on *lily*. From each *it* it divides most of its weight between *dog* and *gave*, and the second *it* puts slightly more on *gave* (Figure 49b). Many other rows read *lily* or *dog* as well: the head reads the characters of the story from many positions, not only from pronouns.

Removing the head tests whether the model relies on it. Remove one head at a time, by setting its read to zero, and measure the increase in log loss on 500 validation stories, over all predictions and over the predictions made at a pronoun (the positions whose input is *she*, *he*, *it*, *they*, *her*, or *him*):

Head removed	**All predictions (nats)**	**At a pronoun (nats)**
the head of Figure 49	0.07	0.05
block 2, head 3	0.16	0.30
any one of the sixteen	0.02 to 0.20	0.01 to 0.30

Every head matters somewhat, and the head of Figure 49 matters less at a pronoun than overall. The head that matters most at a pronoun, another in the same block, has a pattern that has nothing to do with pronouns: it is one of the two heads that put the most weight on the previous token, 0.33 on average over held-out text. A pattern that looks like a relation is not evidence that the model relies on the head for that relation. Whatever the pronoun head contributes may show only later in a story, when the model must say who did what; these measurements average over every prediction.

D.3 One word in two contexts

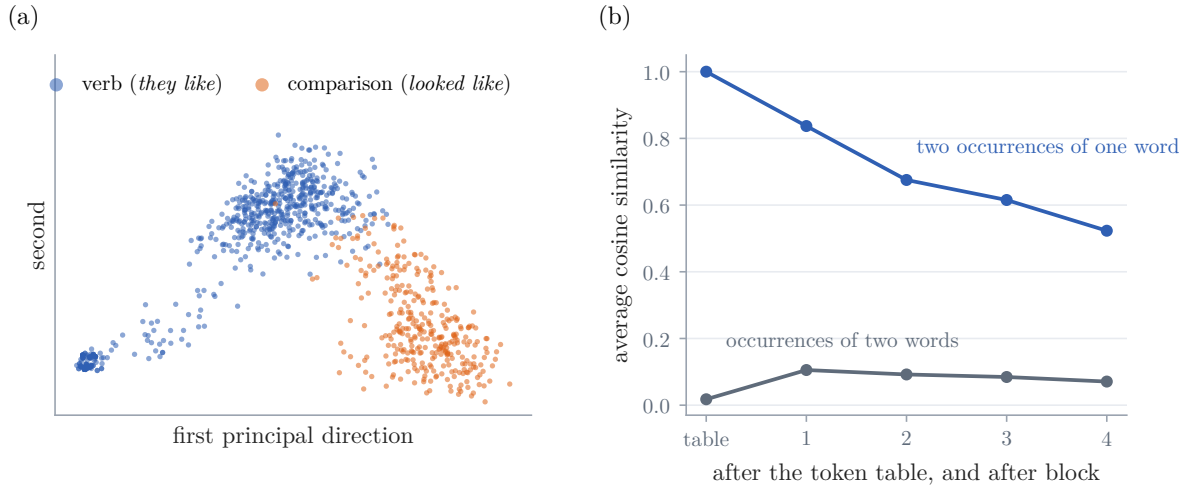


Figure 50. The two uses of *like* get different vectors. (a) The vectors of the residual stream after the last block of the word model at 981 occurrences of *like* in held-out stories, projected onto their two main directions of variation: the verb (*they like*, *didn't like*) and the comparison (*looked like*, *just like*). In the token table all of them are one point. (b) The average cosine similarity between the vectors of two occurrences of the same word, and of two different words, after the token table (0) and after each block, for the 200 most frequent words.

The claim of §3, that self-attention gives a word different vectors in different contexts, can be checked directly. In the children's stories, *like* has two common uses: the verb (*they like to play*, *i don't like it*) and the comparison (*it looked like a bird*, *just like mom*). In the first 255 words of each held-out story, 981 occurrences of *like* follow a word that settles which use it is:

652 verbs, after words such as *they*, *not*, *didn't*, and *you*, and 329 comparisons, after words such as *looked*, *felt*, *just*, and *sounded*. The test has three steps:

1. **Take the vectors.** At each occurrence, take the vector in the residual stream after the token table and after each block. In the token table every occurrence has the same vector, by construction.
2. **Assign each occurrence to a use.** Average the vectors of each use, leaving out the occurrence being assigned, and assign it to the use whose average is nearer. After the first block this is right 99.4% of the time (averaged over the two uses), and the two uses stay apart up to the last block (Figure 50a).
3. **Rule out the previous word.** Split the preceding words into two halves. Compute the averages only from the occurrences that follow words of one half, and assign the occurrences that follow words of the other half. They are still assigned correctly 96% to 98% of the time after every block, so the separation is not just a record of the previous word.

And the difference is the one the model needs: at the top, the verb predicts *to* (0.42 on average), *the*, and *it*, and the comparison *a* (0.38), *the*, and *it*.

Across the vocabulary the same effect appears as a spread (Figure 50b). For each of the 200 most frequent words, the average cosine similarity between the vectors of two of its occurrences is 1 in the token table, and it falls with every block, to 0.52 after the fourth; two occurrences of different words stay much less similar, at most 0.11. Ethayarajh (2019) found the same fall with depth in large pretrained models.

Appendix E: From 2017 to now

The Transformer of 2017 translated single sentences with 65 or 213 million parameters. The largest open models of 2026 have hundreds of billions of parameters and read contexts of hundreds of thousands of tokens, yet little of the block has changed. Most of the changes to the architecture answer a cost or a failure that shows only at scale, and the training recipe changed with them.

E.1 The block

Stable training. Large models are fragile in training: while the largest PaLM model trained, its loss spiked about twenty times, and each time training was restarted from an earlier checkpoint (Chowdhery et al., 2023). The first remedy was pre-norm (§6). **RMSNorm** (Zhang and Sennrich, 2019) drops the mean and the shift from layer normalization, $\gamma \odot \mathbf{h} / \sqrt{\frac{1}{d}\sum_i h_i^2 + \epsilon}$, at lower cost and with equal results. Most models removed the biases from their linear layers. And the queries and keys of each head are often normalized before their dot product: the $\sqrt{d_k}$ of §3 fixes the scale of the scores at initialization, but nothing stops it from growing in training, and when a Transformer for images was scaled up, at about 8 billion parameters the scores grew until the softmax put almost all its weight on one key and training diverged; normalizing queries and keys let the authors train one of 22 billion (Dehghani et al., 2023). The same instability, and the same remedy, appear in small models trained at high learning rates (Wortsman et al., 2024).

More parameters per operation. The MLP holds two thirds of the parameters of a block (§4). A **mixture of experts** replaces it by many smaller MLPs, the experts, and a router, a linear layer that scores the experts for each token and sends the token to the few with the highest scores (Shazeer et al., 2017, between LSTM layers). The parameters grow with the

number of experts, but the work per token only with the number chosen: DeepSeek-V3 holds 671 billion parameters but uses 37 billion per token.

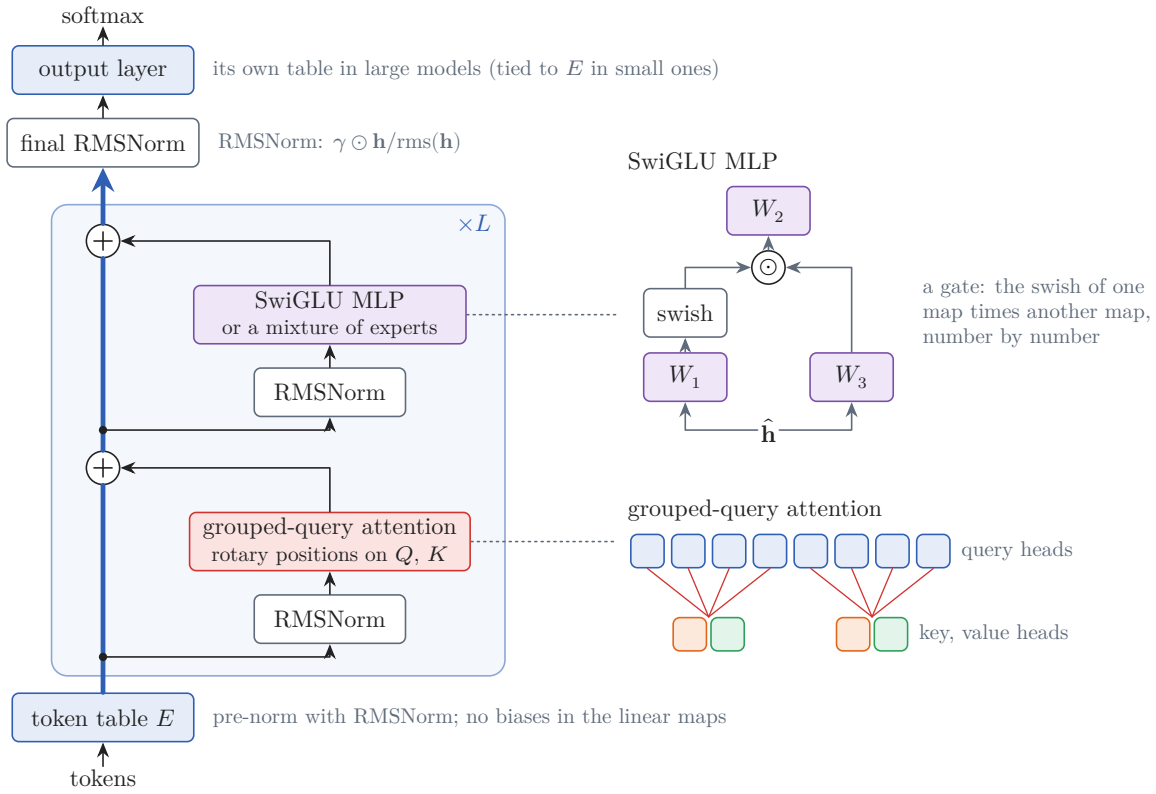


Figure 51. A block of a present-day language model. Pre-norm, with RMSNorm before each sublayer and once more at the top; grouped-query attention, in which several query heads share one key head and one value head, with rotary positions applied to the queries and the keys (§9); a gated SwiGLU MLP, or a mixture of experts; no biases in the linear maps; and in large models an output layer of its own. Sharing key and value heads shrinks the memory of keys and values that generation keeps (§10).

Positions and smaller changes. Rotary positions (§9) are nearly universal. The MLP became gated: **SwiGLU** computes $(\mathrm{swish}(\mathbf{h}W_1) \odot \mathbf{h}W_3)W_2$, where $\mathrm{swish}(z) = z\,\sigma(z)$ acts on each number and $\odot$ multiplies number by number, with a hidden width of about $\frac{8}{3}d$, which keeps the parameter count and trains slightly better (Shazeer, 2020); its author attributed its success "to divine benevolence". Large models no longer tie the output layer to the token table (§7), while small ones, for which a table of over 100,000 tokens is a large share, still do: Llama 3's models of 8 billion parameters and more keep two tables, while Llama 3.2's of 1 and 3 billion and Qwen3's up to 4 billion share one (Qwen Team, 2025). Figure 51 draws a block with these changes.

Five models show where these changes have led, read from their papers and released configurations. Two of the entries, key–value heads shared by several query heads and a compressed cache, save memory in generating text (§10). The architectural choices:

Model	**Attention**	**Normalization**	**MLP**	**Positions**
GPT-2 XL, 2019	25 heads	LayerNorm, before	GELU	learned
Llama 3.1 405B, 2024	128 query heads sharing 8 key–value heads	RMSNorm, before	SwiGLU	rotary
DeepSeek-V3, 2024	128 heads, compressed cache	RMSNorm, before	SwiGLU, experts	rotary, on part of each head
gpt-oss-120b, 2025	64 query heads, 8 key–value heads; half the blocks see only the last 128 tokens	RMSNorm, before	SwiGLU, experts	rotary
Qwen3.5, 2026	3 of every 4 blocks recurrent; the 4th with 32 query heads, 2 key–value heads	RMSNorm, before; also on queries and keys	SwiGLU, experts	rotary, on a quarter of each head

and by their sizes:

Model	**Parameters**	**Used per token**	**Blocks**	**Width**	**Experts used per token**	**Context**
GPT-2 XL, 2019	1.5B	1.5B	48	1,600		1,024
Llama 3.1 405B, 2024	405B	405B	126	16,384		131,072
DeepSeek-V3, 2024	671B	37B	61	7,168	8 of 256, and 1 shared	131,072
gpt-oss-120b, 2025	117B	5.1B	36	2,880	4 of 128	131,072
Qwen3.5, 2026	397B	17B	60	4,096	10 of 512, and 1 shared	262,144

(Sources: Radford et al., 2019; Llama Team, 2024; DeepSeek-AI, 2024b; OpenAI, 2025; Qwen Team, 2026; surveys by Raschka, 2025 and 2026.)

E.2 The recipe now

Much of the recipe of 2017 (§7) survives. Adam remains, as **AdamW**, which also shrinks every weight by a small fraction of itself at each step (weight decay), separately from the step along the gradient; the second moment's decay rate β_2 is usually 0.95 rather than 0.98; and the learning rate warms up and then decays along a cosine instead of an inverse square root. Dropout and label smoothing are gone from pretraining: PaLM was "trained without dropout" (Chowdhery et al., 2023), and the released configurations of Llama 3 and OLMo 2 set dropout to zero. Gradients are clipped, and the loss sometimes carries a small extra term that keeps the logits from drifting (Wortsman et al., 2024). The batch is counted in tokens, millions per step, and grows during training. Four published recipes:

	GPT-3 175B, 2020	**Llama 3 405B, 2024**	**OLMo 2 13B, 2025**	**DeepSeek-V3, 2024**
Optimizer	Adam, $\beta = (0.9, 0.95)$	AdamW	AdamW, $\beta = (0.9, 0.95)$	AdamW, $\beta = (0.9, 0.95)$
Weight decay	0.1	not stated	0.1	0.1
Peak learning rate	6×10^{-5}	8×10^{-5}	9×10^{-4}	2.2×10^{-4}
Warmup	375 million tokens	8,000 steps	2,000 steps	2,000 steps
Decay	cosine, to 10%	cosine, then a linear anneal to zero	cosine	constant, then cosine, then constant
Gradient clipping	1.0	not stated	1.0	1.0
Tokens per batch	3.2 million	4 to 16 million	8.4 million	12.6 to 63 million
Training tokens	300 billion	15.6 trillion	5 trillion	14.8 trillion
Context in training	2,048	4,096, then 8,192, then 128K	4,096	4,096, then 32K and 128K

(Sources: Brown et al., 2020, Appendix B; Llama Team, 2024, Section 3.4; OLMo Team, 2025, Table 3; DeepSeek-AI, 2024b, Section 4.2.)

Each of these models costs far more to train and to run than the word model of §8, and both costs follow from its weight matrices and the length of the text it reads (§10).